

nwcommands: Network Analysis in Stata

Thomas U. Grund

August 18, 2015

Contents

1	About	9
1.1	Citation	9
1.2	Contact	9
1.3	Resources	9
2	Introduction	11
2.1	General introduction	11
2.2	Existing software	12
2.3	General design and limitations	14
2.4	Installation and resources	15
2.5	Network names and network lists	18
2.6	Network expressions	20
3	Getting started	23
3.1	Set network data	23
3.1.1	Example: Set network data	24
3.1.2	Syntax for setting network data	25
3.1.3	Options for setting network data	25
3.2	Import network data	26
3.2.1	Example: Import network	26
3.2.2	Syntax for importing networks	29
3.2.3	Options for importing networks	29
3.3	Dealing with node attributes	29
3.4	Export networks	30
3.4.1	Syntax for exporting networks	31
3.4.2	Options for importing networks	31
3.5	Save and use networks	31
3.5.1	Example: Use network data from the internet	31
3.6	Load a network as Stata variables	32
3.6.1	Example: Load network as Stata variables	33
3.6.2	Syntax for loading network as Stata variables	34
3.6.3	Options for loading network as Stata variables	34
3.7	Display tie values of a network	34
3.7.1	Example: Network tie values	35
3.7.2	Syntax for returning tie values	35

<i>CONTENTS</i>	3
3.8 Obtaining meta-information	36
3.8.1 Syntax for obtaining/changing meta-information	37
3.8.2 Options for obtaining/changing meta-information	37
4 Characterize networks	39
4.1 Summarize a network	39
4.1.1 Example: Network summary	39
4.1.2 Syntax for network summary	40
4.1.3 Options for network summary	40
4.2 Dyad and triad census	40
4.2.1 Syntax for dyad and triad census	41
5 Manipulate networks	43
5.1 Drop and keep networks	43
5.1.1 Syntax for dropping/keeping networks or nodes	45
5.1.2 Options for dropping/keeping network or nodes	45
5.2 Add nodes to a network	46
5.2.1 Syntax for adding nodes	46
5.2.2 Options for adding nodes	46
5.3 Collapse nodes of a network	46
5.3.1 Syntax for collapsing nodes	46
5.3.2 Options for collapsing nodes	47
5.4 Replace and recode networks	47
5.4.1 Syntax for replacing networks	47
5.4.2 Assign constants to dyads	47
5.4.3 Assign values from other networks to dyads	48
5.4.4 Assign values with if-conditions	49
5.4.5 Recode tie values	50
5.4.6 Example: Recode tie values	50
5.4.7 Syntax for recoding networks	50
5.4.8 Options for recoding networks	50
6 Generate networks	51
6.1 Network generators	51
6.1.1 Random network	51
6.1.2 Ring-lattice network	54
6.1.3 Lattice network	55
6.1.4 Small-world network	56
6.1.5 Preferential-attachment network	58
6.1.6 Network with tie probabilities	60
6.2 Generate networks with network expressions	62
6.3 Generate networks with network functions	64
7 Expansion and permutation	67
7.1 Expansion of variable to network	67
7.1.1 Example: Expand variable to network	67

7.1.2	Syntax for network permutation	69
7.1.3	Options for network permutation	69
7.2	Permutation of a network	69
7.2.1	Example: Network permutation	70
7.2.2	Syntax for network permutation	71
7.2.3	Options for network permutation	71
8	Hypothesis testing	73
9	Tabulate networks	77
9.1	One-way tabulate of network	77
9.1.1	Syntax for one-way tabulating networks	77
9.1.2	Options for one-way tabulating networks	77
9.2	Two-way tabulate of two networks	78
9.2.1	Syntax for two-way tabulating networks	78
9.2.2	Options for two-way tabulating networks	78
9.3	Two-way tabulate of network and variable	78
9.3.1	Syntax for two-way tabulating network and variable	80
9.3.2	Options for two-way tabulating networks	80
10	Correlation with networks	81
10.1	Correlate nodes with each other	81
10.1.1	Example: Correlate nodes with each other	81
10.2	Correlate two networks	82
10.2.1	Example: Correlate two networks	82
10.3	Correlate network with a variable	84
10.3.1	Example: Correlate network with a variable	84
10.4	Syntax for correlating nodes	87
10.5	Syntax for correlating two networks	87
10.6	Syntax for correlating network and variable	87
10.7	Options for correlation with networks	87
11	Dissimilarities and similarities of nodes	89
11.1	Dissimilarities of nodes	89
11.1.1	Euclidean distance	89
11.1.2	Manhattan distance	90
11.1.3	Hamming distance	90
11.1.4	Jaccard distance	90
11.1.5	Nonmatches distance	90
11.1.6	Syntax for node dissimilarities	90
11.1.7	Options for node dissimilarities	91
12	Network neighbors	93
12.1	Network neighbors and context information	93
12.2	Example: Obtain network neighbors	94
12.2.1	Syntax for obtaining network neighbors	94

12.2.2 Options for obtaining network neighbors	95
12.3 Example: Calculate context variable	95
12.3.1 Syntax for calculating context variables	97
12.3.2 Options for calculating context variables	97
13 Paths between nodes	99
13.1 Paths and shortest paths	99
13.1.1 Syntax for paths between nodes	101
13.1.2 Options for paths between nodes	101
13.2 All shortest paths	101
13.2.1 Syntax for all shortest paths	102
13.2.2 Options for all shortest paths	102
14 Node centrality	103
14.1 Degree centrality	103
14.1.1 Example: Degree centrality	104
14.1.2 Syntax for degree centrality	105
14.1.3 Options for degree centrality	105
14.2 Betweenness centrality	105
14.2.1 Example: Betweenness centrality	106
14.2.2 Syntax for betweenness centrality	107
14.2.3 Options for betweenness centrality	107
14.3 Eigenvector centrality	107
14.3.1 Example: Eigenvector centrality	107
14.3.2 Syntax for eigenvector centrality	108
14.3.3 Options for eigenvector centrality	108
14.4 Closeness centrality	109
14.4.1 Example: Closeness centrality	109
14.4.2 Syntax for closeness centrality	110
14.4.3 Options for closeness centrality	110
15 Node-level characteristics	111
15.1 Clustering coefficient	111
15.1.1 Example: Clustering coefficient	112
15.1.2 Syntax for clustering coefficient	113
15.1.3 Options for clustering coefficient	113
15.2 Component membership	113
15.2.1 Example for network components	114
15.2.2 Syntax for network components	115
15.2.3 Options for network components	115
16 Visualization of networks	117
16.1 Schemes for networks	117
16.2 Plot networks	117
16.2.1 Syntax for plotting networks	119
16.2.2 Example: Plotting networks	119

16.3	Plot networks as sociomatrix	125
16.3.1	Example: Plot networks as sociomatrix	125
16.4	Animate network	131
17	Advanced programming with networks	133
17.1	Getting the adjacency matrix	133
17.1.1	Syntax for obtaining adjacency matrix	134
17.1.2	Options for obtaining adjacency matrix	135
17.2	Unabbreviate network	135
17.2.1	Example: Unabbreviate networks	135
17.2.2	Syntax for unabbreviating networks	135
17.2.3	Options for unabbreviating networks	135
17.3	Parsing networks	136
17.3.1	Syntax for parsing networks	136
17.3.2	Options for parsing networks	136
17.4	Example: Own network program	136
17.5	Example: Another network program	137

List of Figures

2.1	Simple network	12
2.2	User menu for network analysis	16
2.3	Manual for the nwcommands	17
3.1	Simple network	25
3.2	Pajek NET format, <i>pajek_example.net</i>	26
3.3	Compressed edgelist format, <i>compressed_example.txt</i>	27
3.4	Edgelist format, <i>edgelist_example.txt</i>	27
3.5	Another simple network	28
3.6	Adjacency matrix with attributes, <i>excel_example.xlsx</i>	28
4.1	Triad census	42
6.1	One-dimensional lattice	55
6.2	Two-dimensional lattice	56
6.3	Small-world network	57
6.4	Preferential-attachment network	59
7.1	Permutation of a network	71

8.1	Distribution of transitivity in random networks	75
9.1	EI-index plot	80
10.1	Correlation between "flomarriage" and "flobusiness"	84
10.2	Correlation between "flomarriage" and Birthplace	86
15.1	Clustering coefficient	112
16.1	s1network scheme	118
16.2	s2network scheme	118
16.3	s3network scheme	119
16.4	Florentine marriage network	122
16.5	Florentine marriage network, wealth and seat attributes	123
16.6	Florentine marriage network, only families with seat	123
16.7	Florentine marriage network, edges colored by business network	124
16.8	Florentine marriage network as matrix-plot	127
16.9	Florentine marriage network as matrix-plot, s2network scheme	128
16.10	Florentine marriage network as matrix-plot, sorted by wealth	128
16.11	Florentine marriage network as matrix-plot, grouped by seat	129
16.12	Florentine marriage network as matrix-plot, overwrite colors	129
16.13	Florentine marriage network as matrix-plot, tievalue	130
16.14	Gang network, valued	130
17.1	Own network program	137
17.2	Another network program	138

List of Tables

2.1	Adjacency matrix for simple network	13
2.2	Adjacency list for simple network	13
2.3	Stata commands and network equivalents	15
2.4	The four core packages of the nwcommands	16
2.5	General Stata concepts and network equivalents	18
2.6	Examples for network lists	19
3.1	Supported network file-formats	29
3.2	Suboptions for nwimport	29
3.3	Suboptions for nwexport	31
3.4	Network example datasets on the nwcommands server	32

3.5	Variables produced by <code>nwload</code>	33
3.6	Examples for network subsetting	35
6.1	Options for all network generators	52
7.1	Valid expand modes	69
11.1	Types of dissimilarities	91
11.2	Context for dissimilarities	91
12.1	Definition of network neighborhood	96
12.2	Statistic for combining values of network neighborhood	96
16.1	Node options for <code>nwplot</code>	120
16.2	Node suboptions for <code>nwplot</code>	120
16.3	Label options for <code>nwplot</code>	120
16.4	Edge options for <code>nwplot</code>	120
16.5	Edge suboptions for <code>nwplot</code>	120
16.6	Arrow options for <code>nwplot</code>	121
16.7	Arc-styles for <code>nwplot</code>	121
16.8	Layout options for <code>nwplot</code>	121
16.9	Layout styles for <code>nwplot</code>	121
16.10	Layout suboptions for <code>nwplot</code>	121
16.11	Other options for <code>nwplot</code>	122
16.12	Label options for <code>nwplotmatrix</code>	126
16.13	Patch options for <code>nwplotmatrix</code>	126

Chapter 1

About

1.1 Citation

This manuscript is a first draft for a book on network analysis using Stata. Please do not distribute this draft. If you want to cite the software already, please use the following citation:

Grund, Thomas U. (2015) "nwcommands. Network Analysis in Stata". Retrieved from <http://nwcommands.org>

1.2 Contact

Thomas U. Grund
Institute for Analytical Sociology
Linköping University
Sweden

thomas.u.grund@gmail.com
<http://www.grund.co.uk>

1.3 Resources

Website: <http://nwcommands.org>
Email list/forum: <https://groups.google.com/forum/#!forum/nwcommands>
GitHub: <https://github.com/ThomasGrund/nwcommands>

Chapter 2

Introduction

2.1 General introduction

The field of network analysis is one of the most rapidly growing fields of the social sciences. Social network analysis focuses on the relationships that exist between individuals (or other units of analysis) such as friendship, advice, trust, or trade relationships. But network analysis is also popular in other disciplines, such as biology, physics, and many more. This manuscript introduces the *nwcommands*, a collection of over 90 commands for network analysis in Stata. It includes commands for importing, exporting, loading, saving, handling, manipulating, replacing, generating, visualizing, and animating networks. There are also commands for measuring various properties of networks and individual nodes, for detecting network patterns, for measuring the similarity of networks and for advanced statistical analyses of networks. Furthermore, the software comes with dialog boxes for almost all commands and tools for programmers to write extensions of the *nwcommands*.

Network analysis is becoming more and more popular in the social sciences. But also in other disciplines, such as epidemiology, biology, or physics, scholars increasingly draw on network analysis to answer substantive research questions. In sociology, for example, researchers are interested in friendship, advice, trust or trade networks (and many more). Questions include are relationships between some actors more likely than between others, who are the "important" actors, or how far away are two actors from each other in the network?

The basic unit of analysis in networks is the dyad. A dyad is a pair of nodes or actors, which can be persons, organizations or other units. Mathematically, a network is defined as the graph $G = (V, E)$, where $V = \{1, 2, 3, \dots, n\}$ is a set of vertices, or nodes, and $E \subseteq \{\langle i, j \rangle | i, j \in V\}$ is a set of edges, also referred to as ties or arcs (for directed networks). Edges are pairs of vertices, e.g. $E = \{\langle 1, 3 \rangle, \langle 2, 3 \rangle\}$.¹ In valued networks, edges are defined as $E \subseteq \{\langle i, j, k \rangle | i, j \in V\}$, where k indicates the value of a relationship.

¹Notice that theoretically an edge is not limited to two vertices. In so-called hypergraphs an edge can connect any number of vertices. In practice, however, edges almost always refer to pairs of vertices.

A useful representation of a network is the *adjacency matrix*. A network with n nodes is stored as the $n \times n$ matrix Y , where $Y_{ij} = 0$ if the nodes i and j are unrelated to each other and $Y_{ij} \neq 0$ if there is a relationship between the two nodes. Often, one deals with binary networks where ties either exist ($Y_{ij} = 1$) or do not exist ($Y_{ij} = 0$). A network can be undirected, then, relationships have no directionality, e.g. two families are related with each other by marriage. In this case, the adjacency matrix is symmetric $Y_{ij} = Y_{ji}$. In other cases, however, networks are directed and it makes sense to speak of one node sending a tie to somebody else, e.g. one family owes another family money. When plotting networks, one signals the directionality of relationships by connecting nodes with arrows instead of lines. Notice that in directed networks relationships do not have to be (but can be) reciprocal.

The adjacency matrix of the simple network displayed in Figure 2.1 is given by the adjacency matrix Y in Table 2.1. For example, the value 0 in the first row and second column means that node 1 is unconnected to node 2. In contrast, the value 1 in the third row/first column means that node 3 sends a tie to node 1. Another representation of a network is the *adjacency list*. In this notation only the existing relationships are stored. Two columns represent the sender (also called *ego*) and the receiver (also referred to as *alter*) of network ties. Table 2.2 also shows the simple network displayed in Figure 2.1. Here, the first line entry means that node 1 sends a tie to node 4.

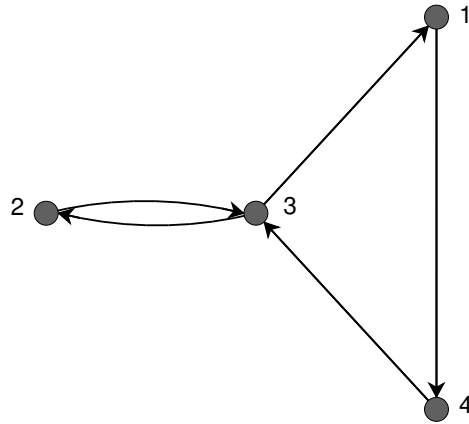


Figure 2.1: Simple network

2.2 Existing software

Scholars use all sorts of software for network analyses. Some of the most popular software are Pajek (Batagelj and Mrvar, 1998; de Nooy et al., 2005), Ucinet (Borgatti et al., 2002) or Gephi (Bastian et al., 2009). All three of them are purpose-built

	1	2	3	4
1	0	0	0	1
2	0	0	1	0
3	1	1	0	0
4	1	0	0	0

Table 2.1: Adjacency matrix for simple network

ego	alter
1	4
2	3
3	1
3	2
4	3

Table 2.2: Adjacency list for simple network

and have graphical user interfaces. Some other popular software include the Excel-plugin NodeXL (Hansen et al., 2010), the Python packages networkX (Hagberg et al., 2008) and networkKit (Staudt et al., 2014) and several packages in R, including *statnet* (Handcock et al., 2003) or *igraph* (Csardi and Nepusz, 2006), which also have large user-communities.

So far, only few user-written commands exist for network analysis in Stata. For example, `stata2pajek` (Rossman, 2009) generates a Pajek .net file from edgelist data stored in two variables in Stata; `pajek2stata` (Corten, 2010) produces edgelist data from a Pajek .net file. `centpow` (Neal, 2012) calculates centrality and power indices for each node in a network. It requires as input a comma-delimited file containing a square, symmetric matrix representing the network. Another user-written command is `netsis` (Miura, 2012), which computes betweenness centrality, distances, and clustering. Data needs to be given edgelist format, where two variables indicate the sender and receiver of network ties. `netplot` (Corten, 2011) displays networks in edgelist/arclist format. It allows the addition of node labels or arrowheads, but it is unable to plot nodes in different sizes, colors or symbols. Furthermore, relationships are displayed as simple lines between nodes and cannot be plotted with different colors, patterns, or thickness. All these attempts add important network capabilities to Stata, but a unifying framework and standard that allows handling and dealing with networks straightforwardly is still missing. Furthermore, the types of analyses that can be done with these commands are quite basic and more advanced features are missing. There are also no interfaces or the general infrastructure that would facilitate simple additions and user-written commands. So far, everybody starts from scratch and has to implement even the simplest data structures for dealing with networks. Compared to other existing software, Stata is clearly lagging behind when it comes to network analysis. This is especially intriguing because Stata (Mata) already provides the matrix computation facilities needed for sophisticated network analysis.

2.3 General design and limitations

The current software attempts to provide a unifying framework for network analysis in Stata that goes beyond purpose-built commands for specific tasks, e.g. calculating centrality scores or plotting a network. It does so by introducing a base layer for network analysis, which takes care of saving, handling, storing, loading, and manipulating networks. There are commands and interfaces to interact with networks. Providing such commands and interfaces has the additional advantages that the internal structure for dealing with networks can be optimized in future updates, e.g. with the respect to very large networks, without changes in the external design or programs that draw on these interfaces. The software introduces a standard, which hopefully, encourages other users to develop network commands on this platform. On the top of this base layer, the software comes with lots of commands for network analysis that can be used straightforwardly and extends all previous commands for network analysis in Stata.

The software provides the tools to deal with networks as if they were normal Stata variables. Such an approach has lots of advantages. For example, one deals with networks directly and does not have to interact with Stata variables that represent an adjacency matrix or an adjacency list. Additionally, one can parse networks or lists of networks, just like one parses variables or lists of variables.

The *nwcommands* are designed in such a way that commands resemble standard Stata syntax and intuition. Almost all *nwcommands* begin with the prefix **nw**. Often, this is followed by a familiar Stata command, e.g. **nwclear**, **nwdrop**, **nwkeep**, **nwreplace**. These commands behave very similar to their Stata counterparts, but refer to networks instead of variables (see Table 2.3).

Obviously, the software also has some limitations. Originally, it was developed for small and medium-sized networks (< 1000 nodes). And all commands are tested for networks of such size. But many commands work for larger networks as well. Most network calculations are performed in Mata, which should make the software reasonably fast. Currently, the software uses a combination of Stata global macros and Mata matrices to store network quasi-objects. For example, for each network one global macro stores the labels of nodes as a single string. But macros in Stata cannot exceed 67,784 characters, which is the major limiting factor in the current implementation. Future updates will resolve this limitation, without changing the interfaces that the software introduces. Furthermore, the actual network data is stored as adjacency matrix in Mata. Although this data structure facilitates fast calculations it becomes inefficient for very large and sparse networks. Many *nwcommands* draw on existing commands in Stata, which can be another limiting factor. For example, in order to unabbreviate network lists with **nwunab** in more or less the same way as one unabbreviates variable lists, **nwunab** first generates temporary dummy variables for each network and then executes the normal **unab** command. Such a procedure is elegant as it allows passing on options to standard Stata commands, but also becomes inefficient for many networks because generating or replacing Stata variables is time-intensive. In a similar way, for example, **nwplot** visualizes networks and, by default, draws on multidimensional scaling (**mdsmat**) to calculate the coordinates of network nodes. Plotting larger networks can therefore be slow because multidimensional scaling is slow.

Stata command	nwcommand	Description
stset	nwset	Set network data
webuse	webnwuse	Use network data from the internet
use	nwuse	Use network data in .dta format and initialize networks
save	nwsave	Save network data in .dta format
clear	nwclear	Clear all network data from memory
keep	nwkeep	Keep specific networks
drop	nwdrop	Drop specific networks
generate	nwgenerate	Generate a network from existing networks
egen	nwgen	Advanced network generation
replace	nwreplace	Replace a network with the result of a network expression
recode	nwrecode	Recode the tie values of a network
summarize	nwsummarize	Produce summary statistics for a network
unab	nwunab	Unabbreviate a network list
tabulate	nwtabulate	Tabulate networks (one-way and two-way tabulate)
correlate	nwcorrelate	Correlate networks with each other
expand	nwexpand	Expand variables to a network
collapse	nwcollapse	Collapse network nodes
syntax	_nwsyntax	Parse syntax including networks

Table 2.3: Stata commands and network equivalents

2.4 Installation and resources

Currently, the *nwcommands* consist of 268 different files, including 92 Stata commands, various help-files, scheme-files and dialog-boxes. Due to its large size the software is split-up in four packages (see Table 2.4).² Other extension packages, e.g. for hierarchical clustering of network nodes, blockmodelling and structural equivalence are in preparation. The installation of the *nwcommands* is very straightforward. The only package that needs to be manually installed is *nwcommands-ado*. A simple search in Stata finds the package (`search nwcommands`). Alternatively, one can install the package directly from the *nwcommands* Server.³

```
. net from "http://nwcommands.org"
. net install "nwcommands-ado"
. nwinstall, all

. help nwcommands
```

After installation of the *nwcommands-ado* package, all dependencies are installed with `nwinstall, all`. Installation via this route has several advantages: 1) `nwinstall` keeps track of developments and installs all released extensions. 2) A new menu item

²For historical reasons, a Stata package can only have a maximum number of files (not lines of code). This makes it necessary to split-up the software in different packages.

³<http://nwcommands.org>

Package	Description
<i>nwcommands-ado</i>	Ado-files for network analysis
<i>nwcommands-hlp</i>	Help-files for the core commands
<i>nwcommands-dlg</i>	Dialog-boxes for the core commands
<i>nwcommands-ext</i>	Extensions for network analysis

Table 2.4: The four core packages of the nwcommands

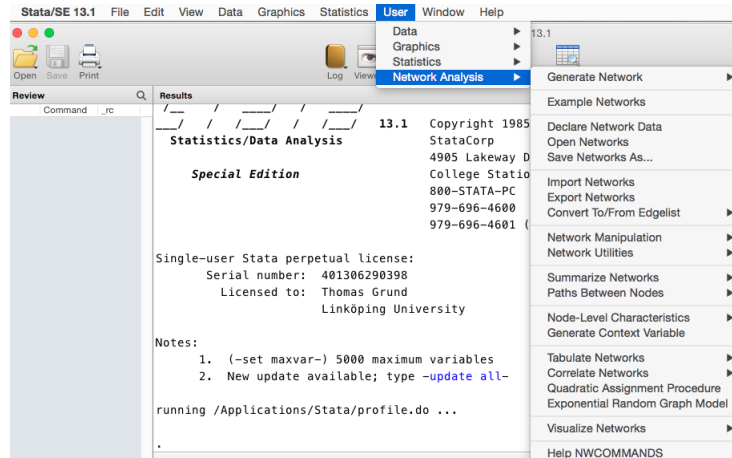


Figure 2.2: User menu for network analysis

(called *Network Analysis*) shows up under the *User* menu and gives access to dialog-boxes for almost all *nwcommands* (see Figure 2.2). These dialog-boxes make network analysis (and teaching network analysis to students) in Stata even easier. Users can click their way through (e.g. like in Ucinet) and learn about functionalities/options of the *nwcommands*. At the same time, users can program and calculate with networks (e.g. like in R). 3) The line `nwinstall, usermenu` is added to the *profile.do*.⁴ This makes the *Network Analysis* menu automatically appear every time Stata starts. The software is under constant development. Packages can be normally updated, e.g. with `adoupdate, update all`, like any other package.

After successful installation, a good place to start is looking at the *nwcommands*-manual (Figure 2.3). This manual includes a general introduction to network analysis in Stata [NW-1], a topical [NW-2] and alphabetical list of available network commands [NW-3]. Furthermore, there is a section on how to get started [NW-4], as well as a more advanced section on how to use the *nwcommands* to write own network routines and commands [NW-5]. Another valuable resource is the website <http://nwcommands.org>. On this website, you can find more materials, slides, tutorials and videos around network analysis in Stata. There are also instructions for how to sign-up to the *nwcommands* email list and forum.⁵ This list is a great place to ask questions and stay informed about latest updates and workshops on social network analysis using Stata. There is a growing community of *nwcommands*-users who already signed-up during the beta-testing of the software.

Section	Description
[NW-1]	Introduction and concepts
[NW-2]	Topical list of network commands
[NW-3]	Alphabetical list of network commands
[NW-4]	Getting started
[NW-5]	Network programming
[NW-6]	Install Stata menus/dialogs
*! Date	: 28april2015
*! Version	: 1.3.1
*! Authors	: Thomas U. Grund
*! Contact	: thomas.u.grund@gmail.com
*! Web	: http://nwcommands.org
*! Bugs	: mailto:bug@nwcommands.org

Figure 2.3: Manual for the *nwcommands*

⁴The *profile.do* is a do-file that Stata automatically runs at start-up. In case no *profile.do* is found, the command attempts to create a new *profile.do* in the Stata folder and adds the relevant command.

⁵The list is set-up as a Google-group, which let's you choose between receiving emails, receiving email digests, or using the group like a normal forum.

2.5 Network names and network lists

A general design feature of the *nwcommands* is that the software not only includes specialized commands for certain routines and algorithms for network analyses, but also the general infrastructure to handle and deal with networks in Stata. It provides basic functionality that can be used in further developments and user-written network routines. This makes the software comparable to the R-packages *network* (Butts et al., 2014; Butts, 2008a) and *sna* (Butts, 2014, 2008b).

Stata	nwcommand	Description
<i>varname</i>	<i>netname</i>	Unique name of a network
<i>newvarname</i>	<i>newnetname</i>	Unique name for a new network
<i>varlist</i>	<i>netlist</i>	List of networks
<i>exp</i>	<i>netexp</i>	Network expression

Table 2.5: General Stata concepts and network equivalents

Most importantly, the software introduces the concept of a *netname*, which is very similar to a *varname*. A *netname* is a unique name that identifies a network in Stata memory. Network names may be 1 to 32 characters long and must start with a-z, A-Z, or `_`, and the remaining characters may be a-z, A-Z, `_`, or 0-9. Whenever one wants to refer to a specific network, for example when calculating some sort of network statistic, one uses the *netname* of a network in exactly the same way as when using the name of a Stata variable. This makes handling, generating, dropping, keeping, replacing or recoding networks extremely easy. Internally, a *netname* is saved in a global macro, which points to additional meta-information (stored in other global macros) and to the actual network data saved in a Mata matrix. One does not have to worry about how or where the networks are stored. They just exist as quasi-objects in Stata memory and can be accessed through Stata-like syntax like normal variables. I recommend interacting with networks and changing meta-information only through the provided *nwcommands* (only advanced users should change the relevant macros or the underlying Mata matrix directly).⁶ But one can also directly interact with the underlying Mata matrix that stores a network.⁷

Furthermore, the *nwcommands* introduce the concept of a *netlist*. Again, this mirrors the concept of a *varlist* and usually refers to existing networks. A *netlist* is a list of network names and can also hold just a single network. Almost all special characters that one knows from variable lists can be used as well. Table 2.5 shows examples for valid network lists. The `*` character indicates to match one or more characters. All networks matching the pattern are returned. The `~` character also indicates to match

⁶The reason is that in future updates the internal design and structure might change. However, the external design and the interfaces for the way one deals with networks through the *nwcommands* will remain the same.

⁷The command `nwtomata` returns a copy of the underlying adjacency matrix. It is useful when programming own network commands. The command `nwtomatafast` returns the actual name of the Mata matrix in the return vector.

one or more characters, but unlike `*`, only one network is allowed to match. If more than one network matches, an error message is presented. The `?` character matches one character. All networks matching the pattern are returned. The `-` character indicates that all networks in the dataset, starting with the network to the left of the `-` and ending with the network to the right of the `-` are to be returned. The keyword `_all` stands for all networks in memory.

Example	Description
<code>mynet</code>	Just one network
<code>mynet1 mynet2</code>	Two networks
<code>mynet*</code>	All networks starting with <code>mynet</code>
<code>*net</code>	All networks ending with <code>net</code>
<code>my*t</code>	All networks starting with <code>my</code> and ending with <code>t</code>
<code>my~t</code>	One network starting with <code>my</code> and ending with <code>t</code>
<code>my?t</code>	All networks starting with <code>my</code> and ending with <code>t</code> and one character in between
<code>mynet1-mynet6</code>	<code>mynet1</code> , <code>mynet2</code> , ..., <code>mynet6</code>
<code>_all</code>	All networks in memory

Table 2.6: Examples for network lists

Introducing the concepts of a *netname* and a *netlist* provides an elegant way to deal with networks in almost the same way as one deals with variables. It also overcomes the difficulties of previous network extensions in Stata concerning multiple networks. Most *nwcommands* require a *netname* or a *netlist* to perform some sort of operation.⁸ Furthermore, the software recognizes when a user tries to refer to a network that does not exist (just in the same way as when one tries to perform an operation on a variable that does not exist). Internally, this is implemented with the `_nwsyntax` command, which parses input in a similar way as `syntax`, but with respect to networks (and can be used for advanced programming of network routines).

Each node in a network has a *nodeid* and a *nodeiab*. A *nodeid* is an integer number used to identify nodes in a network. Every node in a network is automatically assigned a *nodeid*. The range of these numbers always goes from 1 to size of the network, where every integer number identifies exactly one node in a network. A *nodeid* corresponds to the observation number `_n`. When a node is deleted, the *nodeids* of all other nodes are automatically changed so that all consecutive numbers from 1 to size of the network correspond to one node. When attributes (saved as Stata variables) are used, the variable value of row *i* corresponds to the attribute of the node with *nodeid* == *i*. For example, any normal Stata variable, e.g. `size`, can be used to plot network nodes with different sizes. In this case, the information in `size[i]` will be used to determine the size of the node with *nodeid* == *i*. One can change the order of nodes within a network with `nwsort`. This does not change the structure of the network, but just changes the

⁸Sometimes a *netname* is optional. In such a case, the program uses the *current network*, which is essentially the last network that has been set (or loaded). This is simply for convenience (see `nwcurrent`).

nodeids that are attached to the nodes (just like when using `sort` to change the order of observations).⁹ A *nodelab* is more flexible than a *nodeid* and attaches a unique word to each node for identification. By default, these node labels are used when node-level information is displayed or when a network is plotted with node labels.¹⁰ One can easily find out the *nodelab* of a node in a network with a specific *nodeid* with the command `_nwnodelab`. The other way around one can find out the *nodeid* of a node with a specific *nodelab* with `_nwnodeid`. Lastly, each node holds a *nodevar*, which is the name for a corresponding Stata variable. This meta-information is used when the adjacency matrix of a network is "loaded" as Stata variables (see `nwload` and Section 3.6).

2.6 Network expressions

Another general concept this software introduces are *network expressions*. Basically, a *network expression* is very similar to a normal *expression* in Stata, but extends to networks. Whenever an expression is evaluated, some sort of calculation occurs. In normal Stata, an expression returns a single number, e.g. $5 + 4 = 9$, or a variable, e.g. `gen myvar = uniform() + 1`. An expression in Stata can include constants, operators, variables and functions. In contrast, *network expressions* always return a network. Furthermore, the *netnames* of existing networks can be used inside a *network expression* together with operators, constants, variables and functions. Together this allows very complicated expressions and calculations with networks.

A *network expression* evaluates in the following way. First, it detects the dimensions $n \times n$ of the smallest network inside the expression. Second, all normal constants and variables are expanded to the dimensions of this smallest network. In the case of a constant, a temporary $n \times n$ matrix is generated, where each matrix cell gets the value of the constant. In the case of a variable, by default, a temporary $n \times n$ matrix is generated, where each column of the matrix has the values of the first n observations of the variable (see also `nwexpand`). For normal networks, the underlying adjacency matrix of the network is used. This leaves an expression with only matrices and operators and functions. After that, each operator and function is executed on an element-by-element basis for the underlying matrices. The result is a $n \times n$ matrix, which is returned as a network.

Like for normal expressions in Stata there are two main uses for *network expressions*. They can be used in *if* conditions that accept network expressions (e.g. together with `nwreplace`) or when creating new networks `=netexp`. Generating networks with *network expressions* works almost exactly like generating new variables. Instead of `generate` or `egen`, one uses `nwgenerate` or `nwgen`. For example, assuming there are two networks called "first" and "second" in memory, I can create a new network "third"

⁹For most network commands this order of nodes within a network does not matter. An exception is `nwplotmatrix`, which displays a sociomatrix of a network and puts nodes with lower *nodeid* at the top.

¹⁰A *nodelab* can only be a single word and when the users tries to attach a multi-word *nodelab* to a node it is transformed with the `strtoname()` function. But one can also plot networks with labels stored in another variable, e.g. `nwplot mynet, labels(mynodelabels)`. In this case, multi-word labels are allowed. See more on plotting networks in section 16.2.

with `nwgen third = first + second`. This little command evaluates the *network expression* `first + second`. Basically, it adds the values of the underlying adjacency matrices of the two networks together on an element-by-element basis. This temporary matrix is then used to initialise the new network "third". Expressions like this can be extremely powerful (see more on this in Section 6.2). The second main use for *network expressions* is together with *if* conditions. Just like a normal expression can be used together with `replace`, a network expression can be used together with `nwreplace` to limit changes to a certain set of ties (see more on this in Section 5.4).

Chapter 3

Getting started

3.1 Set network data

One of the first things when working with relational data in Stata is to "set" the network data with `nwset`. After this initial step, networks are available in Stata memory and can be referenced by their *netname*. Such a declaration of data as network data works in a similar way as when one sets data as e.g. time-series (`stset`) or panel data (`xtset`). Notice that many networks can exist in Stata memory at the same time. Technically, setting network data generates a quasi-object for each network that consists of 1) several global macros that hold meta-data of a network and 2) the adjacency matrix of a network in Mata. Every command which generates (see e.g. `nwgen`), imports (see `nwimport`) or uses (see `nwuse`, `webnwuse`) a network, implicitly sets network data in this way.

But one can also set network data explicitly with `nwset`. This is useful when one enters network data directly in Stata for the first time (without having saved it in another file-format before) or when one wants to import data from a format that is not supported yet. In essence, one just needs to transform/reshape data in such way that it can be understood by `nwset`. There are two general ways how network data can be set: 1) call `nwset` followed by a *varlist* or 2) call `nwset` with the option `mat(matamatrix)`. In the first case, the variables in *varlist* are interpreted either as columns of an adjacency matrix or as columns representing the origin and destination nodes of an edgelist (when option `edgelist` is specified).¹ In the second case, a network is initialized from an adjacency matrix in Mata. Such a matrix can be created on-the-fly or with some other Mata code before. Isolates (nodes that are not connected to any other nodes) are indicated by setting the appropriate cell entries to zero when setting an adjacency matrix or, in the case of an edgelist, by setting the second variable (representing the destination node) to missing.

¹Notice that one can also set an edgelist with three variables. In this case, the third variable denotes the value of a tie.

3.1.1 Example: Set network data

The following example demonstrates all possibilities to set network data explicitly. First, I load the *example1.dta* dataset from the nwcommands server², which contains data for the network displayed in Figure 3.1. The first three variables in this dataset (*v1*, *v2*, *v3*) constitute the adjacency matrix of a network with three nodes, where e.g. *v2*[1] = 1 means that node 1 sends a tie to node 2. The variables *from* and *to* store exactly the same network, but in edgelist format. Here, each case stands for one (directed) tie. For example, the values *from*[1] = 1 and *to*[1] = 2 also indicate that node 1 sends a tie to node 2. A third variable can be specified for the value of a tie; when only two variables are specified the command produces a binary network. By default, *nwset* recognizes when a network is symmetric ($Y_{ij} = Y_{ji}$, for all nodes i and j) and sets such a network as undirected.³ But one can also overwrite this automatic detection and explicitly set a network as directed or undirected. In the latter case, the network is automatically symmetrized (see *nwsym*).

```
. use http://nwcommands.org/data/example1.dta
. list _all
```

	v1	v2	v3	from	to
1.	0	1	0	1	2
2.	0	0	1	2	3
3.	0	1	0	3	2

The following three commands all "set" the network displayed in Figure 3.1. The first alternative interprets the variables *v1*, *v2*, *v3* as adjacency matrix⁴, the second uses a (Mata) adjacency matrix on-the-fly and the last alternative interprets the data stored in the variables *from* and *to* as an edgelist. Notice that the edgelist format is also the one used by previous network commands such as *netplot* or *networksis*.⁵ After executing this code, there are three network objects in Stata memory. Although all have exactly the same structure, they are different networks and are stored separately.

```
. nwset v1 v2 v3
. nwset, mat((0,1,0\0,0,1\0,1,0))
. nwset from to, edgelist
```

Whenever a network is "set" it gets a unique *netname*. By default, *nwset* tries to use the name "network" unless the option *name(newnetname)* is specified. When a network name already exists, the command automatically comes up with an alternative

²<http://nwcommands.org>

³Internally, this sets some meta-information e.g. `global nwdirected_1 = "false"`. Subsequent commands execute differently depending on this meta-information. For example, *nwplot* automatically displays arrowheads for directed, but not for undirected networks (where it does not make sense to indicate the directionality of ties).

⁴Notice that writing *nwset v** would work as well.

⁵In fact, one can easily go back and forth from this edgelist format to the nwcommands representation of networks as quasi-objects with *nwfromedge* (same as *nwset, edgelist*) and *nwtoedge*, which generates an edgelist from a network object that can be used with e.g. *netplot*.

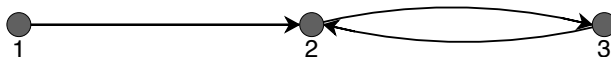


Figure 3.1: Simple network

by adding a number to the desired name (e.g. "network_1" or "network_2").⁶ In our example, without any other networks in Stata memory, the three networks are called: "network", "network_1" and "network_2". Other network generators (programs that produce different network structures, e.g. random network, small-world network) use different default names.

At any moment, one can get a simple list of all networks in Stata memory with `nwds`. In fact, it works almost exactly like the Stata command `ds` (and even accepts most options one can normally use with `ds`). Another way to get a list of all networks in memory is typing `nwset` without anything else. Both commands save a list of networks in the return vector as well. Notice that these commands can also be used to unabbreviate a *netlist* (e.g `nwds net*`; see also `nwunab`).

```
. nwds
network    network_1  network_2

. nwset
(3 networks)
-----
      network
      network_1
      network_2
```

3.1.2 Syntax for setting network data

```
nwset varlist[, edgelist directed undirected name(newnetname) labs(string)
      labsfromvar(varname) vars(string) keeporiginal xvars]
```

```
nwset, mat(matamatrix) [directed undirected name(newnetname) labs(string)
      labsfromvar(varname) vars(string) xvars]
```

3.1.3 Options for setting network data

`edgelist` interprets the first three variables in *varlist* as edgelist data.

`directed` explicitly sets network as a directed network.

`undirected` explicitly sets network as a undirected network.

`name(newnetname)` assigns a specific *netname* to the network.

⁶The command `nwvalidate` checks whether a specific *netname* already exists and, if yes, suggests an alternative *netname*.

`labs(string)` explicitly assigns node labels.
`labsfromvar(varname)` generates node labels from data stored in *varname*.
`xvars` does not generate Stata variables for adjacency matrix.
`keeporiginal` generates a variable `_nodeoriginal` with original node id's (when setting from an edgelist).

3.2 Import network data

Alternatively, one can also import networks saved in other network-formats. Currently, `nwimport` supports Ucinet DL, Pajek NET, GML, and GraphML. Furthermore, network data can be in simple adjacency matrix or (compressed) edgelist format, e.g. saved as `xlsx`, `csv` or `txt`. In essence, `nwimport` simply populates a Mata matrix with data and invokes `nwset`.

3.2.1 Example: Import network

In the next example, I import the Pajek NET file *pajek_example.net* from the `nwcommands` server. Pajek is a widely used software for social network analysis and its native file-format is supported by lots of other software as well. In Figure 3.2 you see the content of *pajek_example.net*, which is the network displayed in Figure 3.1. Without going into the details of the Pajek network-format, the first line ***Vertices 3** says that the network has three nodes. The keyword ***Arcs** indicates that the remaining lines should be interpreted as directed ties (arclist), e.g. node 1 sends a tie to node 2. When the option `type(pajek)` is specified, `nwimport` understands this network-format. By default, the imported network is named after the filename. Here, the imported network is called "pajek_example" (unless we already have a network called "pajek_example" in memory, in which case our new network would be named "pajek_example_1"). After importing the network in *pajek_example.net*, we have four networks in Stata memory: "network", "network_1", "network_2", "pajek_example".

```
*Vertices 3
*Arcs
1 2
2 3
3 2
```

Figure 3.2: Pajek NET format, *pajek_example.net*

```
. nwimport http://nwcommands.org/data/pajek_example.net, type(pajek)

Importing successful
(4 networks)

network
network_1
```

```
network_2
pajek_example
```

Another popular network-format that `nwimport` understands is a *compressed edgelist*. In this format, the first entry in each line defines the sending node. All other entries within the same line define the receiving nodes. For example, the data in Figure 3.8 represents a network where Thomas sends ties to both Peter and Tim, Peter sends a tie to Thomas, Tim sends a tie to Johan and Johan sends a tie to Thomas (see Figure 3.5). Figure 3.4 shows the same network in simple *edgelist* format. Both formats can be imported by specifying `type(compressed)` or `type(edgelist)`. Notice that in both formats nodes can be indicated by both numbers or strings. When the nodes are indicated by strings (as in Figure 3.8 and 3.4), node labels are automatically set. Internally, `nwimport` recognizes distinct nodes and assigns them consecutive *nodeids*. The suboption `type(edgelist, keeporiginal)` generates a variable `_nodeoriginal`, which holds the original names or numbers of nodes in the raw data. This can be useful when you have non-consecutive labelling of your nodes in the raw data and want to match the imported network with node attributes saved elsewhere.

```
Thomas, Peter, Tim
Peter, Thomas
Tim, Johan
Johan, Thomas
```

Figure 3.3: Compressed edgelist format, *compressed_example.txt*

```
Thomas, Peter
Thomas, Tim
Peter, Thomas
Tim, Johan
Johan, Thomas
```

Figure 3.4: Edgelist format, *edgelist_example.txt*

```
. nwimport http://nwcommands.org/data/compressed_example.txt, type(compressed)
-----
Importing successful
(5 networks)
-----
network
network_1
network_2
pajek_example
compressed_example
```

Another possibility is importing network data from an adjacency matrix, e.g. saved in Excel. Figure 3.6 shows the network in Figure 3.5 saved as a spreadsheet. The first

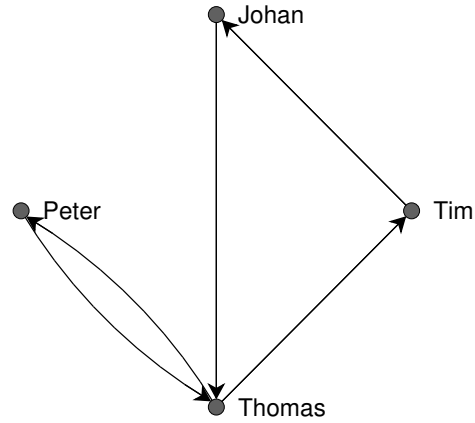


Figure 3.5: Another simple network

five columns show the adjacency matrix of the network. The last two columns hold additional node-level attributes (**Nationality**, **x**). Furthermore, the first column and the first row contain the node names (Peter, Thomas, Tim, Johan). By default, `nwimport` recognizes such node names, but one can also make this explicit with `type(matrix, rownames colnames)`.

```
. nwimport http://nwcommands.org/data/excel_example.txt, type(matrix, rownames colnames)
```

```
Importing successful
(6 networks)
```

```
network
network_1
network_2
pajek_example
compressed_example
excel_example
```

	A	B	C	D	E	F	G
1		Peter	Thomas	Tim	Johan	Nationality	x
2	Peter	0	1	0	0	Swedish	60
3	Thomas	1	0	1	0	German	34
4	Tim	0	0	0	1	German	32
5	Johan	0	1	0	0	Swedish	35

Figure 3.6: Adjacency matrix with attributes, *excel_example.xlsx*

3.2.2 Syntax for importing networks

```
nwimport filename, type(import_type[,type_sub]) [name(newnetname) nwclear]
```

3.2.3 Options for importing networks

`type(import_type)` specifies the network format.

`name(newnetname)` assigns a new name to the network.

`nwclear` clears all network data from memory before importing new networks.

<i>import_type</i>	Description
<code>pajek</code>	Network is in Pajek NET format
<code>ucinet</code>	Network is in Ucinet DL format
<code>graphml</code>	Network is in GraphML format
<code>gml</code>	Network is in GML format
<code>matrix</code>	Network is in raw adjacency matrix format (e.g. Excel file)
<code>edgelist</code>	Network is in raw edgelist format (e.g. Excel file)
<code>compressed</code>	Network is in compressed edgelist format

Table 3.1: Supported network file-formats

<i>type_sub</i>	Description
<code>rownames</code>	First row in matrix contains node labels
<code>colnames</code>	First column in matrix contains node labels
<code>delimiter(string)</code>	Explicitly specify delimiter for Matrix data

Table 3.2: Suboptions for `nwimport`

3.3 Dealing with node attributes

In contrast to other software, e.g. *statnet* in R, node attributes are not explicitly attached to a network. Instead, they are simply loaded as normal Stata variables. You can do that with the `insheet` or `import` command (or just click your way through *File* → *Import*).

The value of observation number i of these variables are automatically matched to the node with `nodeid = i`. This gives you a lot of flexibility for calculating node attributes or doing something else with them using normal Stata commands. When you import or set your data from a matrix, there is not much to consider because node attributes are matched automatically to the correct nodes. For example, the data displayed in Figure 3.6 contains the additional node attributes `Nationality` and `x`. When you import this data with `nwimport` as shown before, the command first checks

the number of observations `_N`. After that it only interprets the first `_N` variables in this dataset as adjacency matrix. The remaining variables `Nationality` and `x` are still imported to Stata, but are ignored when it comes to setting the network. When your network data is in matrix format, your node variables are already sorted correctly. For example, the entry `Age[2] = 34` stands for "Thomas" and will be matched with the node with `nodeid = 2`, who also happens to be "Thomas" when setting this network.

One thing to consider is how *nodeids* are generated and how you can match your attribute data to nodes when your network data is in edgelist format. *Nodeids* are always consecutive numbers from 1 to size of the network. Let's say you import the edgelist data in Figure 3.4. Here, the node identifiers in the raw data are "Thomas", "Peter", "Tim" and "Johan". These raw identifiers are translated into *nodeids* according to their alphabetical order. That means "Johan" gets assigned `nodeid = 1`, "Peter" gets `nodeid = 2`, "Thomas" gets `nodeid = 3` and Tim gets `nodeid = 4`. You can simply verify this by looking at the variables `_nodelab` and `_nodeid` that are automatically generated when you import this network. By default, the *nodelab* of a node is always the raw identifier.

If you want to merge in attribute data from elsewhere, e.g. "attributes.dta" it makes sense to generate a variable called `_nodelab` that identifies the nodes in exactly the same way as they are identified in the raw network data. For example, below I prepared a little attribute file that has three variables `_nodelab`, `Nationality` and `x`. Let us assume this data is saved as "attributes.dta" in the working directory. After importing the network displayed in Figure 3.5 (as outlined before), I could then simply merge in the attribute file with `merge 1:1 _nodelab using attributes.dta`.

```
. list _all
```

	<code>_nodelab</code>	<code>Nation-y</code>	<code>x</code>
1.	Peter	Swedish	60
2.	Thomas	German	34
3.	Tim	German	32
4.	Johan	Swedish	35

3.4 Export networks

So far, networks can be exported with `nwexport` and saved in Pajek `.net` and Ucient `.dl` format. Both are very popular network-formats and can be read by many other software. The command is used straightforwardly with a single *netname*. The option `type(export_type)` needs to be specified. Allowed types are `pajek` and `ucinet`. By default, the command saves the exported network under the name *netname* in the working directory, unless option `fname(filename)` is specified. For example, to export the network "network_1" and save it in Pajek-format, I write `nwexport network_1, type(pajek)`.

3.4.1 Syntax for exporting networks

`nwexport` *netname*, `type`(*export_type*) [`,fname`(*filename*) *replace*]

3.4.2 Options for importing networks

`type`(*export_type*) specifies the network format.

`fname`(*filename*) name of file. By default, the command saves the network with the name *netname* in the working directory.

<i>export_type</i>	Description
<code>pajek</code>	Export network to Pajek .net format
<code>ucinet</code>	Export network to Ucinet .dl format

Table 3.3: Suboptions for `nwexport`

3.5 Save and use networks

Network data can be saved and used just like normal data (see `nwsave` and `nwuse`). Practically, `nwsave` transforms all networks in current memory in edgelist or matrix format and saves them (together with some meta-information and any other normal variables in a dataset) as a normal dta-file.⁷ Any networks that have been previously saved with `nwsave` can be used with `nwuse`. In essence, `nwuse` loads a dta-file and uses the meta-information that `nwsave` generated before to "set" the network data. This makes networks accessible by their *netname*. Both commands are used in exactly the same way as their normal Stata equivalents. Similarly, `webnwuse` loads a network dataset from the internet. It works just like `webuse`, but also generates the relevant network objects. By default, `webnwuse` tries to load network data from the `nwcommands` server, but this can be changed (just like for `webuse` as well). Currently, there are several example datasets on the `nwcommands` server. A description of these datasets is shown with `netexample` (see Table 3.4).

3.5.1 Example: Use network data from the internet

In the next example, I load the network dataset `florentine`. This is a dataset of marriage and business ties among Renaissance Florentine families collected by John Padgett from historical documents (Padgett and Ansell, 1993). In this dataset are two networks: 1) `flomarriage` and 2) `flobusiness`. The first is a network of marriage alliances (between the families) and the second is a network of business relationships (recorded financial ties such as loans, credits and joint partnerships).

⁷Most likely, you will never have to interact with the Stata file-format for networks that I introduce directly. However, in case you want to know more about how network data is saved look at the help-file for `nwsave`.

<i>netexample</i>	Networks	Description
gang	gang, gang_valued	Network of co-offending in a youth gang, including node-level attributes, e.g. age, birthplace, and so on (Grund and Densley, 2012).
glasgow	glasgow1, glasgow2, glasgow3	Friendship network in a Scottish high-school (Pearson and West, 2003) at three different points in time. Includes node-level attributes, smoking and drinking behavior at different time-points.
florentine	flomarriage, flobusiness	Marriage and business relationships between important families in Renaissance Florence (Padgett and Ansell, 1993). Data includes wealth and political representation of the families.
klas12b	klas12b_wave1- klas12b_wave4 klas12b_primary	Friendship network in a Dutch high-school (Snijders et al., 2010, see)) at five different points in time. Additional node-level information on delinquency, alcohol consumption and so on.
hpotter	hpbook1- hpbook6	Support relationships of characters in the Harry Potter books (Bossart and Meidert, 2013).

Table 3.4: Network example datasets on the nwcommands server

```
. webnwuse florentine
Loading successful
(7 networks)

network
network_1
network_2
pajek_example
compressed_example
flomarriage
flobusiness
```

All these routines are ways to have relational data declared as networks. **nwset** is the low-level command to declare network data explicitly. Other commands, such as **nwimport**, **nwuse**, **webnwuse** and all other network generators (commands that simulate networks or generate new networks based on existing networks or attributes) invoke **nwset** implicitly.

3.6 Load a network as Stata variables

Networks exist as quasi-objects in memory. After a network has been imported, generated or explicitly set, one can refer to it by its *netname*, just as if one would refer to a normal variable. There are different ways to quickly examine or eye-ball a network. The

command `nwload` generates a set of Stata variables (the names of these variables are stored in the meta-information for a network) and populates these variables with the adjacency matrix of a network. "Loading" a network as Stata variables can be useful if one wants to interact with (or look at) the network as Stata variables. But notice that changing one of these Stata variables does not change the underlying network as well, unless used in combination with `nwsync`. The Stata variables simply represent a copy of the adjacency matrix (not a view).

3.6.1 Example: Load network as Stata variables

In the next example, I drop all Stata variables and "load" the network called "network" that we previously "set". This generates a bunch of variables. Some of them are meta-information (`_nodelab`, `_nodevar`, `_nodeid`) and some stand for the adjacency matrix of the network (`v1`, `v2`, `v3`).⁸ The variables in Table 3.5 are created to simplify interaction with, e.g. node labels. All Stata variables can be deleted without deleting the underlying networks (the networks ultimately exist in Mata with some meta-information stored in global macros). In case variables already exist, they are overwritten. If one wants to permanently drop a network one needs to use `nwdrop` or `nwclear` (very similar to how one would drop or clear normal variables).

Variable	Description
<code>_nodelab</code>	This variable holds the node labels (see <i>nodelab</i>).
<code>_nodeid</code>	This variable holds the node ids (see <i>nodeid</i>).
<code>_nodevar</code>	This variable holds the information about the variables that are used to represent the adjacency matrix of the network as Stata variables.

Table 3.5: Variables produced by `nwload`

```
. drop _all
. nwload network
. list _all
```

	<code>_nodelab</code>	<code>_nodevar</code>	<code>_nodeid</code>	<code>v1</code>	<code>v2</code>	<code>v3</code>
1.	<code>v1</code>	<code>v1</code>	1	0	1	0
2.	<code>v2</code>	<code>v2</code>	2	0	0	1
3.	<code>v3</code>	<code>v3</code>	3	0	1	0

Furthermore, `nwload` makes whatever *netname* is loaded the *current network*, unless option `nocurrent` is specified. The *current network* is normally the last network that has been imported or generated (see `nwcurrent`). For many `nw`commands (although they do something with a network) a *netname* is optional. When no *netname* is specified, a `nw`command automatically runs with the current network. This is simply for

⁸One can also load the node labels of a network as a Stata variable without loading the whole network as variables with the option `labelonly`.

convenience.⁹ By default, all commands that generate a network (see Section 6.1) also load the network as Stata variables. However, these network generators almost always have the option `xvars`, which prevents `nwload` from executing after creating a new network. This can be useful when one deals with many networks or very large networks at the same time and reaches the limits of variables that can be handled in Stata. Notice that `nwload` does not import or create a network, it simply creates Stata variables to represent a network. Only networks that already exist in Stata, i.e. have been "set", imported or created by another `nwcommand`, can be loaded as Stata variables. I recommend to use the representation of a network as Stata variables only for inspection. Changing networks like this certainly works, but one needs to sync the variables with the actual network afterwards; it is also not the most efficient way to change values of a network.¹⁰

3.6.2 Syntax for loading network as Stata variables

```
nwload [netname] , [id(#) nocurrent labelonly force]
```

3.6.3 Options for loading network as Stata variables

`id(#)` *networkid* of the network that should be loaded. In case the command is called with `netname`, priority is given to *networkid*.

`nocurrent` only loads network as Stata variables, but does not make it the current network.

`labelonly` only loads the node labels as Stata variable.

`force` generates Stata variables. By default `nwload` is not executed for networks with more than 1000 nodes unless `force` is specified.

3.7 Display tie values of a network

The command `nwvalue` allows extraction of specific tie values. But it can also be used to return subsets or a whole network as a Stata matrix, which can then be normally displayed. For example, when used without any further specification, `nwvalue network` returns all tie values of "network" in the Stata matrix `r(values)`. The main purpose of the `nwvalue` command, however, is to return specific tie values (for example, does node 2 send a tie to node 3?). It uses the concept of *network subsetting*, which is very similar to subsetting matrices in Mata. For example, to find out if the node with `nodeid == 2` sends a tie to the node with `nodeid == 3` in network "network", I can type `nwvalue network[2,3]`. In this case, `r(values) == 1`, which means that node 2 sends a tie to node 3. Network subsetting is a powerful feature that becomes especially relevant when manipulating networks (see Section 5.4). In essence, whatever is written in *subset* is

⁹For example, the command `nwplot` will plot the *current network* when used without a specific *netname*.

¹⁰Hence, changing some of these variables, e.g. `replace v1 = 1 in 2` does not change the network, but just the Stata variable. To change the values of the underlying network directly use `nwreplace` (e.g. `nwreplace network[1,2] = 1`), but more on manipulating networks in Section 5.4.

directly passed on to the underlying Mata matrix. Hence, all ways for subsetting Mata matrices are allowed when subsetting networks as well (see Table 3.7), including list subscripts, range subscripts and so on (see [M-2] **subscripts**).

Example	Description
<code>mynet</code>	The whole network.
<code>mynet[2,3]</code>	Specific tie value; toe that node 3 received from node 2.
<code>mynet[(2::4),3]</code>	All ties that node 3 receives from nodes 2 to 4.
<code>mynet[(2::4),(3::4)]</code>	All ties that nodes 3 to 4 receive from nodes 2 to 4.
<code>mynet[((2,3)\(4,4))]</code>	All ties that nodes 2 to 4 send to nodes 3 to 4.

Table 3.6: Examples for network subsetting

3.7.1 Example: Network tie values

Another way to print the underlying adjacency matrix is to get hold of it with `nwvalue` and then display it like any other Stata matrix. When `nwvalue` is used without *network subsetting* it automatically refers to the whole network and returns all tie values in the return vector `r(values)`. More generally, `nwvalue` returns a Stata matrix in `r(values)` when more than one specific tie value is requested.¹¹ In the next example, I type `nwvalue network[2,3]` to obtain a specific tie value. Because I request only one tie value, `nwvalue` populates the return vector differently. It generates the scalars `r(ego)` with the *nodeid* of the sending node, `r(alter)` with the *nodeid* of the receiving node and `r(value)` with the tie value.

```
. nwvalue network
. mat list r(values)
r(values)[3,3]
      c1  c2  c3
r1    0   1   0
r2    0   0   1
r3    0   1   0

. nwvalue network[2,3]
. return list
scalars:
      r(ego) = 2
      r(alter) = 3
      r(value) = 1
```

3.7.2 Syntax for returning tie values

`nwvalue` [*netname*] [*subset*]

¹¹Notice that this command generates a Stata and not a Mata matrix. The latter can be obtained with `nwtomata`, which is intended for advanced network programming. But of course, it can also be used to simply display an adjacency matrix.

3.8 Obtaining meta-information

Each network is saved together with some meta-information about the network. This meta-information includes the size of the network, whether it is directed or undirected, the labels of the nodes in the network, a list of variables that should be used when loading the network as Stata variables. All this information is returned in the `r()` vector with `nwname`. The command is used straightforwardly followed by a *netname*. When used without a *netname*, the command refers (like all *nwcommands* where a *netname* is optional) to the *current network*. When a network with a particular *netname* does not exist, the `nwname` throws an error. Hence, it can also be used to quickly check if a network exists.

```
. nwname flomarriage
. return list
scalars:
      r(id) = 2
      r(nodes) = 16
macros:
      r(name) : "flomarriage"
      r(directed) : "false"
      r(vars) : "acciaiuoli albizzi barbadori bischeri castellani ginori.."
      r(labs) : "acciaiuoli albizzi barbadori bischeri castellani ginori.."
```

Besides obtaining the meta-information of a network, the command can also be used to change some of this meta-information. First, one can change the name of a network with option `newname(newnetname)`.¹² One can also change whether the network should be treated as directed or undirected. This is important for many commands (e.g. `nwplot` when plotting). One can specify either `newdirected(true)` to set a network as directed or `newdirected(false)` to set it as undirected. Notice that the last option only changes the meta-information of a network and does not symmetrize the actual network data. You have to use `symmetrize` to symmetrize a network, which also updates the meta-information. Some other meta-information that can be changed explicitly are the node labels of a network. By default, every node in a network gets assigned a node label. When you import a network as the one in Figure , node labels are automatically set. There are two ways to explicitly change the node labels that are saved together with a network. First, you can use the option `newlabs(lab1 lab2 ..)`. Inside the option you need to specify as many words as there are nodes in the network. When used with a non-valid number of labels (more or less words than there are node labels), the command stops and does not throw an error. Second, you can assign new node labels to a network from a Stata variable *var*. In this case, the value of the variable in the *i*'th row is assigned to the node with *nodeid* = *i*. Lastly, you can also change the meta-information about the variables that should be generated when loading a network as Stata variables. Again, this works very similar to changing node labels. You can use the option `newvars(var1 var2 ..)`. Inside the option you need to specify as many words as there are nodes in the network.

¹²Notice that you can also do this with `nwrename`.

3.8.1 Syntax for obtaining/changing meta-information

```
nwname [ netname ] , [ id(#) newname(newnetname) newdirected(boolean)  
newvars(var1 var2 ..) newlabs(lab1 lab2 ..) newlabsfromvar(varname) ]
```

3.8.2 Options for obtaining/changing meta-information

`id(#)networkid` of the network. In case the command is called with *netname*, priority is given to *networkid*.

`newname(newnetname)` changes the name of the network.

`newdirected(boolean)` changes the directionality of the network. Notice, this does not make a network symmetric.

`newvars(var1 var2 ..)` changes the Stata variables that should be used when "loading" the network.

`newlabs(lab1 lab2 ..)` changes node labels that are attached to the network.

`newlabsfromvar(varname)` changes node labels that are attached to the network.

Chapter 4

Characterize networks

4.1 Summarize a network

Another possibility to look at a network is using `nwsummarize`, which displays summary statistics for a network. It can also be called with a *netlist* and results can be saved in an external file (by specifying option `save(filename)`). When one of the options `mat` or `matonly` is specified, the underlying adjacency matrix of a network is shown.

By default `nwsummarize` shows the name of a network, its directionality, the number of nodes in the network, the number of arcs/edges, the minimum and maximum tie value and network density. The density of a network is simply the fraction of ties that do exist in a network.

Furthermore, with option `detail` the command also calculates a bunch of additional network statistics, e.g. reciprocity, transitivity and centralization scores. Reciprocity is the fraction of dyads that are reciprocated out of those dyads with at least one non-zero tie. A dyad is reciprocated when $Y_{ij} == Y_{ji}$. And a dyad can potentially be reciprocated when either $Y_{ij} \neq 0$ or $Y_{ji} \neq 0$. Transitivity is the fraction of transitive triplets out of those triplets that can be potentially transitive.

4.1.1 Example: Network summary

In the following example, I use `nwsummarize` to give a summary of the network with the *netname* "network". By default, this displays the name of a network, its *networkid*, its directionality, the number of nodes, the number of arcs/edges, the minimum and maximum tie value and the network density. The option `mat` prints the underlying Mata adjacency matrix.

```
. nwsummarize network, mat
```

```
Network name: network
Network id: 1
Directed: true
Nodes: 3
Arcs: 3
```

```

Minimum value: 0
Maximum value: 1
Density: .5
  1  2  3
1  0  1  0
2  0  0  1
3  0  1  0

```

4.1.2 Syntax for network summary

```
nwsummarize [ netlist ] , [ mat matonly detail save(filename) ]
```

4.1.3 Options for network summary

mat displays adjacency matrix.

matonly only displays adjacency matrix.

detail calculates additional summary statistics.

save(*filename*) saves summary statistics in a file.

4.2 Dyad and triad census

One way to characterize a network is describing the kinds of dyads and triads a network has. A dyad is a pair of nodes i and j . A network with n nodes has $n(n-1)/2$ dyads. Any dyad in a directed network can be either 1) mutually connected, 2) asymmetrically connected, or 3) not connected at all. A dyad with the nodes i and j is mutually connected when there is a tie from node i to j and another tie from node j to i ($Y_{ij} = Y_{ji} \neq 0$). In contrast a dyad between two nodes i and j is asymmetrically connected when there is a tie from one node to the other, but not the other way around ($Y_{ij} \neq 1$ and $Y_{ji} = 0$). Lastly, the pair of nodes i and j can be not directly connected with each other at all, i.e. there is neither a tie from node i to j , nor a tie from node j to node i ($Y_{ij} = Y_{ji} = 0$). In undirected networks, by definition, there are no asymmetric dyads and dyads can only be mutual or null.

Counting the number of "M"utual, "A"symmetric, and "N"ull dyads is called the dyad census. It gives for each network the so-called MAN code. One can understand this MAN code as some sort of "fingerprint" of a network. Most networks will have different MAN codes (although some may have the same code). But comparing MAN codes of different networks with each other already says a lot about differences (or similarities) of different networks (Holland and Leinhardt, 1970).

The command **nwdyads** returns the dyad census of a network. Let me load a directed network with **webnwuse glasgow**. This dataset contains three networks "glasgow1", "glasgow2" and "glasgow3". The command **nwdyads glasgow1** gives the dyad census for network "glasgow1". In this network, 38 dyads are mutual, i.e. there exists a tie from node i to j and a tie from node j to i . 35 dyads in this network are asymmetric, i.e. there is a tie from node i to j , but not the other way around. Most dyads in this network

are null (1151). The command also returns the reciprocity coefficient for the networks. Reciprocity is calculated as $M/(M+A)$. In this case, $reciprocity = 39/(39+35) = .527$.

```
. webnwuse glasgow, nwcLEAR
. nwdyads glasgow1
    Dyad census:  glasgow1
    Mutual |      Asym |      Null
    -----|-----|-----
           |           |
           39 |         35 |        1151

    Reciprocity: .527027027027027
```

One can similarly count the number of triads a network has (Davis and Leinhardt, 1972). For a directed network, there are 16 possible triadic configurations (see Figure 4.1). For example, three nodes i , j and k can all be unconnected with each other ($Y_{ij} = Y_{ji} = Y_{ik} = Y_{ki} = Y_{jk} = Y_{kj} = 0$). In the MAN-notation this kind of triad has the MAN-code 003.

```
. webnwuse glasgow, nwcLEAR
. nwtriads glasgow1
    Triad census:  glasgow1
    003 |      012 |      021D |      021U |
    ----|-----|-----|-----|
    16243 |      1470 |          5 |         18 |
    021C |      030T |      030C |         102 |
    ----|-----|-----|-----|
           21 |          5 |          0 |        1724 |
    120D |      120U |      120C |        111D |
    ----|-----|-----|-----|
           6 |          5 |          2 |         42 |
    111U |      201 |         210 |        300 |
    ----|-----|-----|-----|
           30 |         15 |          9 |          5 |

    Transitivity: .3870967741935484
```

4.2.1 Syntax for dyad and triad census

`nwdyads` [*netlist*]

`nwtriads` [*netlist*]

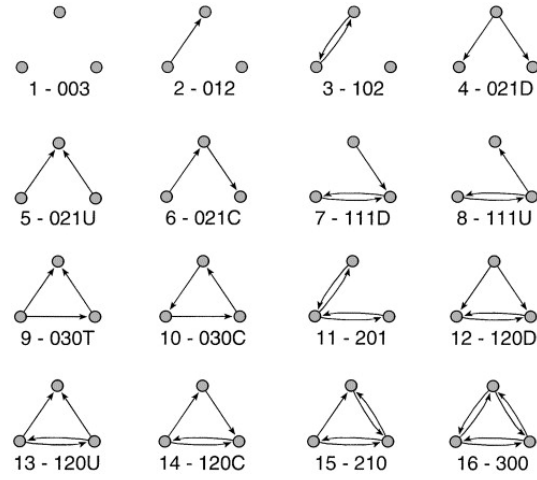


Figure 4.1: Triad census

Chapter 5

Manipulate networks

5.1 Drop and keep networks

A major feature of the `nw` commands is that one can deal and interact with networks almost as if they were normal Stata variables, e.g networks can be dropped or kept. The command `nwdrop` drops a list of existing networks from memory and is used in almost exactly the same way as the normal `drop` command. In fact, one can even make use of `*`, `_all` or other abbreviations because `nwdrop` is used with a *netlist* and a *netlist* allows the same abbreviations as a *varlist*. Technically, whenever a network is dropped, the corresponding Mata adjacency matrix is dropped as well. Furthermore, the corresponding meta-information is dropped too and the ordering of all other networks is adjusted.¹ For example, after running all these different examples there are a bunch of networks in Stata memory. I can drop the networks "pajek_example", "compressed_example", "flobusiness" and "flomarriage" with `nwdrop` followed by the names of the networks. Alternatively, I could have also just kept all networks starting with "net" by typing `nwkeep net*`.

```
. nwset
(7 networks)
-----
network
network_1
network_2
pajek_example
compressed_example
flomarriage
flobusiness

. nwdrop pajek_example compressed_example flomarriage flobusiness

. nwset
(3 networks)
-----
```

¹Internally, networks also have a *networkid*, which is a consecutive number from 1 to number of networks in memory. However, this *networkid* is for internal programming only and all `nw` commands work by default with a *netname*.

```

network
network_1
network_2

```

Both commands **nwdrop** and **nwkeep** can also be used to drop/keep specific network nodes. This is just like dropping/keeping specific observations from a dataset.² One can do that by either specifying *[if]* or *[in]*. In both cases, the observation number **_n** corresponds to the *nodeid* of nodes. If one wants to drop the node with *nodeid* == *i* from network "network", one specifies **nwdrop network if _n == i**. Notice that this allows the use of any other Stata variable in the criterion for dropping or keeping nodes as well. For example, one can first calculate whether or not a node belongs to the largest component (the largest set of nodes that can all reach each other), e.g. **nwgen large = lgc(network)** and then drop nodes for which **large == 0** with **nwdrop network if large == 0**. Notice that when a network has been loaded as Stata variables before, these variables are updated after dropping/keeping nodes.

When a specific node is dropped, all network ties that include this node are dropped as well.³ A complicating issue is how to deal with node-level attributes. The **nwcom** commands do not explicitly assign attributes to nodes, but rather match the observations of normal Stata variables with the *nodeid* of nodes, i.e. the *i*'th observation of a variable is matched to the node with *nodeid* == *i*. This is very straightforward and allows easy calculation and manipulation of node-level attributes. After all, they are just normal Stata variables. But it becomes tricky when nodes are deleted because it can change the *nodeid*'s of some nodes.⁴ Let me illustrate the issue by generating a new variable. This could be any attribute where observation number *i* corresponds to the node with *nodeid* = *i*. For illustrative purposes, I generate a consecutive number **gen myattr = _n**. Next, I drop the first node of **network** with **nwdrop network if _n == 1**. After having dropped this node, you see that the first row now corresponds to what was previously the second node with *nodeid* = 2. The representation of the network as Stata variables is adjusted as well. When nodes are dropped *nodeid*'s are automatically adjusted to always range from 1 to size of the network. But now, the value **myattr[1]** = 1 does not reflect the attribute of the node in the first row anymore.

```

. gen myattr = _n
. list _all

```

	_nodelab	_nodevar	_nodeid	v1	v2	v3	myattr
1.	v1	v1	1	0	1	0	1
2.	v2	v2	2	0	0	1	2
3.	v3	v3	3	0	1	0	3

```

. nwdrop network if _n == 1
. list _all

```

²There are also the command **nwdroptnodes** and **nwkeepnodes**. They work in exactly the same way, except that the *nodeids* of the nodes that should be dropped/kept are specified as *numlist*.

³See also **nwsubset** for getting sub-networks. In contrast to **nwdrop**, which simply deletes nodes from a network, the **nwsubset** command generates a new network for the desired sub-network.

⁴More precisely, when node *k* is deleted, it changes the *nodeid*'s of all nodes with a *nodeid* greater than *k*. Remember *nodeid*'s are consecutive numbers from 1 to size of the networks.

	_nodelab	_nodevar	_nodeid	v2	v3	myattr
1.	v2	v2	1	0	1	1
2.	v3	v3	2	1	0	2
3.			.	.	.	3

Dropping cases in the dataset when dropping specific network nodes would be problematic as well because you might have other attributes in your dataset that correspond to other networks. Luckily, there is an elegant solution for this issue. The command **nwdrop** (also **nwkeep**) has the option **attribute(varlist)**, which allows you to specify which variables should be included in the drop/keep. Let me illustrate this on network "network_1". Remember, originally it was exactly the same network as network "network" (before the drop). First, let me drop all variables and "load" the network as Stata variables. Then, I create the variable **gen myattr = _n** again. Lastly, I drop the first node of network "network_1", but this time, I include the variable **myattr** in the drop by specifying the option **attribute(myattr)**. You see that after this operation **myattr[1] = 2**.

```
. drop _all
. nwload network_1
. gen myattr = _n
. nwdrop network_1 if _n == 1, attribute(myattr)
. list _all
```

	_nodelab	_nodevar	_nodeid	v2	v3	myattr
1.	v2	v2	1	0	1	2
2.	v3	v3	2	1	0	3

Dropping and keeping networks work just like dropping and keeping variables. A related command is **nwclear**, which clears all normal data and all network data from memory. It corresponds to **clear**. If you just want to delete all networks, but keep other Stata variables in memory, you can use **nwdrop _all**. By default, the Stata variables representing a network are dropped as well when you drop a network.

5.1.1 Syntax for dropping/keeping networks or nodes

nwdrop [*netlist*] [*if*][*in*], [**attributes(varlist)** **netonly**]

nwkeep [*netlist*] [*if*][*in*], [**attributes(varlist)** **netonly**]

5.1.2 Options for dropping/keeping network or nodes

attributes(varlist) defines the variables that should be included in the drop/keep.
netonly only drops/keeps the network, but leaves the Stata variables, which represent a network as they are (in case the network has been "loaded" before).

5.2 Add nodes to a network

Nodes cannot only be dropped, but also added to a network. This can be useful when writing own *network generators* or simply when manipulating a network. The relevant command is called `nwaddnodes`. It adds isolates (nodes who are not connected to anybody else) to a network. It also assigns some meta-information to the new nodes, including a node label (*nodelab*) and a Stata variable (*nodevar*) that should be used for representing the new node when the network is loaded with `nwload`. By default, new nodes are labelled "new_1", "new_2", and so on unless the option `labs(lab1 lab2...)` is specified. Furthermore, the same labels are used as *nodevar* as well. For example, let me add two nodes to the network "network_1". Furthermore, let me assign the node labels "A" and "B" to these two nodes explicitly. I can just type `nwaddnodes network_1, nodes(2) labs(A B)`.

5.2.1 Syntax for adding nodes

```
nwaddnodes [netname], newnodes(#) [vars(newvarlist) labs(lab1 lab2...)  
generate(newnetname)]
```

5.2.2 Options for adding nodes

`newnodes(#)` specifies the number of isolates that should be added to a network.
`vars(newvarlist)` specifies the Stata variables that should be used for representing the additional nodes when loading the network with `nwload`. Needs to have as many words as new nodes that are added.

5.3 Collapse nodes of a network

There is also a straightforward way to collapse network nodes with `nwcollapse`. This works very similar to collapsing observations in normal Stata, but it collapses nodes in a network. Essentially, this command merges network nodes together. With option `by(varname)` one specifies which nodes should be merged. All nodes for which the values in *varname* are the same are collapsed to a single node. The rule for collapsing nodes is specified in (*stat*). Again, this is very similar to the normal `collapse` command and all stats that can be used for `collapse` can be used with `nwcollapse` as well. By default *stat* = `max`. For example, when nodes A and B are collapsed to node Z, Z inherits all ties from both nodes A and B. By specifying, e.g. (`min`) Z would only inherit only those ties to other nodes C to whom both A and B originally had ties with.

5.3.1 Syntax for collapsing nodes

```
nwcollapse [(stat)] [netname], [by(varlist) options]
```

5.3.2 Options for collapsing nodes

`by(varlist)` specifies the variables that should be used for collapsing nodes.

5.4 Replace and recode networks

Networks can be manipulated in different ways. There are specific commands like `nwsym`, which symmetrizes a network, or `nwtranspose`, which transposes a network. But there is also a much more general and powerful way to manipulate networks with `nwreplace`. In essence, it works very similar to the normal `replace` command. It can replace whole networks, sub-networks or only specific dyads with another network or with a *network expression*.⁵ A *network expression* is very similar to a normal expression in Stata, but also accepts *netnames*. Furthermore, `nwreplace` accepts different sorts of conditions (*if*, *ifego*, *ifalter*) on the dyad and on the node-level.⁶

5.4.1 Syntax for replacing networks

```
nwreplace netname[subnet] = netexp [if] [ifgo] [ifalter] [in]
```

5.4.2 Assign constants to dyads

One can use *subnet* to change specific dyads in a network. The syntax for *subnet* works just like subsetting a Mata matrix. Actually, the command just passes *subnet* on to the relevant Mata matrix. Let me change a single tie value of the network "network_2". More precisely, I change the value of the tie that node 1 sends to node 2 with `nwreplace network_2[2,1] = 55`. Originally, this value is 0, i.e. there is no tie between node 1 and node 2. Now, the value of this is 55. We could also just replace all dyads Y_{ij} of a network with a constant. In this case, there is no need to specify *subnet*. One simply types `nwreplace network_2 = 99` to set all tie values to 99.

```
. nwreplace network_2[2,1] = 55
. nwsummarize network_2, matonly
```

	1	2	3
1	0	1	0
2	55	0	1
3	0	1	0

```
. nwreplace network_2 = 99
. nwsummarize network_2, matonly
```

⁵One can also change the entire adjacency matrix of a network with an existing Mata matrix using `nwreplacemat`.

⁶Of course, one can also replace dyads in networks by loading a network as Stata variables (see `nwload`), changing the Stata variables (see `replace`) and, then, syncing Stata variables and network afterwards (see `nwsync`). However, replacing the networks directly (as I show here) is the faster and preferred method.

```
[symmetric]
      1    2    3
1  0
2  99   0
3  99  99   0
```

But we can also limit the assignment of values to a true subset of a network. In this example, I assign the value 3 to all ties between the first and the second node. Again, this makes use of *network subsetting*, which works exactly like when subsetting matrices in Mata.

```
. nwreplace network_2[(1:2),(1::2)] = 3
. nwsummarize network_2, matonly
[symmetric]
      1    2    3
1  0
2  3     0
3  99  99   0
```

5.4.3 Assign values from other networks to dyads

So far, I only assigned constants to specific dyads. It is also possible to assign values from other networks to dyads. This gives you a taste of the powerful calculations you can do with *network expressions*.⁷ For example, let me generate two random networks "first" and "second" with 6 nodes each (see `nwrandom`). In detail, the first command generates a random network with 0.2 density and the second command generates a random network with 1 density, i.e. a complete network where everybody is connected with everybody. Afterwards, I replace the network "first" with whatever is in network "second". More precisely, this replaces the adjacency matrix of network "first" with the adjacency matrix of network "second" and updates the relevant meta-information if necessary (e.g the number of nodes).

```
. set seed 123
. nwrandom 6, density(.2) name(first)
. nwrandom 6, density(1) name(second)
. nwreplace first = second
```

Again, one can apply network subsetting. In the next example, I multiply the values in network "first" (which holds now the adjacency matrix of network "second") by 5. Afterwards, I replace a subset of network "first" with the constant 0.

```
. nwreplace first = first * 5
. nwreplace first[(1::3),(1::3)] = 0
```

⁷Network expressions accept almost all mathematical and logical operators one knows from normal expression. In detail, a network expression transforms each operator that it finds into a Mata element-by-element operator, e.g. `==` or `.*` and uses them together with the underlying adjacency matrices for each network to perform some calculation in Mata. The result is another Mata matrix, which is then assigned to network where one replaces somethings.


```
. nwsummarize first, matonly
[symmetric]
```

	1	2	3	4	5	6
1	0					
2	0	0				
3	0	0	0			
4	5	5	5	0		
5	5	5	5	5	0	
6	5	5	5	5	5	0

5.4.4 Assign values with if-conditions

Another feature of `nwreplace` is that it allows conditional assignments to dyads. For networks there are three different conditions that can be used: 1) *if netexp*, 2) *ifego exp*, 3) *ifalter exp*. The first one evaluates the following expression as a network expression, which generates a new temporary network, and only replaces dyads for which the temporary network has the value 1. For example, this code replaces the network "first" with the values of the network "third", but only for the ties where the network "fourth" is 1.

```
. set seed 123
. nwrando 6, prob(.3) name(third)
. nwrando 6, prob(.5) name(fourth)
. nwreplace first = third if fourth == 1
. nwsummarize first, matonly
```

	1	2	3	4	5	6
1	0	0	0	0	5	0
2	0	0	0	5	5	5
3	0	0	0	5	5	0
4	5	1	1	0	0	5
5	5	0	5	0	0	5
6	5	5	5	5	5	0

The *ifego* condition only replaces those dyads Y_{ij} where the expression *exp* is true for the sender of a tie. For example, this code replaces the values of ties with the constant 2 where the sender has a *nodeid* < 5. The *ifalter* condition works in a similar way, but limits changes to those ties where the receiver of a tie has a certain attribute. Both *ifego* and *ifalter* can be combined (and could even be combined with the normal *if* condition as well).

```
. nwreplace first = 2 ifego _n < 5
. nwsummarize first, matonly
```

	1	2	3	4	5	6
1	0	2	2	2	2	2
2	2	0	2	2	2	2
3	2	2	0	2	2	2
4	2	2	2	0	2	2
5	5	0	5	0	0	5

6	5	5	5	5	5	0
---	---	---	---	---	---	---

5.4.5 Recode tie values

In almost exactly the same way as one recodes the values of variables one can recode the tie values of networks as well. Instead of using the normal `recode` command one uses `nwrecode` and instead of using it with a *varname* one uses it with a *netname*. In fact, internally `nwrecode` transforms a network to an edgelist first, and then applies the normal `recode` command, before transforming the recoded edgelist back to a network. This means that all *rules* one knows from `recode` can be applied.

5.4.6 Example: Recode tie values

After all the fancy `nwreplace` operations we just performed, we can revert the network "first" back to a binary network by recoding its tie values with `nwrecode first (2/5 = 1)`. Notice that we can also make clever use of *network expressions* to achieve the same goal. In this case we would overwrite the network "first" with `nwreplace first = (first != 0)`. Again, you can see how this is very similar to normal Stata syntax. Often using *network expressions* is faster than `nwrecode`. The reason is that `nwrecode` transforms the network in an edgelist first (as Stata variables) and then runs a normal recode operation. But not all recoding operations can be reformulated as a network expression.

5.4.7 Syntax for recoding networks

```
nwrecode [netlist] (rule) [(rule) ...] [, generate(newnetlist) prefix(str)
test]
```

5.4.8 Options for recoding networks

`generate(newnetlist)` produces new recoded networks instead of replacing the networks in *netlist*.

`netonly` generates new network with *str* as a prefix.

`test` tests that rules are invoked and do not overlap.

Chapter 6

Generate networks

6.1 Network generators

Several network commands can be used to generate networks. Together these commands are called *network generators* because they either simulate a stylised network (e.g. random network, small-world network) with some parameters or they calculate a new network based on an existing network (e.g. reachability). In all cases, however, the outcome is a new network. Almost all *network generators* have some options in common, e.g. for naming new networks, specifying *nodelabs* and *nodevars* or "not-loading" a new network as Stata variables (see Table 6.1). Let me first discuss some *network generators* that simulate networks. In particular, I show how to generate 1) random networks, 2) ring lattice networks, 3) lattice networks, 4) small-world networks, and 5) networks based on dyadic tie probabilities.¹ A bunch of other *network generators* produce new networks based on existing networks. Basically, these are commands that either transform a network, e.g. duplicate, symmetrise, permute or transpose a network, or commands that calculate dyad-level values, such as reachability or shortest path between nodes. Lastly, there are *network expression* and *network functions*. These are very general concepts that allow generating new networks based on other networks. This is a bit like calculating with networks. For example, given there are two networks called "first" and "second", I can generate a new network "third" with `nwgen third = first + second`.

6.1.1 Random network

One of the most straightforward ways to generate random networks is the Erdős-Rényi model (Erdős and Rényi, 1959). In this model, each potential tie between nodes i and j has the same, uniform probability *prob* to exist, independently of other ties. This model generates random graphs with varying number of ties. The expected number of

¹Notice there are more *network generators* that I cannot present in detail here. For example, there are generators for preferential-attachment (or scale-free) (see `nwpref`) and homophilous networks (see `nwhomophily`).

<i>generator_options</i>	Description
name (<i>newnetname</i>)	Name of the new network.
labs (<i>lab1 lab2...</i>)	Node labels (<i>nodelabs</i>) for new network. There need to be as many words as there are nodes.
vars (<i>var1 var2...</i>)	Stata variables that should be used to represent the adjacency matrix of the network when "loading" it.
xvars	Generate network, but do not "load" it as Stata variables.

Table 6.1: Options for all network generators

ties for undirected networks with n number of nodes is $\binom{n}{2}prob$; for directed networks the expected number of ties is $2\binom{n}{2}prob$. A very closely related model for generating random networks has been introduced by Gilbert (1959). This model generates a fixed number of ties m , where the m ties are randomly selected from the pool of all potential ties with uniform probability. One can express this model also in terms of *density*, which is defined as $density = m/\binom{n}{2}$ for undirected networks (and $density = m/2\binom{n}{2}$ for directed networks).² Both *prob* and *density* range from 0 to 1. Due to the law of large numbers, both network models will behave similarly when $n \rightarrow \infty$.

With **nwrandom** both models can be used to generate random networks. To draw a random network based on a uniform tie probability one simply types **nwrandom** followed by the number of nodes one wants to create and adds the option **prob(float)**. To generate a random network with a fixed number of ties, one uses the option **density(float)**. In the latter case, the actual number of ties is calculated as $m = \text{round}(density \cdot \binom{n}{2})$.

In network analysis, a major concern is what a "random network" actually is. A large body of work has been dedicated to generate random networks that have certain properties other than e.g. a fixed number of ties. One property that one can also "fix" with **nwrandom** is the dyad-census (see Holland and Leinhardt 1970). A dyad-census is a characterisation of a network that simply counts how many 1) mutual, 2) asymmetric, and 3) null-dyads a network has. A dyad is simply a pair of two nodes i and j (where the order does not matter). Hence, a network with n nodes has $\binom{n}{2}$ dyads. A dyad is "mutual" when there is a tie from i to j and also a tie from j to i . A dyad is "asymmetric" when there is one tie from i to j , but no tie from j to i . Lastly, a dyad is a "null"-dyad when there is no tie between i and j in either direction. One can think of the dyad-census as a fingerprint of a network. There are many different networks that can have the same dyad-census, but each network has just one dyad-census. The option **census(mutual[asym null])** generates a "random" network with a fixed dyad-census. More precisely, it samples a network with uniform probability from all potential networks that have a specific dyad-census.

One of the three options **prob()**, **density()** or **census()** has to be specified together with **nwrandom**. In all three cases, a directed network is generated by default,

²For both models, this excludes self-loops.

unless option `undirected` is specified. By default, `nwrandom` tries to give the new network the name "random", unless the command is used with option `name(newnetname)`. If the name already exists, the command comes up with an alternative, e.g. "random_1". Furthermore, `nwrandom` has the option `ntimes(#)`, which generates many random networks at the same time. This can be very useful when one wants to write a statistical procedure that requires a distribution of random networks.

Example: Generate random network

Let me demonstrate the three different ways to use `nwrandom`. The command `nwrandom 20, prob(.1)` generates a (binary and directed) random network called "random". The network has 20 nodes and each tie has the same probability 0.1 to exist. The command `nwrandom 20, density(.1)`, however, produces a network with a fixed density of 0.1; i.e. exactly $38 = 0.1 \cdot \binom{20}{2}$ ties. Because there already is a network called "random", the new network is automatically called "random_1". After executing these commands, I obtain a simple network summary for these two new networks with `nwsummaries random*`. You can see that for network "random_1" the density is indeed 0.1, while for network "random" it is only approximately 0.1.

```
. set seed 123
. nwrandom 20, prob(.1)
. nwrandom 20, density(.1)
. nwsummaries random*
```

```
Network name: random
Network id: 1
Directed: true
Nodes: 20
Arcs: 36
Minimum value: 0
Maximum value: 1
Density: .0947368421052632
```

```
Network name: random_1
Network id: 2
Directed: true
Nodes: 20
Arcs: 38
Minimum value: 0
Maximum value: 1
Density: .1
```

Lastly, the command `nwrandom 20, census(25 35)` produces a network with exactly 25 mutual and 35 asymmetric dyads. In this case, there will be exactly 130 null dyads, because a network with 20 nodes always has $\binom{20}{2} = 190$ dyads in total. Leaving the number of null dyads out when calling the command has the advantage that one does not have to manually calculate the number of null dyads that remain. As there already exist the networks "random" and "random_1", the new network is automatically called "random_2". After generating the last network, I obtain the dyad-census with `nwdyads random_2`. As you can see, the new random network "random_2" has indeed 25 mutual and 35 asymmetric dyads.

```
. nwrandom 20, census(25 35)
. nwdyads random_2
  Dyad census:  random_2
  Mutual |      Asym |      Null
  -----|-----|-----
         |         |
         25 |      35 |      130
         |         |
Reciprocity: .1612903225806452
```

Syntax for generating random network

```
nwrandom nodes, prob(float) [undirected ntimes(#) generator_options]

nwrandom nodes, density(float) [undirected ntimes(#) generator_options]

nwrandom nodes, census(mutual[asym null]) [undirected ntimes(#)
generator_options]
```

Options for generating random network

`prob(float)` specifies the uniform probability for a tie to exist.
`density(float)` specifies the density for the random network.
`census(mutual[asym null])` specifies the dyad-census for the random network.
`undirected` generates undirected instead of directed network.
`ntimes(#)` generates multiple random networks.

6.1.2 Ring-lattice network

A very structured network is the so-called "ring-lattice" (or one-dimensional lattice). In this network, all nodes are situated on a circle and each node has ties to k neighbors on both sides. The command `nwring` generates such networks. As before, one types the command followed by the number of nodes the new network should have. Option `k(#)` specifies the number of neighbors that each node is connected with on each side, i.e. each node is ultimately connected to $2k$ other nodes. For example, `nwring 10, k(2) undirected` generates the network displayed in Figure 6.1. Again, more than one ring-lattice can be generated at the same time.

Syntax for generating ring-lattice network

```
nwring nodes, k(#) [undirected ntimes(#) generator_options]
```

Options for generating ring-lattice network

`k(#)` specifies the number of neighbors on each side.
`undirected` generates undirected instead of directed network.
`ntimes(#)` generates multiple ring-lattices.

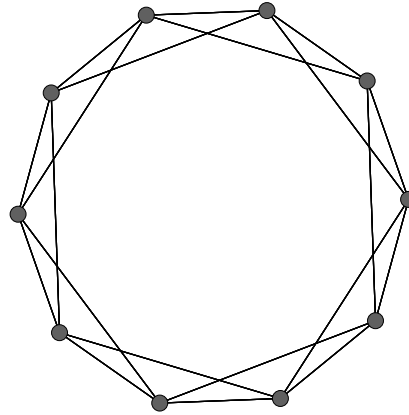


Figure 6.1: One-dimensional lattice

6.1.3 Lattice network

In contrast, the normal lattice (or two-dimensional lattice) is a graph where nodes are originally placed on a two-dimensional grid with a specified number of *rows* and *columns*. Next, each node is connected with the nodes directly above, or directly next to them. Only horizontal and vertical connections (and no diagonals) are allowed. For example, the network in Figure 6.2 has 48 nodes in 6 rows and 8 columns. By default, a lattice network does not "wrap" around, e. g. the node on the top left is not connected to the node on the top right or to the node on the bottom left. The option **xwrap** connects all nodes in the first column with the respective nodes in the last column (see Figure 6.2); the option **ywrap** connects all nodes in the first row with the respective nodes in the last row. Hence, Figure 6.2 is produced with `nwlattice 6 8, xwrap`.

Syntax for generating lattice network

```
nwlattice rows columns, [xwrap ywrap undirected ntimes(#)  
    generator_options]
```

Options for generating lattice network

xwrap connects nodes in first column with nodes in last column.
ywrap connects nodes in first row with nodes in last row.
undirected generates undirected instead of directed network.
ntimes(#) generates multiple ring-lattices.

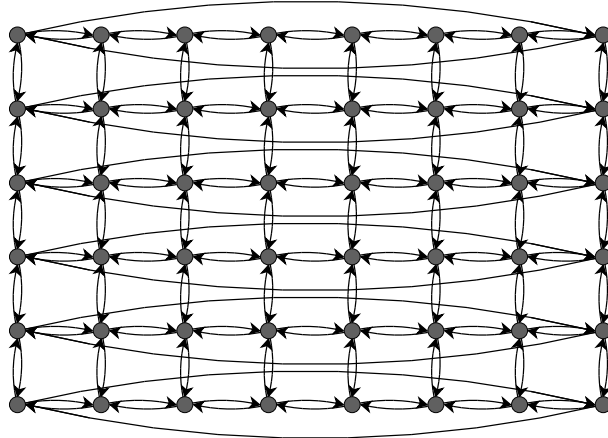


Figure 6.2: Two-dimensional lattice

6.1.4 Small-world network

Another commonly used algorithm to generate networks is the Watts-Strogatz model (Watts and Strogatz, 1998). This algorithm produces networks that have "small-world" properties, i.e. they are characterised by low average shortest path length between the nodes (the average number of steps it takes to get from one node to another) and high local clustering of nodes (Barrat and Weigt, 2000, see). Small-world networks capture the idea that individuals are embedded in densely-knit clusters, where within such a cluster everybody seems to be connected to everybody else, while at the same time one can reach anybody within the wider network in only a few steps (Watts, 1999). The latter phenomenon has been made popular with the notion of "six degrees of separation", which refers to the phenomenon that two randomly selected people in the world can be connected with each other in only six steps (Travers and Milgram, 1969, see).

Often, when we talk about a "small-world" network, we mean networks produced by the Watts-Strogatz algorithm (Watts and Strogatz, 1998). This algorithm works in the following way. First, it generates a ring-lattice with N nodes, where each node has ties to k neighbors on each side (just like the normal ring lattice). But then for every node i , it takes every edge (i, j) and rewires it with probability *prob*.³ "Rewiring" means that the tie (i, j) is changed into a tie (i, l) , where l is chosen with uniform probability from all possible values that avoid self-loops $l \neq i$ and link duplication. These "rewired" ties introduce "shortcuts" to the original network. The relevant *network generator* is `nwsmall`. It is used in almost exactly the same way as `nwring` and requires the option `k(#)`, which indicates the number of network neighbors on each side in the ring-lattice.⁴

³Often, this probability is referred to as β in the literature.

⁴Notice that the original Watt-Strogatz model uses K to denote the total number of network neighbors. Hence, each node has $K/2$ network neighbors on each side. Here, k refers to $K/2$.

The probability for each tie to "rewire" is specified in option `prob(float)`. But there is also a second way to generate a small-world network with `nsmall`. Instead of specifying the probability for ties to "rewire" one can specify how many ties should rewire in total. In essence, this rewires ties (from the original ring-lattice) with uniform probability until the desired number of "shortcuts" has been reached. This option selected by specifying the option `shortcuts(#)`, where the number of shortcuts needs to be valid number. The latter mechanism for producing a small-world network has the advantage that the number of rewired ties will always be the same, whereas this number can vary when using the former mechanism. One of the options `prob(float)` or `shortcuts(#)` needs to be specified. By default, a new small-world network is called "small". Furthermore, the command produces a directed network, unless `undirected` is specified. Lastly, one can produce many small-world networks at the same time with option `ntimes(#)`.

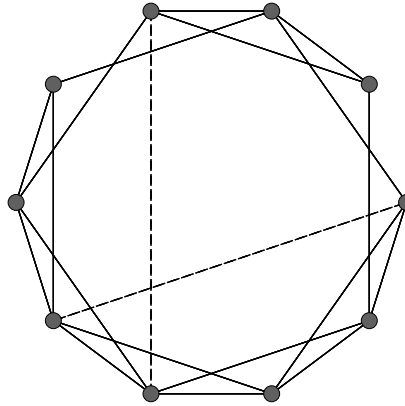


Figure 6.3: Small-world network

Example: Generate small-world network

Let me generate a small network with 10 nodes in total. Furthermore, each node should have $k = 2$ network neighbors on each side in the ring-lattice to begin with. This basically generates the network already displayed in Figure 6.1. But now, I want to explicitly "rewire" 2 ties. This generates a ring-lattice where for two ties the receiving node is changed. Let me furthermore specify the option `undirected` to generate an undirected network. Together the command `nsmall 10, shortcuts(2) undirected` produces the network shown in Figure 6.4. For illustrative purposes, I changed the pattern of the "rewired" ties and also arranged the nodes in a circle as before.⁵

⁵In fact, this graph is easily produced. All that is needed is a second network (e.g. "myring") generated with `nwing 10, k(2) name(myring)`. I can then use the network "myring" to color the edges of network "small" with `nwplot small, edgecolor(myring) layout(circle)`.

```
. set seed 123
. nsmall 10, k(2) shortcuts(2) undirected
. nwplot small, layout(circle)
```

Syntax for generating small-world network

```
nsmall nodes, k(#) prob(float) [undirected ntimes(#) generator_options]

nsmall nodes, k(#) shortcuts(#) [undirected ntimes(#)
generator_options]
```

Options for generating small-world network

`k(#)` specifies the number of neighbors on each side in the ring-lattice.
`prob(float)` specifies the probability for each tie to get "rewired".
`shortcuts(#)` specifies the number of shortcuts, i.e. total number of ties that get "rewired".
`undirected` generates undirected instead of directed network.
`ntimes(#)` generates multiple small-world networks.

6.1.5 Preferential-attachment network

Another well-known algorithm for generating networks is the Barabási-Albert model, which produces scale-free networks with the preferential-attachment mechanism (Barabasi and Albert, 1999). A scale-free network is a network whose degree distribution follows a power law. More precisely this means that the fraction $P(k)$ of nodes with k connections to other nodes asymptotically follows $P(k) \sim k^{-\gamma}$ for large values of k where γ is typically between 2 and 3. Put in words, such scale-free networks are characterized by a few nodes being involved in lots of relationships whereas most nodes have only a small number of network ties.

The principle of preferential attachment means that more connected nodes are also more likely to receive additional network ties. The Barabási-Albert model grows a network based on this principle. The algorithm starts with m_0 nodes who are all connected with each other. New nodes are added one at a time. Every new node connects with probability *prob* to $m < m_0$ already existing nodes uniformly at random. With probability $1 - \text{prob}$ a new node connects to $m < m_0$ already existing nodes, where the probability to connect to a specific node j is proportional to the (in-)degree of node j (the number of ties that j already has).

Example: Generate preferential-attachment network

In this example, I generate a network with 15 nodes. Originally, 3 nodes ($m_0 = 3$) exist and all are connected with each other. After that 12 additional nodes are added one at a time. Each new node has 2 ties ($m = 2$) ties, and these ties connect to other nodes with a probability that is proportional to their current degree. I also ask the network to

be undirected. As you can see the original three nodes are much more likely to receive additional ties for two reasons: 1) Every new node can only connect to the already existing nodes. This gives nodes that are generated early on a higher chance to get selected. 2) In this example, the probability to connect to the existing nodes uniformly at random is set 0. This means with probability 1 a new node connects to the existing nodes with a probability that is proportional to the degree of the already existing nodes. This gives an additional advantage to already highly connected nodes to get selected again. By default, the new network is called "pref" unless option `name(newnetname)` is specified. I also display the *nodeid* of the nodes as labels with the command `nwplot pref, label(_nodeid)`. You can see that nodes with lower *nodeids* tend to have more ties.

```
. set seed 123
. nwpref 15, m0(3) m(2) prob(0) undirected
. nwplot pref, label(_nodeid) layout(circle)
```

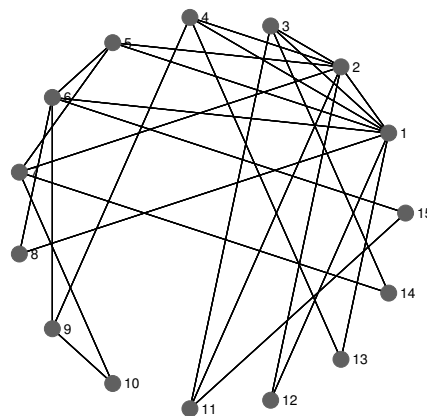


Figure 6.4: Preferential-attachment network

Syntax for generating preferential-attachment network

```
nwpref nodes, [m0(#) m(#) prob(float) undirected ntimes(#)
generator_options]
```

Options for generating small-world network

`m0(#)` specifies the original number of nodes who are all connected with each other; default = 2.

`m(#)` specifies the number of connections that new nodes generate; default = 2.
`prob(float)` specifies the probability for new nodes to connect to the existing ones uniformly at random. With probability $1 - \text{prob}$ new nodes connect to the existing nodes with a probability that is proportional to the (in-)degree of the existing nodes.
`undirected` generates undirected instead of directed network.
`ntimes(#)` generates multiple preferential-attachment networks.

6.1.6 Network with tie probabilities

An extremely useful and general way to generate a network is using heterogeneous tie probabilities saved elsewhere, e.g. another network or a Mata matrix. Remember that in the Erdős-Rényi model each tie has the same probability *prob* to exist. We can easily generalise this model so that the tie between node *i* and *j* has the probability *prob_{ij}*. Again, each tie is evaluated independently of other ties. When *prob_{ij}* = *prob* for all *i* and *j*, this model collapses to the normal Erdős-Rényi model. You can see how one can generate all sorts of networks with this method. One just needs to define (or calculate) the tie probabilities *prob_{ij}* in different ways. The command `nwdyadprob` produces networks like that. The tie probabilities can be specified in either a matrix of another network. First, they can be given in a square Mata matrix *matamatrix*. One would then call `nwdyadprob` with the option `mat(matamatrix)`. The size of the new network is determined by the dimension of *matamatrix*. Notice that the diagonal of *matamatrix* is automatically set to zero to avoid self-loops. When the values in *matamatrix* range from 0 to 1, they are directly interpreted as tie probabilities and each tie in the new network is generated depending on the corresponding probability in *matamatrix*. For example, when *matamatrix*[1,2] = 0.4, there is a 40% chance that the algorithm produces a tie between node 1 and node 2. In contrast, when the values *x_{ij}* in *matamatrix* range from $-\infty$ to ∞ , the probability for a tie between node *i* and *j* is derived as $\text{prob}_{ij} = 1/(1 + \exp(-x_{ij}))$. This logistic function maps each potential tie a probability between 0 and 1. In this case the value *matamatrix*[1,2] = 0 would translate into the probability $1/(1 + \exp(-0)) = 0.5$, i.e. there is a 50% chance for the algorithm to produce a tie between node 1 and 2.

Second, the information about tie probabilities can be given in another network *netname*. In this case, the tie values in *netname* are interpreted in exactly the same way as before. For example, the tie value *netname*[1,2] = 0.4 also means a 40% chance that the algorithm produces a tie between node 1 and node 2 in the new network. As before, when the command detects values smaller than 0 or greater than 1 in *netname* it applies the logistic function to derive tie probabilities.

The number of ties that this algorithm produces depends on the probabilities *prob_{ij}*. However, one can also specify the option `density(float)`, which forces a specific number of ties to be produced. This can be extremely useful when one wants to generate a network with a certain number of ties *m*. Given *n* nodes, the number of ties *m* the algorithm produces is $m = \text{round}(\text{density} \cdot \binom{n}{2})$. The *density* parameter needs to be between 0 and 1. Let me explain a little more what happens when *density* is specified and what happens when it is not. When no *density* parameter is given, `nwdyadprob` simply generates for each potential tie a uniformly distributed random number *r_{ij}* and

compares it with the given tie probability $prob_{ij}$. When $prob_{ij} < r_{ij}$ the algorithm produces a tie from node i to j . When $prob_{ij} \geq r_{ij}$ no tie is produced from node i to j . In contrast, when *density* is given, the command executes unequal probability sampling and samples $m = \text{round}(\text{density} \cdot \binom{n}{2})$ number of ties without replacement from the pool of all potential ties, where each tie has the sampling weight $prob_{ij}$.⁶

Example: Generate network with tie probabilities

Let me show you how a network can be produced from a Mata matrix with tie probabilities. First, I generate the Mata matrix *A*, which is a 4x4 matrix, where each cell gets the value 0.5. I can generate a network based on this matrix with `nwdyadprob, mat(A)`. Each tie has the probability 0.5 to exist. In fact, because all values in *A* are exactly the same, I could have also just written `nrandom 5, prob(.5)`. But here you see the difference to the Erdős-Rényi model. Instead of all tie probabilities being the same, I can define a matrix where cell values differ. Let me define such a matrix *B* (see below). For example, the value $B[2,1] = 0.7$ means there is a 70% chance that the algorithm produces a tie from node 1 to node 2. In contrast $B[1,2] = 0$ means there is a 0 % chance that node 1 sends a tie to node 2. Again, I can produce a network based on this matrix of tie probabilities with `nwdyadprob, mat(B)`. In both cases, this would generate a new network with 4 nodes.

```
. mata: A = J(4,4,0.5)
. mata: A
[symmetric]
      1      2      3      4
1     .5
2     .5     .5
3     .5     .5     .5
4     .5     .5     .5     .5
```

1	.5			
2	.5	.5		
3	.5	.5	.5	
4	.5	.5	.5	.5

```
. mata: B = (0,0,0,0\0.7,0,0,0\0.9,0.6,0,0\0.5,0.9,0.3,0)
. mata: B
      1      2      3      4
1      0      0      .4      0
2      .7      0      0      .8
3      .9      .6      0      0
4      .5      .9      .3      0
```

1	0	0	.4	0
2	.7	0	0	.8
3	.9	.6	0	0
4	.5	.9	.3	0

Let me demonstrate what happens when I specify `density(0.25)`. To show the difference, I first run `nwdyadprob, mat(A) name(netA)`, which generates the network "netA" based on the tie probabilities defined in matrix *A*. Because all tie probabilities in *A* are 0.5, the overall density of the network will be approximately 0.5 as well.⁷ In

⁶In detail, the command transforms the matrix of tie probabilities in an edgelist format. This temporary dataset has three variables, *i*, *j*, *prob*. Then it uses the command `gsample m[aweight=prob], wor` to select *m* ties without replacement. Finally, the selected ties are set to 1 and the edgelist data is transformed back to a network. This also explains why `nwdyadprob` is much faster when no *density* is specified.

⁷Notice that the exact network density can vary because each tie is generated based on a probability.

contrast, when I type `nwdyadprob, mat(A) density(0.25) name(netA_25)`, I generate a network with the exact density of 0.25, in this case 3 ties. However, because the tie probabilities in *A* are the same, each potential tie has the same chance to be among the 3 selected ties.

```
. nwdyadprob, mat(A) name(netA)
. nwdyadprob, mat(A) density(0.25) name(netA_025)
. nwsummarize netA netA_25
```

```
Network name: netA
Network id: 5
Directed: true
Nodes: 4
Arcs: 6
Minimum value: 0
Maximum value: 1
Density: .5
```

```
Network name: netA_25
Network id: 6
Directed: true
Nodes: 4
Arcs: 3
Minimum value: 0
Maximum value: 1
Density: .25
```

Syntax for generating network based on tie probabilities

```
nwdyadprob , mat(matamatrix) [density(float) undirected generator_options]

nwdyadprob netname[, density(float) undirected generator_options]
```

Options for generating network based on tie probabilities

`mat(matamatrix)` specifies the Mata matrix with tie probabilities.

`density(float)` forces the algorithm to produce a network with a pre-defined density.

`undirected` generates undirected instead of directed network.

`ntimes(#)` generates multiple networks.

6.2 Generate networks with network expressions

Another way to generate a network is using *network expressions* and `nwgen`. This is very similar to generating new variables based on other variables, e.g. `generate a = b + c`, but based on other networks. In essence, a *network expression* allows you to calculate with networks. A *network expression* evaluates in the following way. First, it detects the dimensions $n \times n$ of the smallest network inside the expression. Second, all normal constants and variables are expanded to the dimensions of this smallest network. In the case of a constant, a temporary $n \times n$ matrix is generated, where each matrix

cell gets the value of the constant. In the case of a variable, by default, a temporary $n \times n$ matrix is generated, where each column of the matrix has the values of the first n observations of the variable (see also `nwexpand`). For normal networks, the underlying adjacency matrix of the network is used. This leaves an expression with only matrices and operators and functions. After that, each operator and function is executed on an element-by-element basis for the underlying matrices. The result is a $n \times n$ matrix, which is returned as a network.

Let me use the Florentine data with `webnwuse florentine` and demonstrate the use of *network expressions*. A very simple *network expression* creates a new network "mynet" and assigns it the values of another network, e.g. `nwgen mynet = flomarriage`. This is almost like generating a new variable based on another one. After this operation, there is a new network "mynet" in Stata memory, which happens to have the same adjacency matrix as the "flomarriage" network.

```
. nwgen mynet = flomarriage
```

A more complicated *network expression* adds or multiplies the adjacency matrices of different networks with each other (on an element-by-element basis). A reasonable question would be, which Florentine families are related to each other by both marriage and business ties? First, I can tabulate the two networks with each other, just as if I would tabulate two variables. This works with `nwtabulate flomarriage flobusiness`, which gives a cross-table of all tie values found for relationships in the networks "flomarriage" and "flobusiness". For example, there are 7 business relationships that are not complemented by a marriage relationship and 12 marriage relationships that are not complemented by a business relationships. There are, however, 8 pairs of Florentine families who have both a business and a marriage relationship with each other.⁸

```
. nwtabulate flomarriage flobusiness
      Network 1: flomarriage   Directed: false
      Network 2: flobusiness   Directed: false
```

flomarriage	flobusiness		Total
	0	1	
0	93	7	100
1	12	8	20
Total	105	15	120

But which families do have both marriage and business ties? This can be easily addressed with the *network expression* `nwgen overlap = flomarriage * flobusiness`. You can see how this treats networks inside expressions just as if they were variables. This little command generates a new network called "overlap" and assigns it the adjacency matrix that results from evaluating the *network expression* `flomarriage * flobusiness`. As both networks are binary, the resulting network "overlap" takes the value 1 for those relationships where both "flomarriage" and "flobusiness" are 1 and 0 if

⁸Notice that when one of the networks would be valued, there would be more categories than 0 and 1.

one them is 0. Let me run a one-way tabulate of the network "overlap" with `nwtabulate overlap`. This works exactly like the normal `tabulate` command, but displays the frequency of tie values. As expected from the two-way table before, there are 8 ties in the "overlap" network, i.e. 8 pairs of families who share both business and marriage ties.

```
. nwgen overlap = flomarriage * flobusiness
. nwtabulate overlap
  Network:  overlap      Directed: false
  overlap |      Freq.      Percent      Cum.
  -----|-----
         0 |        112        93.33        93.33
         1 |         8         6.67       100.00
  -----|-----
      Total |        120       100.00
```

6.3 Generate networks with network functions

Using `nwgen` together with a *network function* is another way to generate a network. Basically, a *network function* is wrapper for a *network generator*. It works almost exactly like `egen`, but allows some functions that take other networks as arguments. A *network functions* calls the corresponding *network generator* and directly assigns a new name to a network. This is nothing else than a shortcut and introduces a syntax very similar to `egen`, but for networks. The actual work is performed by different programs, which can also be called directly.

For example, there is a function `large(netname)`, which extracts the largest components of *netname* and returns a new network. So, the command `nwgen flolarge = large(flomarriage)` returns a new network "flolarge", which only consists of the nodes in the largest component in "flomarriage". Another example for a *network function* is the command `nwgen mynet = random(10, prob(.2))`, which produces a new (binary) random network with tie probability 0.2, i.e. each tie has the probability 0.2 to exist. Notice how I am using the *network function* `random()` to generate a new random network (see `nwgen`). The same result can also be achieved by calling `nwrandom 10, prob(.2) name(mynet)`, which is the corresponding *network generator*. In a few cases, the actual *network generator* has more options, but generally there is no difference between generating a network through a *network function* or a *network generator*. These are the network functions that generate new networks.

```
duplicate(netname) [, xvars]
```

Duplicate a network (see `nwduplicate`).

```
dyadprob(netname) , density(float) [undirected xvars]
```

Generate a network based on tie probabilities (see `nwdyadprob`).

```
geodesic(netname) [, unconnected(integer) nosym xvars]
```

Generate a network of shortest paths between nodes (see `nwgeodesic`).

`homophily(varname), homophily(float) density(float) [...]`
Generate a homophily network (see `nwhomophily`).

`lattice(rows cols) [, undirected xwrap ywrap xvars]`
Generate a lattice network (see `nwlattice`).

`large(netname)`
Extract the largest component as a network.

`path(netname), ego(nodeid) alter(nodeid) [length(integer) sym xvars]`
Generate a network of paths between nodes (see `nwpath`).

`permute(netname) [, xvars]`
Random permutation of a network (see `nwpermute`).

`pref(nodes) [, m0(integer) m(integer) prob(float) undirected xvars]`
Generate a preferential attachment a network (see `nwpref`).

`random(nodes) [, prob(float) density(float) undirected xvars]`
Generate a random network (see `nwrandom`).

`reach(netname) [, nosym xvars]`
Generate a reachability network (see `nwreach`).

`ring(nodes) , k(integer) [undirected xvars]`
Generate a ring lattice (see `nwring`).

`small(nodes) , k(integer) [prob(float) shortcuts(integer) undirected xvars]`
Generate a small-world network (see `nwsmall`).

`transpose(netname) [, xvars]`
Transpose a network (see `nwtranspose`).

Chapter 7

Expansion and permutation

7.1 Expansion of variable to network

Often, and we are getting to this soon, you want to test hypotheses on networks including certain variables, e.g. are there more ties between individuals who have the same gender? A very useful command in this context is `nwexpand`, which lets you generate networks from variables. This command generates a new network by expanding an existing variable *varname*. When option `nodes(integer)` is unspecified, the command generates a network with `_N` nodes. The value M_{ij} of the adjacency matrix M of the new network is calculated from the values *varname*[i], *varname*[j] and some function *expfcn*, which is defined in option `mode(mode)`. By default, `mode = same`. Other valid modes are: `same`, `dist`, `distinv`, `absdist`, `abdistinv`, `sender`, `receiver`. The new network is called "*expfcn_varname*" unless option `name(newnetname)` is specified.

7.1.1 Example: Expand variable to network

An example demonstrates how this works. First, we generate a small dataset with 6 observations and the new variable `gender`. This new variable takes the value 0 for observations 1-3 and the value 1 for observations 4-6.

```
. nwcLEAR
. set obs 6
. gen gender = (_n > 3)
. list gender
```

	gender
1.	0
2.	0
3.	0
4.	1
5.	1
6.	1

Next, we use `nwexpand gender` to generate a new network from this variable. This generates a new network called "same_gender". By looking closer at the adjacency matrix M of this new network we see how the default `expfcn = same` generated the entries M_{ij} as:

$$M_{ij} = (\text{varname}[i] == \text{varname}[j])$$

```
. nwexpand gender
. nwsummarize same_gender, matonly
[symmetric]
   1   2   3   4   5   6
1   1
2   1   1
3   1   1   1
4   0   0   0   1
5   0   0   0   1   1
6   0   0   0   1   1   1
```

Alternatively, let us select another mode to illustrate the difference. The command `nwexpand gender, mode(dist)` generates a new network called "dist_gender" with the following adjacency matrix:

$$M_{ij} = (\text{varname}[i] - \text{varname}[j])$$

```
. nwexpand gender, mode(dist)
. nwsummarize dist_gender, matonly
   1   2   3   4   5   6
1   0   0   0  -1  -1  -1
2   0   0   0  -1  -1  -1
3   0   0   0  -1  -1  -1
4   1   1   1   0   0   0
5   1   1   1   0   0   0
6   1   1   1   0   0   0
```

Let me give you a third example for a different mode. The command `nwexpand gender, mode(sender)` also generates a network "sender_gender" with 6 nodes, but now the adjacency matrix M is generated as:

$$M_{ij} = \text{varname}[i]$$

```
. nwexpand gender, mode(sender)
. nwsummarize sender_gender, matonly
   1   2   3   4   5   6
1   0   0   0   0   0   0
2   0   0   0   0   0   0
```

3	0	0	0	0	0	0
4	1	1	1	1	1	1
5	1	1	1	1	1	1
6	1	1	1	1	1	1

Generally, expanding variables to networks like this can be extremely useful for many purposes. For example, one can use it to plot the edgecolors of ties differently when two nodes have the same value on some attribute. Or one can use these networks, as we will do shortly, to test hypotheses about homophily in networks, i.e. are ties more likely between nodes who have the same attribute? Table 7.1.1 gives all possible expand modes and the corresponding *expfcn*.

<i>expand_mode</i>	<i>expfcn</i>
same	$(varname[i] == varname[j])$
dist	$(varname[i] - varname[j])$
distinv	$1/(varname[i] - varname[j])$
absdist	$ (varname[i] - varname[j]) $
absdistinv	$1/ (varname[i] - varname[j]) $
sender	$varname[i]$
receiver	$varname[j]$

Table 7.1: Valid expand modes

7.1.2 Syntax for network permutation

`nwexpand varname [ if ] [ , mode(expand_mode) nodes(integer) generator_options ]`

7.1.3 Options for network permutation

`mode(mode)` specifies how variable should be expanded; default = `same`.

`nodes(integer)` number of nodes of the expanded network; default = `_N`.

7.2 Permutation of a network

Another feature that is very useful for hypotheses testing on networks is a *network permutation*. A network permutation can be thought of very easily. What if you could randomly re-allocate your nodes, but keep all the structure constant? Think of this along the lines of re-labelling your network nodes. All you do is scramble the nodes. This gives you a network where nodes get randomly assigned a network position. The structure is exactly the same as before, but your nodes are at a different position within the network.

Such permutations can be extremely useful to find out how a test statistic that involves node attributes would look like if nodes were randomly re-assigned a network position. For example, are there more ties between nodes who are similar? Of course, you could just go ahead and check how many ties there are between similar nodes. You can easily do that with `nwtabulate` using the `attribute(varname)` option. Alternatively, you can also correlate a network with an attribute using `nwcorrelate` (see Section ??). But the next question is, is this a lot and what would we expect to observe anyway for this network? With network permutations you can apply a similar logic as before and test such a hypothesis.

The command `nwpermute` randomly reshuffles the nodes in a network. Notice, that this does not change the overall structure of a network. The structure of a network will remain exactly the same. The only difference is that nodes occupy different positions than they did before. `nwpermute` saves the permutation of a network `netname` as a new network. By default, this new network is called `netname_perm`, unless option `name(newnetname)` is specified.

7.2.1 Example: Network permutation

An example illustrates what `nwpermute` does. Let me generate a small dummy network that is extremely regular. With `nwlattice 3 3, undirected`, I generate an undirected lattice network with 9 nodes (see Section 6.1.3). This lattice network is called "lattice" by default. After that, I generate a permutation of this network with `nwpermute lattice`. By default, this permutation network is called "lattice_perm". Lastly, I plot both networks "lattice" and "lattice_perm" next to each other. As you can see in Figure 7.1, the overall structure of the networks is exactly the same. However, the *nodeids* are different.

```
. nwcLEAR
. nwlattice 3 3, undirected
. nwpermute lattice
. nwset
(2 networks)
-----
      lattice
      lattice_perm
. nwplot lattice, label(_nodeid) labelopt(mlabposition(2) mlabsize(huge)
> mlabcolor(red)) caption("lattice")name(net1)
Calculating node coordinates...
Plotting network...
. nwplot lattice_perm, label(_nodeid) labelopt(mlabposition(2) mlabsize(huge)
> mlabcolor(red)) caption("lattice_perm")name(net2)
Calculating node coordinates...
Plotting network...
. graph combine net1 net2, rows(1)
```

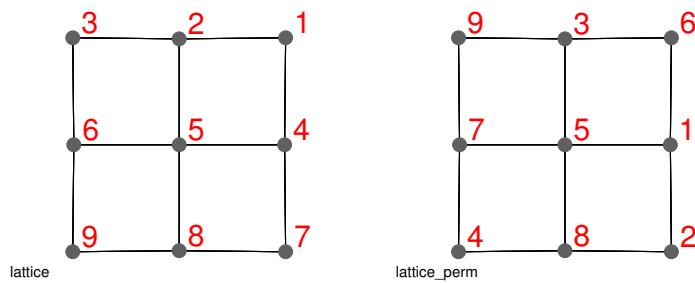


Figure 7.1: Permutation of a network

7.2.2 Syntax for network permutation

```
nwpermute [ netname ] [ , name(newnetname) xvars ]
```

7.2.3 Options for network permutation

name(*newnetname*) specifies the name of the permutation network. By default it is called *netname_perm*.

xvars does not produce Stata variables for the network.

Chapter 8

Hypothesis testing

A major question you might want to answer when you have a network is, is a certain feature of this network likely to be the result of chance or is its presence significant? For example, is there more triadic closure in a network than we would expect by chance?

The problem with networks is that "randomness" does not always equal a score zero. For example, you will get a certain amount of closed triangles in a network simply by chance. And you can see how this differs for different network densities. When a network has a low density (low number of ties) we will observe fewer closed triangles by chance than in a completely random network that has many ties. A very general solution for this problem the following:

1. Calculate your test statistic Z .
2. Calculate your test statistics Z^* for completely random networks that have certain features, e.g. the same density than your original network.
3. Compare your score Z with the scores Z^* and check if it is likely that Z was drawn from the distribution Z^* .

This is a very general strategy that you also use in normal statistics. When you "condition" your network to have certain features we call these conditional uniform graphs. Let us apply this principle to the network "gang" that we load from the internet with `webnwuse gang`, `nwclear`. This is a dataset of co-offending in a London-based youth gang. Data were collected by James Densley (Grund and Densley, 2012, 2014). There are two networks in this data. First, the network "gang.valued" is a valued network, where the tie values mean:

- = 1 (hang out together)
- = 2 (co-offend together)
- = 3 (co-offend together, serious crime)
- = 4 (co-offend together, serious crime, kin)

Second, the network "gang" is a dichotomized network of the "gang_valued" network, where 1 means co-offending (value 2 or above in the "gang_valued" network). One could generate this network with `nwgen gang = (gang_valued >= 2)`. This uses network expressions to generate a new network. Let us ask the following question, is there more or less triadic closure in this network of co-offending than expected? Okay, so how much triadic closure is there in the network "gang" ? We can find this out straightforwardly with `nwsummarize` and specifying the `detail` option or, alternatively, with the `nwtriads` command (see Section 4.1 and 4.2).

```
. webnwuse gang, nwclean
. nwsummarize gang, detail
```

```
Network name: gang
Network id: 2
Directed: false
Nodes: 54
Edges: 133
Minimum value: 0
Maximum value: 1
Density: .092941998602376
Reciprocity: .092941998602376
Transitivity: .3635371179039302
Betweenness centralization: .1066829629553841
Degree centralization: .1973875181422351
```

This gives us a summary of the "gang" network. There are 54 individuals in this network and 133 co-offending ties. The score we are interested is transitivity = 0.363. Is this a lot? To answer this, let us generate 50 random networks with approximately the same number of ties as the "gang" network and calculate the test statistic again. First, I generate random networks with 54 nodes and a density 0.092. This generates random networks with the same number of nodes and ties as the actual "gang" network.

Next, I use the command `nwsummarize _all, detail save(myfile)` to calculate summary statistics for these random networks. Notice how `nwsummarize` accepts the network list `_all` (or any other network list). It displays summary statistics for all random networks. But the command can also be used with the option `save(filename)`, which saves the summary statistics in an external file. In this case, the command saves the summary statistics for the 50 random networks in a file called "myfile.dta" in the working directory. Now we can open this file and plot the distribution of transitivity scores for the random networks with `kdensity`. This gives us an idea about the amount of transitivity we can expect to observe for completely random networks with the same number of nodes and ties as our "gang" network. Furthermore, we can add a vertical line for the observed transitivity of 0.363 with the option `xline(0.363, lpattern(dash))`. This also displays the vertical line dashed.

```
. nwclean
. nwrandom 54, density(0.092) undirected ntimes(50)
. nwsummarize _all, detail save(myfile)

. use myfile, clear
```

```
. kdensity transitivity, xline(0.363, lpattern(dash)) xscale(range(0 .4))
```

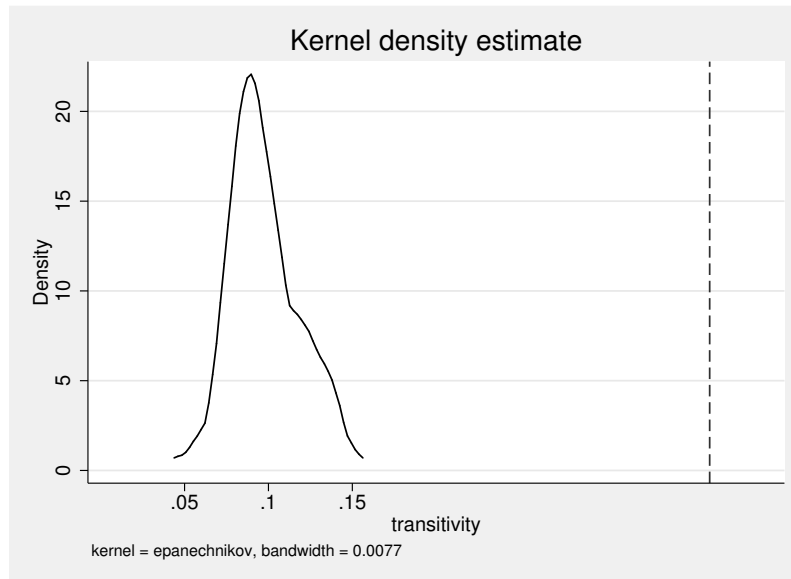


Figure 8.1: Distribution of transitivity in random networks

The result in Figure 8.1 looks pretty obvious. It is extremely unlikely that we would observe a transitivity score of 0.363 by chance for a network with a density of 0.092. Hence, we can conclude that there is more transitivity in the "gang" network than we would expect by chance. Individuals in the "gang" network tend to cluster in groups. And this higher than usual (under conditions of randomness) score for transitivity is significant. In this example, we tested if there is more transitivity than we would expect by chance. But in principal, the same strategy that we used can be applied to all sorts of network statistics, e.g. is the more reciprocity in a network than by chance?

Chapter 9

Tabulate networks

9.1 One-way tabulate of network

Although networks exist as quasi-objects in memory you can interact with them almost as if they were normal variables. And like in normal Stata there is a one-way and a two-way `nwtabulate`. The one-way tabulate of a network shows you how many ties the network has and what the distribution of tie values is. When you type `nwtabulate` followed by a *netname* you get a frequency table of tie values. In fact, internally the command calls `tabulate`, hence, all options from this command are available as well.

For example, let me load the Florentine data from the internet. After that I type `nwtabulate flomarriage` to get a frequency table of tie values in the marriage network. In this case, there are 120 undirected ties in total. This is simply because the network has 16 nodes and $\binom{16}{2} = 120$. Out of these 120 undirected ties, 100 have the value 0, i.e. they do not exist. But there are 20 network ties with value 1.

```
. webnwuse florentine, nwcLEAR
. nwtabulate flomarriage
```

Network: flomarriage	Directed: false		
flomarriage	Freq.	Percent	Cum.
0	100	83.33	83.33
1	20	16.67	100.00
Total	120	100.00	

9.1.1 Syntax for one-way tabulating networks

```
nwabulate [netname] , [selfloop tabulate1-options]
```

9.1.2 Options for one-way tabulating networks

`selfloop` considers self-loops in table.

9.2 Two-way tabulate of two networks

There are also two possibilities for two-way tabulation. The first one is the cross-tabulation of two networks. This is very straightforward. Imagine there are two networks *netname1* and *netname2*. Cross-tabulating these two networks shows you how many ties exist in only one of the networks, how many ties exist in both and how many exist in none. Notice that when your network is valued (ties have more values than just 0 or 1) you will get a table of all possible combinations of tie values. For example, let me cross-tabulate the "flomarriage" and the "flobusiness" network with `nwtabulate flomarriage flobusiness`. Again, you see that there are 120 potential ties in total. Out of these, 93 neither exist in the marriage nor in the business network; 7 undirected ties exist in the "flobusiness", but not in the "flomarriage" network; 12 undirected ties only exist in the "flomarriage" network and 8 undirected ties exist in both networks.

```
. nwtabulate flomarriage flobusiness
      Network 1: flomarriage   Directed: false
      Network 2: flobusiness   Directed: false
```

flomarriage	flobusiness		Total
	0	1	
0	93	7	100
1	12	8	20
Total	105	15	120

9.2.1 Syntax for two-way tabulating networks

```
nwtabulate netname1 netname2 , [plot plotoptions(tabplot_options) eiplot
    eiplotoptions(kdensity_options) permutations(#) tabulate2_options]
```

9.2.2 Options for two-way tabulating networks

`plot` displays tabplot with cell frequencies.

`plotoptions(tabplot_options)` options for tabplot.

`eiplot` displays a plot for significance of EI-index.

`eiplotoptions(kdensity_options)` options for EI-index plot.

9.3 Two-way tabulate of network and variable

Another possibility is to cross-tabulate a network and a variable. In this case, the analysis focuses on those ties that do exist in the network and shows how the attributes of the sender and receiver of these ties are distributed. When a network is undirected, it is unclear who the sender and who the receiver is. Here, the command displays two entries for each edge. The command also calculates the EI-index (Krackhardt and Stern, 1988). The Krackhardt E/I Ratio is a social network measure of the relative density of

internal connections within a social group compared to the number of connections that group has to the external world.

$$EI - index = (E - I)/(E + I)$$

where I (internal) is the number of ties within a social group G and E is the number of ties to the external world (outside of group G). The EI-index ranges between -1 (only within-group ties exist) and 1 (only between-group ties exist). More intuitively, the EI-index simply calculates the number of ties off the diagonal (in the table produced by the command) by the total number of ties. By default, the command runs 100 QAP permutations of the network (see `nwqap`) to obtain a p-value for the EI-index. Basically, the network is randomly permuted and the EI-index is calculated again to obtain a distribution for the EI-index under the condition that the network and the attribute are unrelated.

For example, let me cross-tabulate the network "flomarriage" with the variable `seat` with `nwtabulate flomarriage seat`. First, you can see that the variable `seat` only has the values 0 or 1, which leaves for possible combinations for the attributes of sender and receiver of ties. Either both sender and receiver score 0 on the `seat` variable, only the sender scores 0 and the receiver 1, only the sender scores 1 and the receiver 0, or both sender and receiver score 1. Because the network is undirected we need to divide the values in the table by two. Basically, there are no marriage tie among two families who both did not have a seat in the civic council. In contrast, there are 8 marriage relationships between two families, where one family had a seat and the other one did not. Lastly, there are 12 undirected ties (values on the diagonal need to be divided by two), where both families has seats in the civic council. Such a table can be great to study homophily, i.e. the tendency for ties to form between similar nodes. In our case, the EI-index is negative (pointing towards homophily), but it is not significant.

The option `eipplot` shows how this significance is calculated. Essentially, the command uses the quadratic assignment procedure to obtain the level of significance. By default, the network is permuted 100 times. A permutation of a network is a random re-labelling of the network nodes. The structure of a permuted network remains exactly the same. Only the `nodeid`'s are exchanged. Afterwards, the test-statistic, here the EI-index, is calculated on the permuted networks (while keeping the original node attributes). Such a strategy generates a distribution of EI-indices that one would obtain if one were to randomly re-label network nodes, while keeping the current network structure.

```
. nwtabulate flomarriage seat
      Network: flomarriage      Directed: false
      Attribute: seat

      The network is undirected.
      The table shows two entries for each edge.
```

from_seat	to_seat		Total
	0	1	
0	0	8	8
1	8	24	32
Total	8	32	40

E-I Index: -.2 p-value: .26

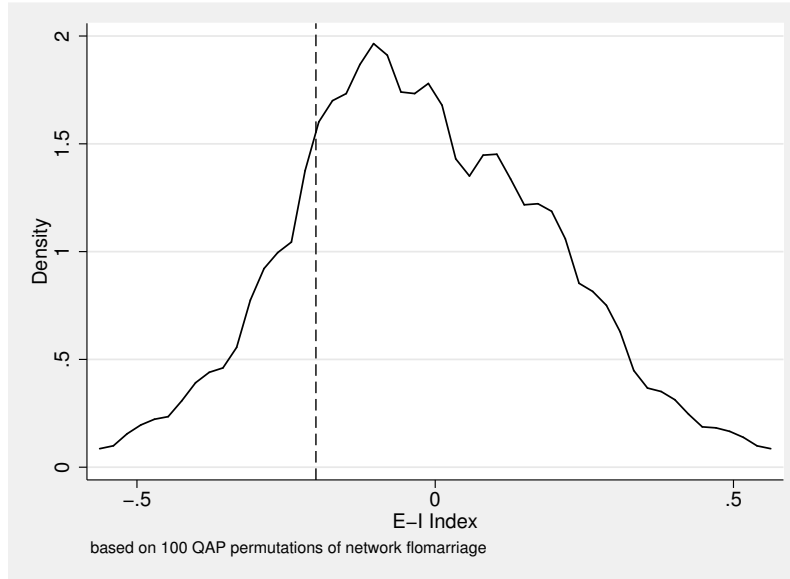


Figure 9.1: EI-index plot

9.3.1 Syntax for two-way tabulating network and variable

```
nwtabulate [netname] varname, [plot plotoptions(tabplot_options) eiplot
    eiplotoptions (kdensity_options) permutations(#) tabulate2_options]
```

9.3.2 Options for two-way tabulating networks

`plot` displays tabplot with cell frequencies.
`plotoptions(tabplot_options)` options for tabplot.
`eiplot` displays a plot for significance of EI-index.
`eiplotoptions(kdensity_options)` options for EI-index plot.
`permutations(#)` specifies the number of QAP permutations for EI-index p-value calculation.

Chapter 10

Correlation with networks

The command `nwcorrelate` is the network version of `correlate`. It can be used in three different ways:

- Correlate nodes of a network
- Correlate two networks with each other
- Correlate one network and one variable

The option `permutation(integer)` creates QAP permutations (see Section 7.2) of the first network and generates a distribution of correlation coefficients under the null-hypothesis that there is no correlation. In practice, rows and columns of *netname* are reshuffled and the correlation coefficient is calculated again and again. Based on this distribution a p-value and a confidence interval is calculated. A plot is displayed and additional information is returned in the return vector.

10.1 Correlate nodes with each other

When the command is used with one *netname* and no `attribute(varname)` option, it correlates the nodes of a network with each other. Such a correlation of nodes, essentially correlates the tie vectors of two nodes with each other. It takes the vector of outgoing, incoming (or both) ties of node *i* and correlates it with the vector of outgoing, incoming (or both) ties of node *j*. The context is specified in `context(context)`. Valid contexts are: `outgoing`, `incoming`, `both`.

10.1.1 Example: Correlate nodes with each other

In this example, I load the Florentine marriage data and calculate node correlations with `nwcorrelate flomarriage`. This generates the new network `"_corr"`. For example, the node correlation between the first node ("Acciaiuoli") and the second node ("Albizzi") is 0.5311. Let me show you how this value is calculated. The network "flomarriage"

is undirected, hence, it does not really matter if we calculate node correlations on the basis of outgoing or incoming ties. The tie vectors for outgoing and incoming ties are exactly the same. Because of this the correlation will be the same as well. Essentially, the value 0.5311 is derived by correlating the tie vectors for node 1 ("Acciaiuoli") and node 2 ("Albizzi") with each other (excluding the observations for node 1 and node 2). Let me do this explicitly. First, I load the network as Stata variables with `nwload flomarriage`. After that I correlate the incoming tie vectors with each other with `correlate acciaiuoli albizzi if _n > 2`.¹ The first and the second node are excluded from this correlation with `if _n > 2`. The resulting correlation is 0.5311.

```
. webnwuse florentine, nwcLEAR
. nwcorrelate flomarriage
  Network name: flomarriage
  Correlation name: _corr
```

```
Context definition:
Average Correlation Between Nodes: -.1395278334336269
. nwvalue _corr[1,2]
  .5310850045
. nwload flomarriage
. correlate acciaiuoli albizzi if _n > 2
(obs=14)
```

	acciai-i	albizzi
acciaiuoli	1.0000	
albizzi	0.5311	1.0000

10.2 Correlate two networks

When two networks *netname1* and *netname2* are given, the command calculates the correlation of the underlying adjacency matrices M_1 and M_2 for the two networks. Notice, that the diagonal of the matrices are not considered. The correlation coefficient for two networks indicates how much these two networks overlap. It is exactly 1 when the two networks completely overlap ($M_1 := M_2$). It is -1 when the two networks are inverse to each other ($M_1 \neq M_2$).

10.2.1 Example: Correlate two networks

Again, an example can demonstrate how this works. Let me correlate the networks "flomarriage" and "flobusiness" in the Florentine data with each other. The command is `nwcorrelate flomarriage flobusiness`.

```
. webnwuse florentine, nwcLEAR
. nwcorrelate flomarriage flobusiness
```

¹Notice that this works because the network is undirected and the tie vectors for outgoing and incoming ties are the same.

```
Network name: flomarriage
Network2 name: flobusiness
```

```
Correlation: .3718678720805482
```

Let me show you how this correlation coefficient comes about. To make this explicit I transform both networks "flomarriage" and "flobusiness" to edgelist format with `nwtoedge`. This generates a new dataset with four variables: `_fromid`, `_toid`, `flomarriage`, `flobusiness`. This new dataset has $16 * 16$ observations, one for each cell in the adjacency matrix of the networks. For example, the second observation in this dataset means that there is no tie from node 1 to node 2 in neither the "flomarriage" nor the "flobusiness" network. For illustrative purposes, I only list the first ten observations in this new edgelist dataset. After that I correlate the variables `flomarriage` and `flobusiness` with each other, excluding the values in the diagonal for which `_fromid == _toid`. And here we are, the correlation coefficient is 0.3719.

As you can see, the correlation coefficient that I derived explicitly is the same as when using `nwcorrelate` directly (except for a different rounding). One option that is extremely useful in this regard is `permutations(integer)`. Essentially, this option uses the quadratic assignment procedure to generate permutations of the first network (see Section 7.2). The correlation coefficient is then calculated again and again for each of the permutations with the second network. This returns a distribution for the correlation coefficient of `netname1` and `netname2` under the condition of randomness (while keeping the structure of the network constant). Such a distribution is then used to derive a p-value for the correlation coefficient. Furthermore, the command generates a plot that shows the correlation coefficients for the QAP permutations and for `netname1` and `netname2`. For example, Figure 10.1 clearly shows that the correlation between the "flomarriage" and the "flobusiness" network is significant. It would be extremely unlikely that we observe a coefficient of 0.37191 by chance.

```
. nwtoedge flomarriage flobusiness
. list _all if _n <= 10
```

	_fromid	_toid	flomar-e	flobus-s
1.	1	1	0	0
2.	1	2	0	0
3.	1	3	0	0
4.	1	4	0	0
5.	1	5	0	0
6.	1	6	0	0
7.	1	7	0	0
8.	1	8	0	0
9.	1	9	1	0
10.	1	10	0	0

```
. correlate flomarriage flobusiness if _fromid != _toid
(obs=120)
```

	flomar-e	flobus-s
flomarriage	1.0000	

```
flobusiness | 0.3719 1.0000
```

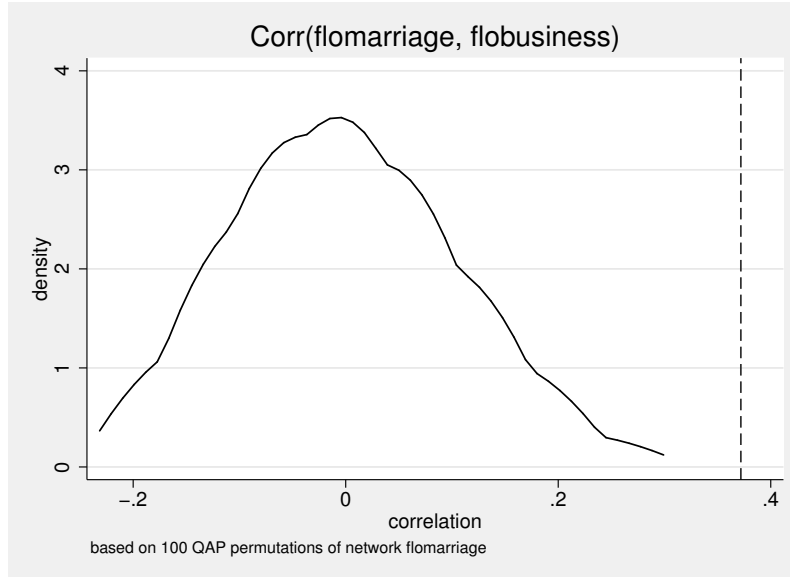


Figure 10.1: Correlation between "flomarriage" and "flobusiness"

10.3 Correlate network with a variable

Lastly, when `nwcorrelate` is called with one *netname* and the option `attribute(varname)` it expands the variable *varname* (see Section 7.1) and correlates this new network with *netname*. By default, the `expand_mode = same` is used, but this can be changed with option `mode(expand_mode)`. All modes from `nwexpand` can be used.

Such dyad-level correlation between network ties and some artificially generated network based on some variable, can be extremely useful to e.g. assess the level of homophily in a network. Homophily refers to the concept that network ties might be more likely between similar individuals. The correlation coefficient between a network and a variable (with `expand_mode = same`) is exactly 1 when ties only exist between individuals who are similar and -1 when ties only exist between individuals who are different when it comes to *varname*. When an attribute is not categorical, but metric instead, it makes a lot of sense to use option `mode(absdist)`.

10.3.1 Example: Correlate network with a variable

In this example, I use the Gang network with `webnwuse gang, nwcLEAR`. In the data, the network "gang" indicates whether two individuals co-offended together or not. Furthermore, the dataset has several node-level attributes saved in normal Stata variables.

For example, there is the variable `Birthplace`, the country of origin of the individual gang members.

```
. webnwuse gang, nwcLEAR
. tab Birthplace
```

Birthplace	Freq.	Percent	Cum.
Caribbean	12	22.22	22.22
East Africa	6	11.11	33.33
UK	24	44.44	77.78
West Africa	12	22.22	100.00
Total	54	100.00	

One question concerning this network is whether there are more co-offending ties between gang members who have the same origin (Grund and Densley, 2012, 2014). Such a question is easily answered with `nwcorrelate`. I simply correlate the network "gang" with the attribute variable `Birthplace` with `nwcorrelate gang, attribute(Birthplace) permutations(100)`. I also add the option `permutations(100)` to generate a p-value and a QAP permutation plot.

```
. nwcorrelate gang, attribute(Birthplace) permutations(100)
```

```
Network name: gang
Attribute: same_Birthplace
```

```
Correlation: .124927833546448
P-value: 0
```

The correlation coefficient is 0.1249 and the p-value indicates that this coefficient is significant. Looking at Figure 10.2 shows the results from the QAP permutations. Remember, in these QAP permutations, the network "gang" gets scrambled 100 times in such a way that nodes (with all their node attributes) are assigned different positions in the network (while keeping the overall structure of the network constant). After that the correlation between the scrambled versions of the "gang" networks and the variable `Birthplace` is calculated. Figure 10.2 clearly indicates that the observed correlation of 0.1249 is significant. It would have been extremely unlikely to find this correlation just by chance.

Again, let me derive the correlation coefficient 0.1249 manually to show you how it is calculated. First, I expand the variable `Birthplace` with the default options (`nwexpand Birthplace`). This generates the new network "same_Birthplace", where the tie values between node i and j indicate whether node i has the same origin as node j (see Section 7.1). After that I transform both networks "gang" and "same_Birthplace" to an edgelist with `nwtoedge gang same_Birthplace`. Again, for illustrative purposes I only list the first 10 observations of this new edgelist dataset. Lastly, the correlation of the variables `gang` and `same_Birthplace` (excluding the diagonal of the network where `_fromid == _toid`) gives the correlation coefficient 0.1249.

```
. nwexpand Birthplace
```

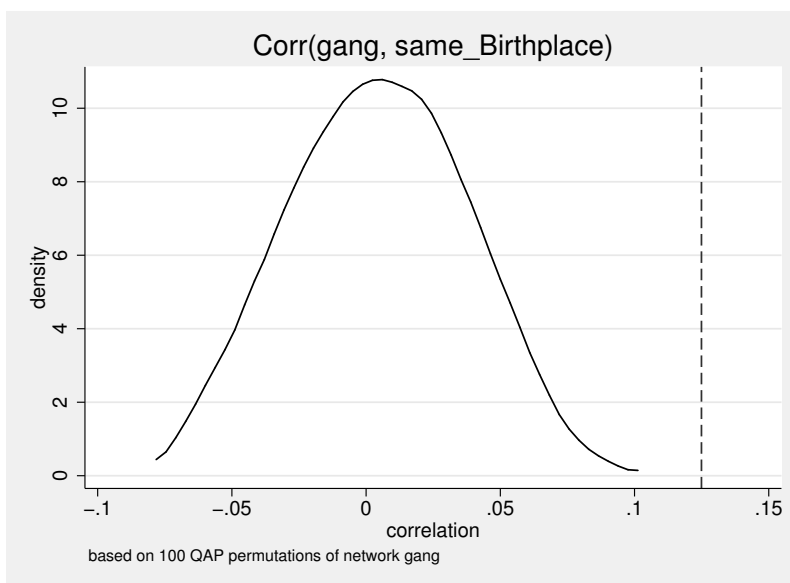


Figure 10.2: Correlation between "flomarriage" and Birthplace

```

. nwds
gang_valued  gang          same_Birth~e
. nwset
(3 networks)
-----
      gang_valued
      gang
      same_Birthplace
. nwtoedge gang same_Birthplace
. list if _n < 10

      _fromid  _toid  gang  same_B~e
-----
1.           1      1      0          1
2.           1      2      0          0
3.           1      3      0          0
4.           1      4      1          0
5.           1      5      0          0
-----
6.           1      6      0          0
7.           1      7      1          0
8.           1      8      1          1
9.           1      9      1          1
-----

. correlate gang same_Birthplace if _fromid != _toid
(obs=1431)

-----
      gang same_B~e
-----

```

gang	1.0000	
same_Birth-e	0.1249	1.0000

10.4 Syntax for correlating nodes

```
nwcorrelate netname [if] [, context(context) name(newnetname)]
```

10.5 Syntax for correlating two networks

```
nwcorrelate netname1 netname2 [if] [, permutations(integer) save(filename)
    kdensity_options]
```

10.6 Syntax for correlating network and variable

```
nwcorrelate netname [if], attribute(varname) [permutations(integer)
    save(filename) kdensity_options]
```

10.7 Options for correlation with networks

`context(context)` determines whether incoming or outgoing ties should be considered when correlating two nodes; default = `both`.

`name(newnetname)` specifies the name of new network with correlations, default = `"_corr"`.

`mode(expand_mode)` specifies how a variable should be expanded; default = `same`.

`permutations(integer)` number of QAP permutations.

`save(filename)` saves QAP permutation results in file.

Chapter 11

Dissimilarities and similarities of nodes

11.1 Dissimilarities of nodes

One way to find out if two nodes i and j have a similar network profile, i.e. are they connected to the same alters, is correlating two nodes (see Section 10.1). This method takes the tie vectors (either based on outgoing, incoming or both kinds of ties) of two nodes and correlates them with each other. Another method is calculating *dissimilarities* between nodes.

The command `nwdissimilar` calculates the dissimilarities between all nodes i and j and saves the result in a new network (just like when nodes are correlated). The dissimilarity between two nodes reflects how dissimilar these nodes are regarding the ties they have to other nodes (tie vectors). By default, the dissimilarity is calculated based on both incoming and outgoing ties (`context(both)`). With `context(incoming)` the dissimilarity between two nodes i and j is calculated only based on the ties they receive (columns). Option `context(outgoing)` only considers outgoing ties (rows) when calculating the dissimilarity between nodes. Practically, option `context(both)` stacks the vectors of outgoing and incoming ties (see Table 11.1.5). There are different ways how dissimilarities can be calculated (see Table ??). By default, the option `type(euclidean)` is selected.

11.1.1 Euclidean distance

The Euclidean distance between two tie vectors is equal to the square root of the sum of the squared differences between them. That is, the strength of actor i 's tie to k is subtracted from the strength of actor j 's tie to k , and the difference is squared. This is then repeated across all the other actors (l, m, n etc.) and summed. Lastly, the square root of the sum is taken. Let Y_{ij} indicate whether there is a tie from node i to node j . For `type(both)` the Euclidean distance between nodes i and j is defined as:

$$D_{ij} = \sqrt{\sum_{k \in V} (Y_{ik} - Y_{jk})^2 + \sum_{k \in V} (Y_{ki} - Y_{kj})^2}$$

11.1.2 Manhattan distance

This distance is simply the sum of the absolute difference between the actor's ties to each alter, summed across the alters.

$$D_{ij} = \sum_{k \in V} |Y_{ik} - Y_{jk}| + \sum_{k \in V} |Y_{ki} - Y_{kj}|$$

11.1.3 Hamming distance

The Hamming distance is the number of entries in the tie vector for one actor that would need to be changed in order to make it identical to the tie vector of the other actor. These differences could be either adding or dropping a tie, so the Hamming distance treats joint absence as similarity.

$$D_{ij} = \sum_{k \in V} (Y_{ik} \neq Y_{jk}) + \sum_{k \in V} (Y_{ki} \neq Y_{kj})$$

11.1.4 Jaccard distance

The Jaccard distance is simply: 1 - Jaccard index. The Jaccard index of two tie vectors A and B is the number of ties that exist in both A and B divided by the total number of ties that exist either in A or in B . When the tie profiles of nodes i and j are exactly the same the Jaccard index equals 1.

$$D_{ij} = \sum_{k \in V} (Y_{ik} \neq 0 \cap Y_{jk} \neq 0) / \sum_{k \in V} (Y_{ik} \neq 0 \cup Y_{jk} \neq 0)$$

11.1.5 Nonmatches distance

Simply gives the percentage of dyads (tie or non-tie) that nodes i and j have in NOT common with the same alters. Notice that i and j are excluded from these alters.

$$D_{ij} = 1 - \frac{\sum_{k \in V} (Y_{ik} == Y_{jk}) + \sum_{k \in V} (Y_{ki} == Y_{kj})}{2(N - 1)}$$

11.1.6 Syntax for node dissimilarities

```
nwdissimilar [netname] [, type(type) context(context) generator_options]
```

<i>type</i>	Description
euclidian	Euclidean distance
manhattan	Manhattan distance
hamming	Hamming distance
jaccard	Jaccard distance
nonmatches	Nonmatches distance

Table 11.1: Types of dissimilarities

<i>context</i>	Description
outgoing	Only outgoing ties
incoming	Only incoming ties
both	Stack outgoing and incoming ties

Table 11.2: Context for dissimilarities

11.1.7 Options for node dissimilarities

type(*type*) specifies how dissimilarity between nodes is calculated; default = **euclidean**.
context(*context*) specifies which ties should be considered; default = **both**.

Chapter 12

Network neighbors

12.1 Network neighbors and context information

Often, when working with networks one wants to know 1) who are the network neighbors of a specific node and 2) what are the attributes of network neighbors. The `nw` commands have two basic commands (`nwneighbor` and `nwcontext`) that are incredibly useful in this regard. `nwneighbor` simply returns all network neighbors for the node specified in option `ego(ego)`, where *ego* can be either a *nodeid* or a *nodelab*. Furthermore, there are different possibilities how the network neighborhood (or context) of a node can be defined. The network neighbors of a node *ego* can be either 1) all other nodes *alter* who receive a tie from *ego*, 2) all other nodes *alter* who send a tie to *ego*, or 3) all other nodes *alter* who either send or receive a tie from *ego*. By default, the first definition is used, but this can be changed with the option `mode(context)` (see Table 12.1).¹

Another useful context-related command is `nwcontext`. It generates a new context variable *newvarname*, which is calculated for each node *i* depending on the values of *i*'s network neighbors on a particular variable *varname* specified in `attribute(varname)`. For example, one might want to know how rich are (on average) the network neighbors of a node? Or how many friends do the friends of nodes have? In essence, `nwcontext` is used with 1) a particular attribute (a normal Stata variable), 2) a context definition, and 3) a statistic that combines the values of network neighbors on the attribute variable. The context can be defined just like before (see also Table 12.1). Again, by default the specification `outgoing` is selected, meaning that the network neighborhood of a node *ego* are all other nodes *alter* who receive a tie from *ego*. Notice that the value of `varname[_n]` is matched to the node with `nodeid == _n`. When a node has more than one network neighbor, one needs to combine the values of these network neighbors on variable *varname* with a *statistic*. One of the most straightforward statistics is the mean, which is also applied by default. In this case, the value of `newvarname[ego]` is derived as the mean of *ego*'s network neighbors values on variable *varname*. However, other statistics can be selected as well (see Table 12.2).

¹When a network is set as "undirected" (either manually or through automatic detection), the context is the same for all three options.

12.2 Example: Obtain network neighbors

For example, in the network "flomarriage" the node with *nodeid* == 2 stands for the "Albizzi" family.² One could ask the question to which other families is the "Albizzi" family directly related by marriage? Questions like this can be easily answered with `nwneighbor`. One would just type `nwneighbor flomarriage, ego(2)`. Alternatively, one can also use the *nodelab* = "albizzi" instead of the *nodeid* == 2 and type `nwneighbor flomarriage, ego(albizzi)`. Depending on whether *ego* is a *nodeid* or a *nodelab* the output is displayed in either *nodeid* or *nodelab* form. However, in both cases all information is saved in the return vector. In fact, the list of neighbors is saved multiple times: 1) the macro `r(neighbors_list1)` is populated with a list of *ego*'s network neighbors saved as *nodeid*, 2) the macro `r(neighbors_list2)` is populated with a list of *ego*'s network neighbors saved as *nodelab*, and 3) the Stata matrix `r(neighbors)` contains the *nodeids* of network neighbors in matrix format. Furthermore, the scalar `r(oneneighbor)` holds the *nodeid* of a randomly selected network neighbor. This can be useful when one wants to select a single network neighbor by chance.

```
. webnwuse florentine, nwcLEAR
. nwneighbor flomarriage, ego(albizzi)
```

```
Network: flomarriage
```

```
      Ego      : albizzi
Neighbors : ginori , guadagni , medici
```

```
. return list
scalars:
      r(ego) = 2
      r(oneneighbor) = 9

macros:
      r(neighbors_list2) : " ginori guadagni medici"
      r(neighbors_list1) : " 6 7 9"

matrices:
      r(neighbors) : 3 x 1
```

12.2.1 Syntax for obtaining network neighbors

```
nwneighbor [ netname ] , ego(nodeid) [ mode(context) ]
```

```
nwneighbor [ netname ] , ego(nodelab) [ mode(context) ]
```

²One can easily find out the *nodelab* of a node with a specific *nodeid* == *i* with `_nwnodeLAB netname , nodeid(i)`. The other way around, given a specific *nodelab* == *j* one can obtain the *nodeid* of a node with `_nwnodeid netname , nodeLAB(j)`.

12.2.2 Options for obtaining network neighbors

`ego(nodeid)` specifies the *nodeid* of the node whose network neighbors should be obtained.

`ego(nodelab)` specifies the *nodelab* of the node whose network neighbors should be obtained.

`mode(context)` defines the network neighborhood as either 1) outgoing (default), 2) incoming, or 3) both (see Table 12.1).

12.3 Example: Calculate context variable

In the next example, I answer the question how rich are (on average) the families that each family is directly connected with by marriage? For each Florentine family this will be a different value depending on their embeddedness in the marriage network "flomarriage" and the wealth of their network neighbors. When we imported the data `florentine` from the internet (with `webnwuse`) we also loaded the Stata variable `wealth`. It holds information about each family's net wealth in the year 1427 (in thousands of lira). Remember that the observation number `_n` is matched with the network node with `nodeid == _n`. As the "flomarriage" network is undirected, there is no need to specify what constitutes the network neighborhood. The network neighborhood of a family comprises simply all other families who are directly related by marriage. Furthermore, in order to obtain the average wealth over all network neighbors there is no need to specify the option `stat(statistic)` as well. By default, `nwcontext` produces the variable `_context.wealth`, unless the option `generate(newvarname)` is specified. To produce the default context variable, I just have to type `nwcontext flomarriage, attribute(wealth)`. But let me show you the results for different statistics. For example, we might want to know how rich is the richest network neighbor of each family? Or, we might want to know how poor is the poorest network neighbor of each family? These questions refer to different ways for combining the attribute values `wealth` over all network neighbors. For the first question, I simply have to specify `stat(max)` and for the second question `stat(min)`. In order to show these differences, I generate the three context variables `wmean`, `wmax`, `wmin` and list these variables together with the node labels. For example, the "Albizzi" family has a wealth of 36. The average wealth of all families that the "Albizzi" family is connected with by marriage is 47.7. The richest network neighbor of the "Albizzi" family has the wealth 103 and the poorest family the "Albizzi"'s are connected with has the wealth 8.

```
. nwcontext flomarriage, attribute(wealth) stat(mean) generate(wmean)
. nwcontext flomarriage, attribute(wealth) stat(max) generate(wmax)
. nwcontext flomarriage, attribute(wealth) stat(min) generate(wmin)
. list _nodelab w*
```

	_nodelab	wealth	wmean	wmax	wmin
1.	acciaiuoli	10	103	103	103
2.	albizzi	36	47.66667	103	8
3.	barbadori	55	61.5	103	20

<i>context</i>	Description
outgoing	Network neighbors of node <i>ego</i> are all nodes <i>alter</i> who receive a tie from <i>ego</i> ; default .
incoming	Network neighbors of node <i>ego</i> are all nodes <i>alter</i> who send a tie to <i>ego</i> .
both	Network neighbors of node <i>ego</i> are all nodes <i>alter</i> who either receive or send a tie to/from <i>ego</i> .

Table 12.1: Definition of network neighborhood

<i>statistic</i>	Description
mean	Mean of <i>varname</i> over network neighbors; default.
max	Maximum of <i>varname</i> over network neighbors.
min	Minimum of <i>varname</i> over network neighbors.
sum	Sum of <i>varname</i> over network neighbors.
sd	Standard deviation of <i>varname</i> over network neighbors.
meanego	Mean of <i>varname</i> over network neighbors and <i>ego</i> .
maxego	Maximum of <i>varname</i> over network neighbors and <i>ego</i> .
minego	Minimum of <i>varname</i> over network neighbors and <i>ego</i> .
sumego	Sum of <i>varname</i> over network neighbors and <i>ego</i> .
sdego	Standard deviation of <i>varname</i> over network neighbors and <i>ego</i> .

Table 12.2: Statistic for combining values of network neighborhood

4.	bischeri	44	67.66666	146	8
5.	castellani	20	83.33334	146	49
6.	ginori	32	36	36	36
7.	guadagni	8	42.5	48	36
8.	lamberteschi	42	8	8	8
9.	medici	103	31	55	10
10.	pazzi	48	10	10	10
11.	peruzzi	49	70	146	20
12.	pucci	3	.	.	.
13.	ridolfi	27	99	146	48
14.	salviati	10	75.5	103	48
15.	strozzi	146	35	49	20
16.	tornabuoni	48	46	103	8

12.3.1 Syntax for calculating context variables

```
nwcontext [netname] , attribute(varname) [mode(context) stat(statistic)
generate(newvarname) mat(string) noweight]
```

12.3.2 Options for calculating context variables

`attribute(varname)` specifies attribute variable for which a context variable should be produced.

`mode(context)` defines the network neighborhood as either 1) outgoing (default), 2) incoming, or 3) both (see Table 12.1).

`stat(statistic)` defines the statistic that is used for combining values over network neighbors; default = mean (see Table ??).

`generate(newvarname)` name of the new context variable; default = `_context_varname`.

`noweight` ignores all tie values.

Chapter 13

Paths between nodes

13.1 Paths and shortest paths

A very common problem when analysing networks is the question are two nodes i and j connected with each other? I already showed how `nwvalue` can be used to find out if there is a direct tie between two nodes (see Section 3.7). But sometimes one wants to know if nodes are indirectly related with each other, i.e. they can be connected with each other through a path in the network. We call the sequence of ties (i, l) , $(l, k) \dots (m, j)$ a path from node i to node j when one gets from node i to node j via some other nodes. The length of a path is determined by the number of ties it has. A direct link between two nodes is also called a path with length = 1. The command `nwpath` gives you all paths between two nodes *ego* and *alter* of a specific length if they exist. In option `ego(nodelab)` or `ego(nodeid)` one specifies the starting point of such a path. In option `alter(nodelab)` or `alter(nodeid)` one specifies the desired end point. When there is no valid path, the command returns nothing. But when there exists one or more paths, they are displayed and saved in the return vector. By default, `nwpath` returns the shortest path to get from *ego* to *alter*, i.e. the fastest way to get from one node to the other. But you can also specify the option `length(#)` to only return paths of another length. For example, in the Florentine marriage network, are the "Medici's" somehow connected to the "Bischeri" family? There is no direct link between the two families, but maybe they are related with each other through some other families. I can get the shortest paths between these two families with `nwpath flomarriage, ego(medici) alter(bischeri)`. Alternatively, I could have also used the *nodeids* of the families to specify the start and end point, e.g. `nwpath flomarriage, ego(9) alter(4)`. After executing this command, one sees that the shortest path between the two families has the length 3. Furthermore, one sees that the selected path length is 3 as well. This simply means that paths of length 3 are shown (we could also show paths with other lengths). And there are indeed three different ways to get from the "Medici" to the "Bischeri" family in the three steps. One path is "Medici" $\Rightarrow$ "Albizzi" $\Rightarrow$ "Guadagni" $\Rightarrow$ "Bischeri". But there are two other paths of the same length as well. A whole lot of information is saved in the return vector as

well. For example, the matrix `r(paths_matrix)` contains all three paths of length 3.

```
. nspath flomarriage, ego(medici) alter(bischeri)
```

```
Network: flomarriage
```

```
Ego           : 9 (medici)
Alter          : 4 (bischeri)
Shortest path length : 3
Selected length   : 3
```

```
Path 1: medici => albizzi => guadagni => bischeri
Path 2: medici => ridolfi => strozzi => bischeri
Path 3: medici => tornabuoni => guadagni => bischeri
```

```
. return list
```

```
scalars:
      r(paths) = 3
      r(path_length) = 3
      r(path_shortest) = 3
      r(ego) = 9
      r(alter) = 4
```

```
macros:
      r(alter_lab) : "bischeri"
      r(ego_lab) : "medici"
```

```
matrices:
      r(paths_matrix) : 3 x 4
```

Let me demonstrate how `nspath` can also be used to obtain paths of different lengths. As mentioned before, by default `nspath` returns the shortest path between two nodes. But sometimes, it can be useful to know if there is a path of a specific length. For example, what are the possible ways to get from the "Medici" to the "Bischeri" family in four instead of three steps? We simply have to specify the option the path length like this `nspath flomarriage, ego(medici) alter(bischeri) length(4)`. You see that there are five possibilities to get from the "Medici" to the "Bischeri" in four steps. Again, all information about the exact paths are saved in the return vector.

```
. webnwuse florentine, nwcLEAR
. nspath flomarriage, ego(medici) alter(bischeri) length(4)
```

```
Network: flomarriage
```

```
Ego           : 9 (medici)
Alter          : 4 (bischeri)
Shortest path length : 3
Selected length   : 4
```

```
Path 1: medici => barbadori => castellani => peruzzi => bischeri
Path 2: medici => barbadori => castellani => strozzi => bischeri
Path 3: medici => ridolfi => strozzi => peruzzi => bischeri
Path 4: medici => ridolfi => tornabuoni => guadagni => bischeri
Path 5: medici => tornabuoni => ridolfi => strozzi => bischeri
```

One can also save the information about each path in network format. With option

`generate(newnetname)`, the command generates a new network for each valid path between *ego* and *alter*. When more than one path exists the networks "newnetname_1", "newnetname_2", and so on are generated. In these new networks, tie values are 1 when a tie is part of a path and 0 otherwise. This can be very useful for plotting paths. For example, one can plot the normal "flomarriage" network and color the edges according to whether they belong to a specific path or not.

13.1.1 Syntax for paths between nodes

```
nwpath [netname], ego(nodeid) alter(nodeid) [length(#) sym
      generate(newnetname)]
```

```
nwpath [netname], ego(nodelab) alter(nodelab) [length(#) sym
      generate(newnetname)]
```

13.1.2 Options for paths between nodes

`ego(nodeid)` specifies the starting point of a path.

`ego(nodelab)` specifies the starting point of a path.

`alter(nodeid)` specifies the end point of a path.

`alter(nodelab)` specifies the end point of a path.

`length(#)` specifies length of paths.

`sym` calculates paths on the symmetrised network.

`generate(newnetname)` saves one network *newnetname_1*, *newnetname_2* ... for each valid path.

13.2 All shortest paths

Often one wants to know for all nodes how many steps does it takes to reach all other nodes. Such information about the (geodesic) distance of one node *i* to all other nodes *j* can be used to generate a distance matrix. `nwgeodesic` calculates the shortest path length between all nodes *i* and *j*, average shortest path length and diameter in a network (length of the longest shortest path). The matrix of distances between nodes is saved in a new network. If nothing is specified this new network is called "geodesic".

By default, the distance between two nodes is calculated for the unvalued and symmetrized network. Option `nosym` can be used to calculate distances between nodes considering the directionality of ties. Sometimes, there is no path between two nodes. In this case, the command assigns the distance -1 by default. Furthermore, non-existent paths are excluded from calculation of average shortest path length. But sometimes one wants to assign a pre-defined distance to a non-existent path. There are two ways to deal with unconnected nodes explicitly. First, the option `length(#)` assigns a fixed distance to all unconnected nodes. Second, the option `length(max)` assigns the maximum existent distance + 1 to unconnected paths. The average shortest path length and the diameter are calculated for the largest component of the network.

For example, let me run `nwgeodesic flomarriage`, which generates a new network "geodesic" with all shortest path lengths. The value `geodesic[9,4] = 3` in this new network indicates that the fastest way to get from node 9 ("Medici") to node 4 ("Bis-cheri") is in three steps.¹

```
. nwgeodesic flomarriage
```

```
Network name: flomarriage
Network of shortest paths: geodesic
```

```
Nodes: 16
Symmetrized : 1
```

```
Paths (largest component) : 105
Diameter (largest component): 5
Average shortest path (largest component): 2.485714285714286
```



```
. nwsummarize geodesic, matonly
[symmetric]
```

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	0															
2	2	0														
3	2	2	0													
4	4	2	3	0												
5	3	3	1	2	0											
6	3	1	3	3	4	0										
7	3	1	3	1	3	2	0									
8	4	2	4	2	4	3	1	0								
9	1	1	1	3	2	2	2	3	0							
10	3	3	3	5	4	4	4	5	2	0						
11	4	3	2	1	1	4	2	3	3	5	0					
12	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	0				
13	2	2	2	2	2	3	2	3	1	3	2	-1	0			
14	2	2	2	4	3	3	3	4	1	1	4	-1	2	0		
15	3	3	2	1	1	4	2	3	2	4	1	-1	1	3	0	
16	2	2	2	2	3	3	1	2	1	3	3	-1	1	2	2	0

13.2.1 Syntax for all shortest paths

```
nwgeodesic [netname] [, unconnected(#) nosym generator_options]
```

13.2.2 Options for all shortest paths

`unconnected(#)` assigns a pre-defined distance to unconnected nodes.

`nosym` calculates geodesic distances on unsymmetrized network.

¹Notice that it does not tell the exact paths anymore, but only the length of the shortest path. If you want to know the exact path use `nwpath`.

Chapter 14

Node centrality

A straightforward question is who are the "important" actors in a network? A large portion of network analysis is concerned with identifying "important" nodes (Bavelas, 1948; Sabidussi, 1966; Freeman, 1979). Different centrality indices are designed to rank the actors according to their position in the network and interpreted as the prominence of actors embedded in a social structure.

14.1 Degree centrality

Probably the most straightforward measure is *degree centrality*. The degree of a node is simply the number of ties a node has, e.g. number of friends. One distinguishes between *indegree centrality* and *outdegree centrality*. The former is the number of ties a node receives and the latter is the number of ties a node sends. When a network is undirected, both in- and outdegree are the same and we simply call it *degree*. Let Y be the adjacency matrix of a binary network, then, Y_{ij} indicates if there is a tie from node i to node j ($Y_{ij} = 1$) or not ($Y_{ij} = 0$). For an undirected network, degree centrality node node i is defined as $C_D(i)$. When the network is directed, outdegree centrality is $C_{Dout}(i)$ and the indegree centrality $C_{Din}(i)$.

$$C_D(i) = \sum_{j=1}^n Y_{ij}$$

$$C_{Dout}(i) = \sum_{j=1}^n Y_{ij}$$

$$C_{Din}(i) = \sum_{j=1}^n Y_{ji}$$

The command `nwdegree netname` generates Stata variables for degree. For undirected networks only one variable for degree is generated; for directed networks the command generates one variable for in- and one for outdegree. Again, the i 'th observation of the new variables corresponds to the node with `nodeid = i`. By default the new variables are called `_degree`, `_out_degree`, `_in_degree`, unless something else is specified in option `generate(newvarlist)`. One can also explicitly ask the command

to produce a variable for isolates, i.e. nodes who are not connected to anybody else. For valued networks one can specify the option `valued`, which calculates node *strength* instead of node *degree*. The *strength* of a node is simply the sum over the tie values of the ties the node is involved in. `nwdegree` not only calculates degree centrality, but also tabulates the distribution of degrees. All sorts of options that one can normally send to `tabulate` one can send to `nwdegree` as well.

Furthermore, the command calculates degree centralization (Freeman, 1978). Centralization captures how unequally distributed individual centrality scores are. A network is highly centralized = 1 when one node has the maximum possible centrality score and all other nodes have the minimum possible centrality score. In contrast, centralization is 0 when all nodes have the same centrality score.

14.1.1 Example: Degree centrality

For example, let me calculate the degree centrality for the Florentine families in the "flomarriage" network. In this case, degree centrality corresponds to the number of other families that a family is connected with by marriage. As it turns out there is one family, which is not connected to any other family; 4 families who have exactly 1 marriage relationship to another family and so on.

```
. webnwuse florentine, nwclear
. nwdegree flomarriage
obs was 0, now 16
```

Network name: flomarriage

Degree distribution			
_degree	Freq.	Percent	Cum.
0	1	6.25	6.25
1	4	25.00	31.25
2	2	12.50	43.75
3	6	37.50	81.25
4	2	12.50	93.75
6	1	6.25	100.00
Total	16	100.00	

Degree centralization:: .2666666666666667

The command also generates the new variable `_degree`, which holds individual degree centrality scores. For example, the "Medici" family has a degree of 6; i.e. they are connected with 6 other families by a marriage relationship. In contrast, the "Pucci" family has a degree of 0. This family is not connected to any other family by marriage ties.

```
. list _nodelab _degree
```

	_nodelab	_degree
1.	acciaiuoli	1
2.	albizzi	3

3.	barbadori	2
4.	bischeri	3
5.	castellani	3
6.	ginori	1
7.	guadagni	4
8.	lamberteschi	1
9.	medici	6
10.	pazzi	1
11.	peruzzi	3
12.	pucci	0
13.	ridolfi	3
14.	salviati	2
15.	strozzi	4
16.	tornabuoni	3

14.1.2 Syntax for degree centrality

```
nwdegree [netlist] [, generate(newvarlist) isolates valued standardize
in(tabulate_opt) out(tabulate_opt) tabulate_opt]
```

14.1.3 Options for degree centrality

`generate(newvarlist)` names of new variables. Default: `_degree`, `_out_degree`, `_in_degree`, `_isolates`.

`isolates` generates a new variable to indicate whether a node is an isolate.

`valued` calculates node *strength* instead of node *degree*.

`standardize` divides the centrality scores by the maximally possible score. After this centrality scores range from 0 to 1.

`in(tabulate_opt)` sends tabulate options to indegree.

`out(tabulate_opt)` sends tabulate options to outdegree.

14.2 Betweenness centrality

Another measure for the importance of network nodes is *betweenness centrality* (Freeman, 1977; Brandes, 2008). It captures the idea that important nodes are important for connecting other nodes among each other. The betweenness centrality of a node is defined as the number of shortest paths among all other nodes (see Section 13.1) that pass through this node. When two nodes can be connected with each other by more than one shortest path, each path is considered proportionally when calculating betweenness centrality. Nodes with high betweenness centrality are intermediaries who matter a lot for the connections between other nodes. They have a large influence on whatever is transferred through a network. Formally, the betweenness centrality of node i is defined as

$$C_B(i) = \sum_{s \neq i \neq t} \frac{\sigma_{st}(i)}{\sigma_{st}}$$

where σ_{st} is the total number of shortest paths from node s to node t and $\sigma_{st}(i)$ is the number of those paths that pass through node i . The command `nwbetween` implements betweenness centrality as proposed in Brandes (2001). It is used straightforwardly, followed by a *netname*. Alternatively, one can also call the command followed by *netlist*. In this case, all calculations are performed for all networks in the list. By default, the command generates (and overwrites) a new variable `_between`, where `_between[i]` holds the betweenness centrality score for node with *nodeid* = i .¹ With option `generate(newvar)` the centrality scores can be saved in another variable. The command also performs all calculations on the symmetrized network, unless option `nosym` is specified. Lastly, the option `standardize` rescales the centrality scores and divides the resulting coefficients by $(N-1)(N-2)$ for directed networks and $(N-1)(N-2)/2$ for undirected networks. These are the maximally possible scores for unvalued networks. In consequence, the standardized betweenness centrality scores will range between 0 and 1 (for unvalued networks).

14.2.1 Example: Betweenness centrality

In this example, I use the Florentine data from the internet. After that I type `nwbetween flomarriage` to calculate betweenness centrality scores for all nodes in the "flomarriage" networks. This command also generates and overwrites the Stata variable `_between`. After that, I simply list this new variable together with the variable `_nodelab`. For example, the "Medici" family has the highest betweenness centrality. There are 95 shortest paths that go through the "Medici" family

```
. webnwuse florentine, nwcLEAR
```

```
. nwbetween flomarriage
```

Network name: flomarriage

Betweenness centrality

Variable	Obs	Mean	Std. Dev.	Min	Max
_between	16	19.5	24.60111	0	95

```
. list _nodelab _between
```

	_nodelab	_between
1.	acciaiuoli	0
2.	albizzi	38.66667
3.	barbadori	17
4.	bischeri	19
5.	castellani	10
6.	ginori	0

¹When the command is called with a *netlist*, it generates several new variables.

7.	guadagni	46.33333
8.	lamberteschi	0
9.	medici	95
10.	pazzi	0
11.	peruzzi	4
12.	pucci	0
13.	ridolfi	20.66667
14.	salviati	26
15.	strozzi	18.66667
16.	tornabuoni	16.66667

14.2.2 Syntax for betweenness centrality

```
nwbetween [netlist] [, generate(newvarlist) nosym standardize]
```

14.2.3 Options for betweenness centrality

`generate(newvarlist)` names of new variables. Default = `_degree`, `_out_degree`, `_in_degree`, `_isolates`.

`nosym` calculates scores on unsymmetrized network.

`standardize` divides the centrality scores by the maximally possible score. After this centrality scores range from 0 to 1.

14.3 Eigenvector centrality

Eigenvector centrality is a measure of the influence of a node in a network. It assigns relative scores to all nodes in the network based on the concept that connections to high-scoring nodes contribute more to the score of the node in question than equal connections to low-scoring nodes.

The eigenvector centrality of a node is calculated based on the eigenvector centrality of other nodes. It is defined as:

$$C(i) = \frac{1}{\lambda} \sum_{t \in M(i)} C(t)$$

where $M(i)$ is the set of network neighbors of node i and λ is a constant. Mathematically, coefficients can be obtained by finding eigenvalues. There will be many different solutions, but only the highest eigenvalue returns numbers in the eigenvector that are all positive. Eigenvector centrality is only defined for connected networks.

14.3.1 Example: Eigenvector centrality

Again, let me use the Florentine marriage network in this example. As we found out above, the network has one isolate (see Section 14.1.1). The "Pucci" family is not

connected to any other network. Hence, eigenvector centrality cannot be calculated for the whole network. One strategy is to extract the largest component of the network with `nwgen flomlarge = large(flomarriage)`. This command makes use of a *network function* (see Section 6.3) to extract the largest component and saves it in the network "flomlarge". This network only has 15 instead of 16 nodes. It is the largest set of nodes who are all connected with each other somehow and all the relations between these nodes from the original network. After that I can calculate eigenvector centrality of the nodes in this sub-network straightforwardly with `nwevcent flomlarge`.

```
. webnwuse florentine, nwcLEAR
. nwgen flomlarge = large(flomarriage)
. nwevcent flomlarge
```

```
Network name: flomlarge
```

Eigenvector centrality					
Variable	Obs	Mean	Std. Dev.	Min	Max
_evcent	15	.2335078	.1140526	.0448134	.4303081

```
. list _nodelab _evcent
```

	_nodelab	_evcent
1.	acciaiuoli	.1321543
2.	albizzi	.2439561
3.	barbadori	.2117053
4.	bischeri	.2828001
5.	castellani	.2590262
6.	ginori	.0749227
7.	guadagni	.2891156
8.	lamberteschi	.0887919
9.	medici	.4303081
10.	pazzi	.0448134
11.	peruzzi	.2757304
12.	ridolfi	.3415526
13.	salviati	.1459172
14.	strozzi	.3559805
15.	tornabuoni	.3258423
16.		.

14.3.2 Syntax for eigenvector centrality

`nwevcent` [*netlist*] [, generate(*newvar*) nosym]

14.3.3 Options for eigenvector centrality

generate(*newvar*) names of new variable. Default = `_evcent`.

`nosym` calculates scores on unsymmetrized network.

14.4 Closeness centrality

Another measure for centrality of network nodes is closeness centrality. This measure is based on the distance between nodes (see Section 13.2; `nwgeodesic`). Nodes with high closeness centrality are close to all other nodes, that is, they can reach all other nodes in only a few steps. In contrast, nodes with low closeness centrality are far away from all other nodes. Let $d(i, j)$ be the shortest distance between nodes i and j . Then, one can calculate *farness* as the sum of steps it takes to reach all other nodes. Based on this one can calculate *nearness* as the inverse of *farness*. Nodes with high nearness are close to all other nodes. Lastly, closeness centrality standardizes *nearness* by multiplying it with $(N - 1)$, where N is the number of nodes in the network.

$$Farness(i) = \frac{1}{\sum_{j \neq i \in V} d(i, j)}$$

$$Nearness(i) = \frac{1}{Farness(i)}$$

$$C(i) = (N - 1)Nearness(i)$$

This score is exactly 1 when a node is directly connected to all other nodes. Closeness centrality is only defined for connected networks, i.e. all nodes can reach all other nodes in the network. Often, however, one wants to calculate closeness centrality also for unconnected networks. One strategy is to assign a specific distance to unconnected nodes.

The command `nwcloseness` calculates closeness centrality for nodes in a network. By default, the distances between nodes are calculated on the symmetric network unless option `nosym` is specified. Furthermore, the command does not calculate centrality scores for unconnected networks, i.e. some nodes cannot be reached by all other nodes. However, in this case, one can specify `unconnected(integer)` and assign a specific value to undefined distances. An other alternative is to assign a value to missing distances, which is just one more than the maximum distance length that is observed in the network otherwise. This is what option `unconnected(max)` does. With option `generate(var1 var2 var3)` one can overwrite the variables that the command generates. The default variables are `_closenss`, `_farness`, `_nearness`.

14.4.1 Example: Closeness centrality

First, let me calculate closeness centrality for the nodes in the Florentine marriage data with `nwcloseness flomarriage`. This generates three new variables: `_closenss`, `_farness`, `_nearness`. But all three variables are set to missing. This is because the

network is not connected. As we have shown earlier (see Section 14.1.1) there is one isolate in the "flomarriage" network. The "Pucci" family is not connected to any other family. But this means also that for none of the nodes closeness centrality can be calculated because the distance to the "Pucci" family is undefined. Strictly speaking closeness centrality is not defined for this network. Missing distances can be assigned a specific value with the option `unconnected(integer)`. An other alternative is to assign a value to missing distances, which is just one more than the maximum distance length that is observed in the network otherwise. This is what option `unconnected(max)` does. After specifying this option the command `nwcloseness flomarriage, unconnected(max)` generates closeness centrality scores for all nodes in the network.

```
. webnwuse florentine, nwcLEAR
. nwcloseness flomarriage
Warning: network flomarriage not connected; specify unconnected() to obtain results.
```

Network name: flomarriage

Closeness centrality					
Variable	Obs	Mean	Std. Dev.	Min	Max
_closeness	0				
_farness	0				
_nearness	0				

```
. nwcloseness flomarriage, unconnected(max)
```

Network name: flomarriage

Closeness centrality					
Variable	Obs	Mean	Std. Dev.	Min	Max
_closeness	16	.363019	.0764849	.1666667	.483871
_farness	16	43.875	13.81243	31	90
_nearness	16	.0242013	.005099	.0111111	.0322581

14.4.2 Syntax for closeness centrality

```
nwcloseness [netlist] [, generate(newvarlist) unconnected(integer) nosym ]
```

```
nwcloseness [netlist] [, generate(newvarlist) unconnected(max) nosym ]
```

14.4.3 Options for closeness centrality

`generate(newvarlist)` names of new variables. Default = `_closeness`, `_farness` `_nearness`.

`unconnected(integer)` defines distance between unconnected nodes.

`unconnected(max)` defines distance between unconnected nodes as the maximum distance found in the network plus one.

`nosym` calculates scores on unsymmetrized network.

Chapter 15

Node-level characteristics

15.1 Clustering coefficient

The *local clustering coefficient* of node i is defined as the proportion of links between the vertices within its neighbourhood divided by the number of links that could possibly exist between them (Watts and Strogatz, 1998). The clustering coefficient of an isolate node is defined as 0. For directed networks, the local clustering coefficient of node i is calculated as:

$$C(i) = \frac{|\{e_{jk} : v_j, v_k \in N_i, e_{jk} \in E\}|}{k_i(k_i - 1)}$$

For undirected networks, the local clustering coefficient is calculated as:

$$C(i) = \frac{2|\{e_{jk} : v_j, v_k \in N_i, e_{jk} \in E\}|}{k_i(k_i - 1)}$$

Figure 15.1 shows three example graphs. The red dotted lines are non-existent ties. In the first graph, the clustering coefficient of the grey node is 1. All network neighbors of the grey node are connected with each other as well. In the second graph, the clustering coefficient of the grey node is 1/3. Only one out of three possible pairings of network neighbors are connected among each other. Lastly, in the third graph, the clustering coefficient of the grey node is 0. None of the grey node's network neighbors are connected with each other.

The coefficients for each node can be summarized to the network-level measure. The **average clustering coefficients** is simply the average of all local clustering coefficients for the nodes in a network.

$$C_{Avg} = \frac{1}{N} \sum_{i \in V} C(i)$$

Another network-level measure for clustering in a network is the *overall clustering coefficient*. It counts the number of closed triplets and divides it by the number of connected triplets. A connected triplet is defined to be a connected subgraph consisting of three vertices and two edges. Thus, each triangle forms three connected triplets. A closed triplet is also a connected triplet, but here, all three nodes are directly connected with each other.

$$C_{Ov} = \frac{\text{number of closed triplets}}{\text{number of connected triplets}}$$

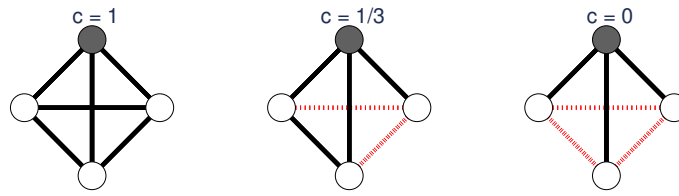


Figure 15.1: Clustering coefficient

15.1.1 Example: Clustering coefficient

The command `nwclustering` calculates the local clustering coefficient for each node and saves it in the variable `_cluster` unless option `generate(varname)` is specified. Let me load the Florentine data and compute the clustering coefficient for the network "flomarriage" with `nwclustering flomarriage`. The average and the overall clustering coefficients are displayed. Furthermore, the command generates the variable `_clustering`. For example, the local clustering coefficient of the "Bischeri" family is 0.33, i.e. one third of all the possible ties between network neighbors of the "Bischeri" family do exist.


```
. webnwuse florentine, nwcLEAR
. nwclustering flomarriage
```

Network name: flomarriage

Average clustering coefficient: .15
 Overall clustering coefficient: .1914893617021277

```
. list _nodelab _clustering
```

	_nodelab	_clust-g
1.	acciaiuoli	0
2.	albizzi	0
3.	barbadori	0
4.	bischeri	.3333333
5.	castellani	.3333333
6.	ginori	0
7.	guadagni	0
8.	lamberteschi	0
9.	medici	.0666667
10.	pazzi	0
11.	peruzzi	.6666667
12.	pucci	0
13.	ridolfi	.3333333
14.	salviati	0
15.	strozzi	.3333333
16.	tornabuoni	.3333333

When Stata variable *varname* already exists, it is overwritten. In case, clustering coefficient is calculated for *z* networks at the same time (e.g. `nwclustering glasgow1 glasgow2`), the command generates the variables *varname_z* for each network.

15.1.2 Syntax for clustering coefficient

```
nwclustering [netlist] [, generate(newvarname)]
```

15.1.3 Options for clustering coefficient

`generate(newvarname)` saves the local clustering coefficient in variable *newvarname*;
 default = `_clustering`.

15.2 Component membership

Components of a graph are sub-graphs that are connected within, but disconnected between sub-graphs. If a graph contains one or more "isolates," these actors are components. The command `nwcomponents` calculates the components of a network or a list

of networks. A component is a set of nodes that are only connected among each other. All calculations are performed on the non-directed version of the networks. Nodes can only belong to one component.

By default, `nwcomponents` generates a new variable `_components`, which stores the component membership unless option `generate(varname)` is specified. When option `lgc` is specified, the command generates a new variable `_lgc` which stores information about membership to the largest component.

15.2.1 Example for network components

Again, I use the Florentine data and calculate the components of the network "flomarriage" with `nwcomponents flomarriage`. In this case, 15 nodes belong to the component "1" and one node belongs to the component "2". Notice that the numbering of these components does not necessarily correspond to their size. When looking at the variable `_component` that the command generates in detail, one sees that only the "Pucci" family belongs to component "2". All other nodes are connected among each other.

```
. webnwuse florentine, nwclean
. nwcomponents flomarriage
```

Network name: flomarriage			
Components: 2			
_component	Freq.	Percent	Cum.
1	15	93.75	93.75
2	1	6.25	100.00
Total	16	100.00	

```
. list _nodelab _component
```

	_nodelab	_component
1.	acciaiuoli	1
2.	albizzi	1
3.	barbadori	1
4.	bischeri	1
5.	castellani	1
6.	ginori	1
7.	guadagni	1
8.	lamberteschi	1
9.	medici	1
10.	pazzi	1
11.	peruzzi	1
12.	pucci	2
13.	ridolfi	1
14.	salviati	1
15.	strozzi	1
16.	tornabuoni	1

15.2.2 Syntax for network components

`nwcomponents` [*netlist*] [, `generate(newvarname)`]

15.2.3 Options for network components

`generate(newvarname)` saves the component membership of nodes in variable *newvarname*; default = `_component`.

`lgc` saves membership to largest component in variable.

Chapter 16

Visualization of networks

16.1 Schemes for networks

A scheme specifies the overall look of a graph. Essentially, a scheme consists of a collection of pre-defined colors, symbols, patterns and other properties (e.g. thickness of lines, size and color of titles, position of legends, and so on) of graphs. All of these properties can also be changed manually, but using schemes has the advantage that a lot of properties of a graph can be changed effortlessly at the same time. Stata already comes with a set of schemes, e.g. "s1color", "s2color", "s1mono", "s2mono", "sj". Each one of these defines a set of default colors and other properties. The *nwcommands* use normal Stata two-way plots to visualize networks. Therefore, all normal (and user-written) schemes can be used straightforwardly. Furthermore, the *nwcommands* come with three additional schemes that are optimized for plotting networks: "s1network", "s2network", and "s3network" (see Figures 16.1, 16.2 and 16.3). These schemes are automatically available after you installed the *nwcommands*.

There are two ways how one can use schemes in Stata. First, one can change the default scheme that Stata uses o making plots. For example, `set scheme s1network` changes the default scheme to "s1network", which is optimized for displaying networks. Second, all commands for plotting networks have the option `scheme(schemename)`. Specifying this option tells Stata to use a particular scheme only once without changing the default scheme.

```
. set scheme s1network
```

16.2 Plot networks

One major feature of this software is the ability to make powerful visualizations of networks. An earlier attempt to plot networks in Stata is the `netplot` command written by Rense Corten (Corten, 2011). The *nwcommands* come with a much more advanced plotting engine for networks in Stata. The command is very simple in usage (much simpler than for example similar commands in R). It builds on twoway plots and accepts

s1network					
Value	Color	Symbol	Edgecolor	Background	
default	 red	 circle	 gs10	 white	
1	 red	 circle	 gs10		
2	 blue	 diamond	 blue		
3	 green	 square	 green		
4	 orange	 triangle	 red		
5	 cyan	 smcircle	 sand		
6	 magenta	 smdiamond	 cyan		
7	 yellow	 smsquare			
8	 khaki	 smtriangle			
9	 sienna				
10	 emidblue				
11	 emerald				
12	 brown				
13	 erose				
14	 gold				
15	 bluishgray				

Figure 16.1: s1network scheme

s2network					
Value	Color	Symbol	Edgecolor	Background	
default	 gold	 circle	 gs12	 black	
1	 gold	 circle	 gs12		
2	 red	 diamond	 red		
3	 green	 square	 gold		
4	 blue	 triangle	 magenta		
5	 erose	 smcircle			
6	 eltblue	 smdiamond			
7	 white	 smsquare			
8	 khaki	 smtriangle			
9	 sienna				
10	 emidblue				
11	 emerald				
12	 brown				
13	 erose				
14	 gold				
15	 bluishgray				

Figure 16.2: s2network scheme

s3network						
Value	Color	Symbol	Edgecolor	Background		
default	 230 205 180	 circle	 black	 white		
1	 230 205 180	 circle	 red			
2	 225 180 140	 diamond	 gold			
3	 220 150 85	 square				
4	 220 125 50	 triangle				
5	 200 100 25	 smcircle				
6	 165 80 10	 smdiamond				
7	 lavender	 smsquare				
8	 khaki	 smtriangle				
9	 sienna					
10	 emidblue					
11	 emerald					
12	 brown					
13	 erose					
14	 gold					
15	 bluishgray					

Figure 16.3: s3network scheme

all normal *twoway-options* (e.g. title, notes, scheme). Notice that it can be used together with a normal *if* condition to plot only sub-networks. Furthermore, it has lots of options by itself to change for example the size, color, symbol of nodes or the color and pattern of edges. One can also change the look of arrows and much more. The command also comes with a set of algorithms to position network nodes. By default, multidimensional scaling is used, but other options e.g. circle or grid layout are available as well. One can also use pre-defined coordinates (saved in normal Stata variables) for assigning specific positions to nodes. In fact, there is even the possibility to write and add own layout algorithms. There are so many options available for **nwplot** that I cannot describe them here all. Basically, the command layers normal scatter, line and pcarrow plots on the top of each other. And each element in these normal twoway plots can be changed, like in any normal twoway plot.

16.2.1 Syntax for plotting networks

```
nwplot [netname] [if] [, node-options label-options edge-options arrow-options
      layout-options other-options twoway-options]
```

16.2.2 Example: Plotting networks

Let me introduce a few options for plotting networks (but there is much more in the help-file). First, I run **set scheme s1network** to use the "s1network" scheme that

<i>node_options</i>	Description
<code>size(varname[,node_sub])</code>	Change the size of the nodes
<code>color(varname[,node_sub])</code>	Change the color of the nodes
<code>symbol(varname[,node_sub])</code>	Change the symbol of the nodes
<code>nodefactor(float)</code>	Multiplies all node sizes by factor

Table 16.1: Node options for `nwplot`

<i>node_sub</i>	Description
<code>norescale</code>	No automatic rescale
<code>legendoff</code>	No legend for this attribute
<code>forcekeys(int...)</code>	List with keys to be used in the legend
<code>colorpalette(colorstyle...)</code>	Overwrite color scheme
<code>symbolpalette(symbolstyle...)</code>	Overwrite symbol scheme
<code>foreground(int...)</code>	Plot these values in the foreground
<code>mlcolor(colorstyle)</code>	Color of line around nodes
<code>mlwidth(linewidthstyle)</code>	Thickness of line around nodes

Table 16.2: Node suboptions for `nwplot`

<i>label_options</i>	Description
<code>lab</code>	Display node labels saved with network
<code>label(varname)</code>	Display node labels saved in variable
<code>labelopt(marker_label_options)</code>	Look-and-feel of node labels (e.g. color, size)

Table 16.3: Label options for `nwplot`

<i>edge_options</i>	Description
<code>edgesize(netname[,edge_sub])</code>	Change the size of the edges
<code>edgecolor(netname[,edge_sub])</code>	Change the color of the edges
<code>edgefactor(float)</code>	Multiply all edge sizes by factor

Table 16.4: Edge options for `nwplot`

<i>edge_sub</i>	Description
<code>legendoff</code>	No legend for this attribute
<code>forcekeys(int...)</code>	List with keys to be used in the legend
<code>edgecolorpalette(colorstyle...)</code>	Overwrite color scheme for edges
<code>edgepatternpalette(pattern...)</code>	Overwrite pattern scheme for edges
<code>foreground(int...)</code>	Plot these values in the foreground

Table 16.5: Edge suboptions for `nwplot`

<i>arrow_options</i>	Description
<code>arcstyle(arcstyle)</code>	Change the look of arcs (curved, straight, automatic)
<code>arcbend(float)</code>	Control the degree of bend for curved arcs
<code>arcsplines(int)</code>	Resolution of curved arcs
<code>arrowfactor(float)</code>	Multiply size of arrowheads by a factor
<code>arrowgap(float)</code>	Control the gap between arrowheads and nodes
<code>arrowbarbfactor(float)</code>	Control look of arrowhead

Table 16.6: Arrow options for `nwplot`

<i>arcstyle</i>	Description
<code>automatic</code>	Plot arcs as curved and straight lines
<code>curved</code>	Plot all arcs as curved lines only
<code>straight</code>	Plot all arcs as straight lines only

Table 16.7: Arc-styles for `nwplot`

<i>layout_options</i>	Description
<code>layout([layoutstyle] [, layout_sub])</code>	Change the layout for coordinates
<code>nodecy(xvar yvar)</code>	Use node coordinates in variables
<code>generate(newxvar newyvar)</code>	Export coordinates of nodes to variables

Table 16.8: Layout options for `nwplot`

<i>layoutstyle</i>	Description
<code>mds</code>	Multidimensional scaling
<code>mdsclassical</code>	Classical multidimensional scaling
<code>circle</code>	Position nodes on circle
<code>grid</code>	Position nodes on grid
<code>nodexy</code>	Use coordinates in <code>nodexy()</code>
<code>_layoutfunction</code>	Advanced user-written layout function

Table 16.9: Layout styles for `nwplot`

<i>layout_sub</i>	Description
<code>lgc</code>	Only plot largest component
<code>iterations(int)</code>	Number of iterations for MDS
<code>columns(int)</code>	Number of columns for grid-layout
<code>norescale</code>	Do not rescale coordinates

Table 16.10: Layout suboptions for `nwplot`

<i>other_options</i>	Description
<code>aspectration(float)</code>	Height/width ratio
<code>lineopt(options)</code>	Send options to all line plots
<code>scatteropt(options)</code>	Send options to all scatter plots
<code>legendopt(options)</code>	Send options to the legend

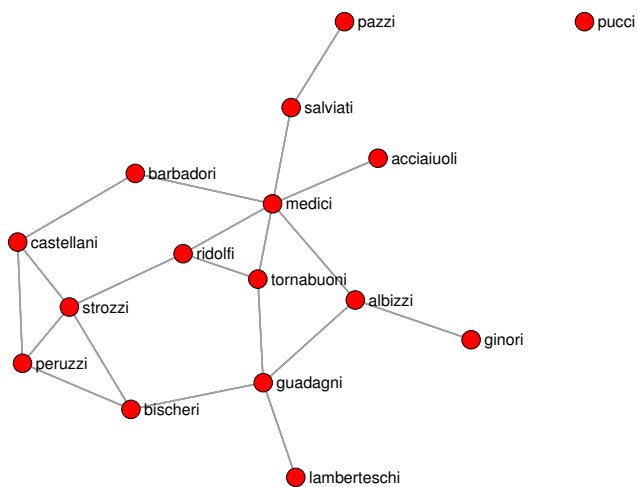
Table 16.11: Other options for `nwplot`

Figure 16.4: Florentine marriage network

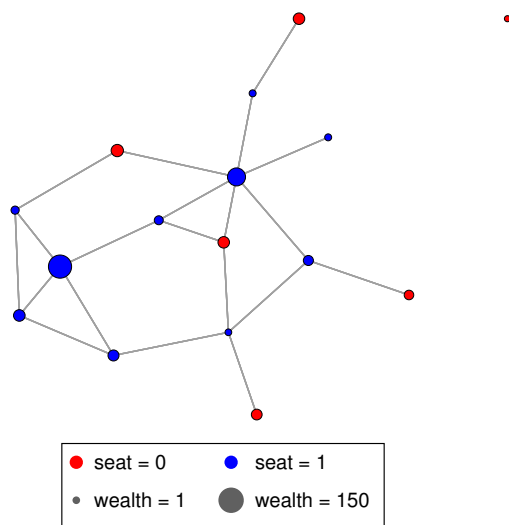


Figure 16.5: Florentine marriage network, wealth and seat attributes

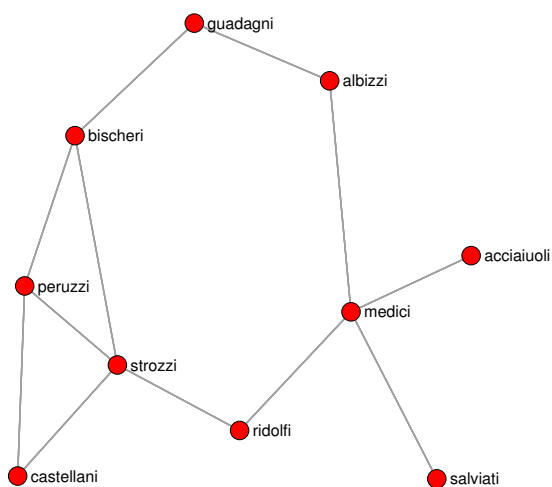


Figure 16.6: Florentine marriage network, only families with seat

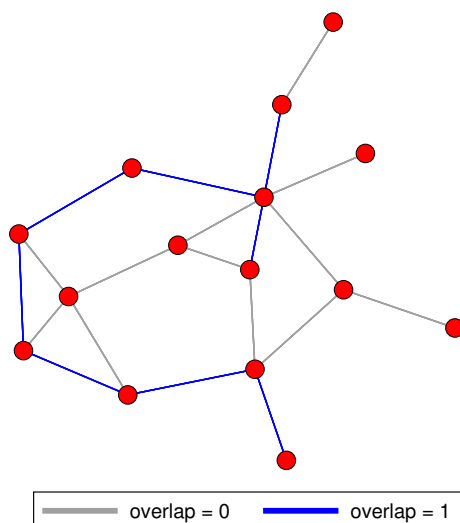


Figure 16.7: Florentine marriage network, edges colored by business network

comes with the software by default. I also `set seed 111`.¹ Next, I load the Florentine marriage data from internet (using `webnwuse florentine, nwclean`). Plotting the network "flomarriage" is as simple as `nwplot flomarriage`. To generate the plot in Figure 16.4, I also specified the option `lab` to display the *nodelabs* that are saved with the network. Notice that I could have also used the option `label(_nodelab)` to achieve the same result. The latter has the advantage that labels saved in any other variable can be used. Again, the entry in the variable `_nodelab[i]` is matched to the node with `nodeid = i`.

When looking at the Florentine dataset you find additional node-level attributes saved in normal Stata variables: `wealth`, `priorates`, `seat`. The variable `wealth` is each family's net wealth in 1427 (in thousands of lira) and `seat` indicates whether a Florentine family had a seat in the civic council between 1282-1344. We can use this information very easily to change the size and the color of the nodes in the network. In order to do so, I use the appropriate option together with a *varname*, where the *i*'th observation of the variable corresponds to the *i*'th node in the network. Let me plot the Florentine marriage network again, but in such a way that the size of each node represents the wealth of a family and the color of each node represents whether a family had a seat in the civic council or not (Figure 16.5). The command to do this is `nwplot flomarriage, size(wealth) color(seat)`. Notice that a legend is automatically added. There are plenty of options to control this legend as well. For

¹Obviously, multidimensional scaling has a random component, hence, the coordinates of nodes can differ.

example, I can force the legend keys for wealth to be 1 and 150 with the sub-option `forcekeys(1 150)`. Let me show you how easy it is to plot a sub-network. For example, `nwplot flomarriage if seat == 1, lab` only displays the families (and the ties between these families) who had a seat in the civic council (see Figure 16.6).

Now let me show you how useful *network expressions* can be for plotting networks. Let me give you an example, where I use a network expression to calculate the overlap of two networks and, then, use this overlap to color the edges in a network plot. The command `nwgen overlap = flomarriage * flobusiness` generates a new network "overlap", which takes the values 1 when two Florentine families have both marriage and business ties (see also Section 6.2). After that I can plot the network "flomarriage" with the option `edgecolor(overlap)`. Basically, I use the network "overlap" (that I just generated) to indicate the edge colors when plotting the network "flomarriage" (see Figure 16.7).

```
. set scheme s1network
. set seed 111
. webnwuse florentine, nwcLEAR
. nwplot flomarriage, lab

. set seed 111
. nwplot flomarriage, size(wealth, forcekeys(1 150)) color(seat)

. set seed 111
. nwgen overlap = flomarriage * flobusiness
. nwplot flomarriage, edgecolor(overlap)
```

16.3 Plot networks as sociomatrix

Another possibility is drawing a network as a sociomatrix with `nwplotmatrix` (see Figure 16.8). A sociomatrix visualizes the adjacency matrix of a network, where the rows indicate the sending nodes and the columns indicate the receiving nodes. The cells in such a plot are colored according to the tie values in the network. The command is used straightforwardly followed by a *netname*. Several options from `nwplot` are available as well. For example, one can display the node labels that are attached to a network with the option `lab`. Alternatively, any other variable can be used as node labels with `label(varname)` (see Table 16.3). This can be very useful, especially when sorting nodes according to another attribute. By default, the nodes are sorted according to their *nodeid*, i.e. the node with the lowest *nodeid* is displayed in the first row/column. An alternative is specifying option `sortby(varname)`. This sorts the nodes according to the values in *varname* in ascending order.

16.3.1 Example: Plot networks as sociomatrix

In this example, I also use the Florentine marriage network. First, I display the Florentine marriage network and display node labels with `nwplotmatrix flomarriage, lab` (see Figure 16.8). In the next plot (Figure 16.9), I simply add the option `scheme(s2network)`.

<i>label_options</i>	Description
<code>lab</code>	Display node labels
<code>label(varname)</code>	Display node labels from variable
<code>labelopt(marker_label_opt)</code>	Change look-and-feel of labels

Table 16.12: Label options for `nwplotmatrix`

<i>patch_options</i>	Description
<code>nodichotomize</code>	Do not dichotomize valued networks
<code>colorpalette(colorstyle ..)</code>	Display node labels from variable
<code>lcolor(colorstyle)</code>	Overwrite color of lines between patches
<code>background(colorstyle)</code>	Overwrite color of background for patches
<code>tievalue</code>	Show tie values as text inside patch
<code>tievalueopt(maker_label_opt)</code>	Change look of text insides patches

Table 16.13: Patch options for `nwplotmatrix`

This uses the default colors and properties outlined in the scheme "s2network". Notice that any other scheme can be used as well (e.g. "sj", "economist", "s2color").

In the next example, I demonstrate the option `sortby` (see Figure 16.10). As before, I type `nwplotmatrix` followed by the name of the network that I want to plot. But this time, I specify the option `sortby(wealth)`. This sorts the families in ascending order according to their wealth. The families with the lowest wealth are displayed in the top rows and the families with the highest wealth in the bottom rows. Before I plot the network, let me generate another variable `gen mylab1 = _nodelab + " (" + string(wealth) + ")"` and use this new variable as node labels. The full command for plotting is `nwplotmatrix flomarriage, label(mylab1) sortby(wealth)`. Now one can see that the families are indeed sorted by their wealth. In the previous example, the "Acciaiuli" family was displayed in the first row/column, now it is the "Pucci" family. This also shows how one can use all sorts of variables as node labels. Lastly, I demonstrate the option `group` (see Figure 16.11). Again, I generate a new variable for node labels. This time I combine the `nodelab` of nodes with the variable `seat`. The command is `gen mylab2 = _nodelab + " (" + string(seat) + ")"`. After that I plot the network with the option `group(seat)`. The nodes in Figure 16.11 are sorted according to the variable `seat`. But there are also horizontal and vertical lines to separate the groups from each other. Such a plot can be extremely useful to see whether network relationships are more (or less) likely between group members. In this case, it is very obvious that there are no marriage ties between two families, who both did not have a seat in the civic council. Notice that the options `sortby(varname)` and `group(varname)` can be combined with each other.

```
. webnwuse florentine, nwcLEAR
. nwplotmatrix flomarriage, lab
```

```

. nwplotmatrix flomarriage, lab scheme(s2network)

. gen mylab1 = _nodelab + " (" + string(wealth) + ")"
. nwplotmatrix flomarriage, label(mylab1) sortby(wealth)

. gen mylab2 = _nodelab + " (" + string(seat) + ")"
. nwplotmatrix flomarriage, label(mylab2) group(seat)

. nwplotmatrix flomarriage, background(sand) colorpalette(black)

. nwplotmatrix flomarriage, tievalue tievalueopt(mlabcolor(green))

. webnwuse gang, nwcLEAR
. nwplotmatrix gang_valued, nodichotomize legend(on)

```

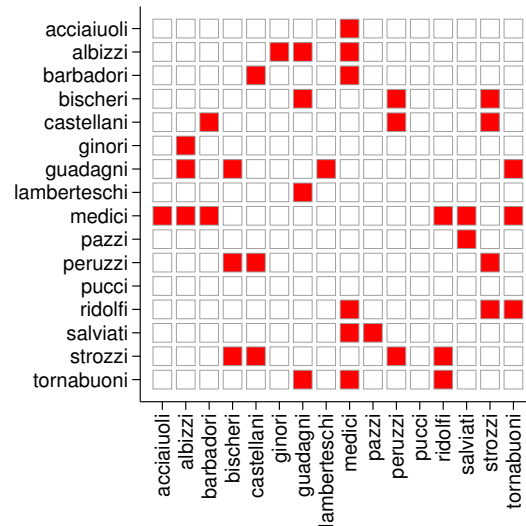


Figure 16.8: Florentine marriage network as matrix-plot

The colors for the patches are drawn from the current scheme. But all colors can be overwritten as well. Figure 16.12 shows a plot with alternative colors. The color of the cells with the lowest tie value is changed with with option `background(colorstyle)`. All other colors are changed with option `colorpalette(colorstyle1 colorstyle2 ...)`. For example, the second lowest tie value is assigned color `colorstyle1` and so on. Notice that in order to display more than two colors you also need to specify `nodichotomize`. Figure 16.12 is produced with the command `nwplotmatrix flomarriage, background(sand) colorpalette(black)`.

One interesting option is `tievalue`, which displays the actual tie values as numbers inside the patches. Figure 16.13 is plotted with `nwplotmatrix flomarriage,`

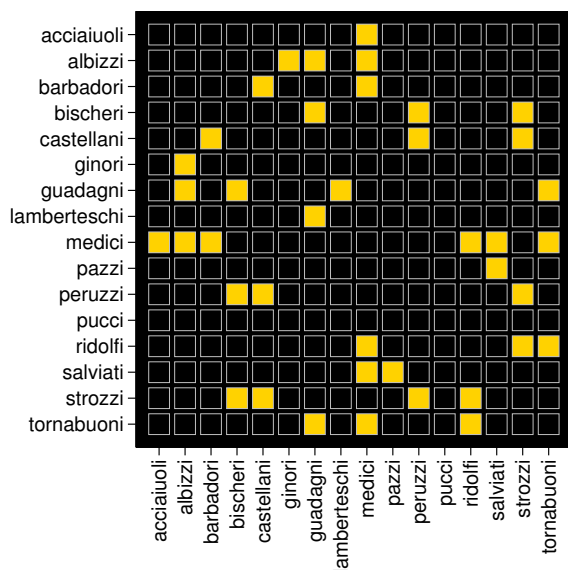


Figure 16.9: Florentine marriage network as matrix-plot, s2network scheme

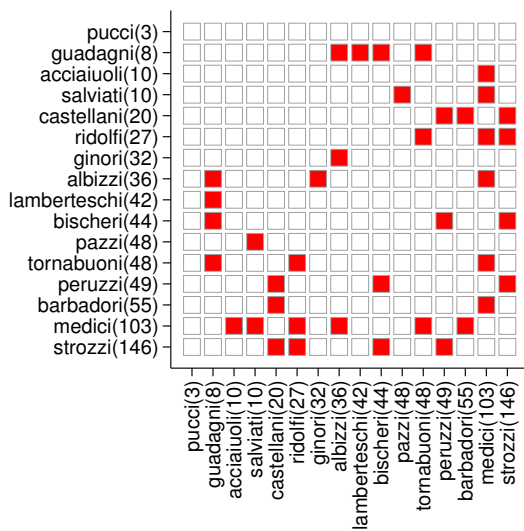


Figure 16.10: Florentine marriage network as matrix-plot, sorted by wealth

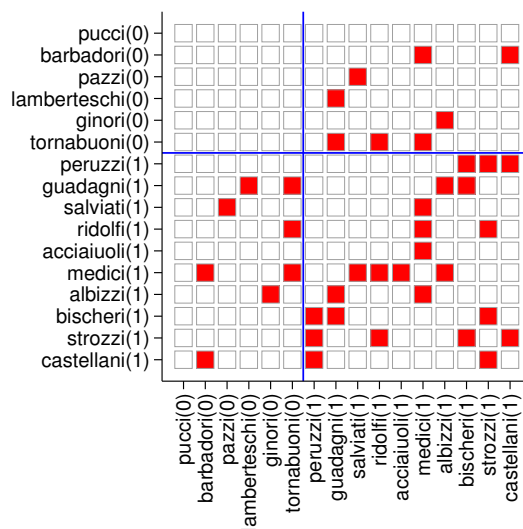


Figure 16.11: Florentine marriage network as matrix-plot, grouped by seat

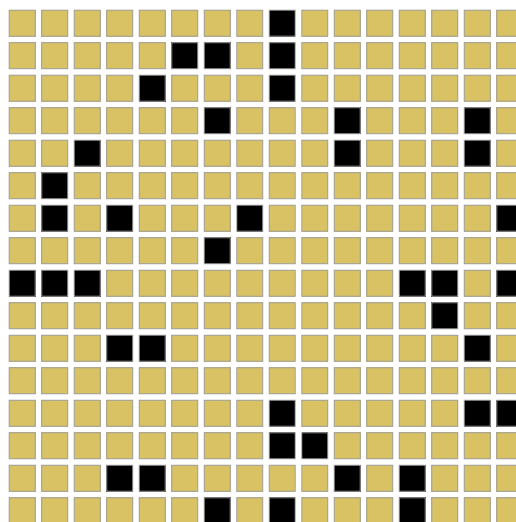


Figure 16.12: Florentine marriage network as matrix-plot, overwrite colors

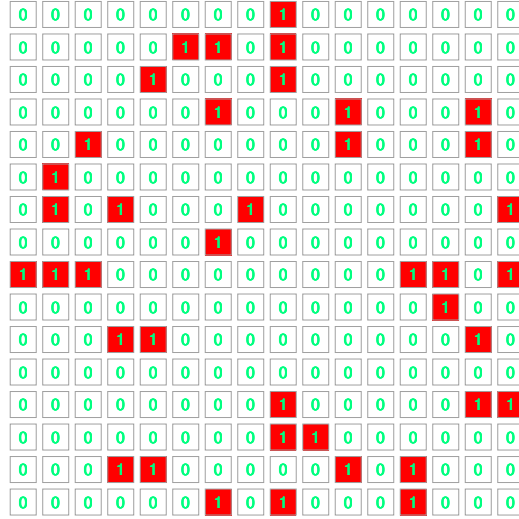


Figure 16.13: Florentine marriage network as matrix-plot, tievalue

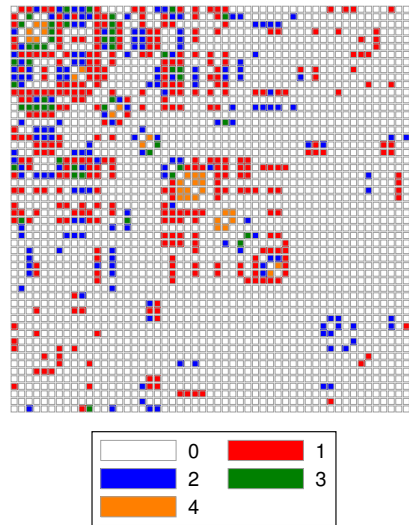


Figure 16.14: Gang network, valued

tievalue. These numbers are plotted using a normal scatter plot and can be formatted like all marker labels in scatter plots. One can communicate with the corresponding scatter plot through the option **tievalueopt(marker_label_opts)**. For example, option **tievalueopt(mlabcolor(mint))** overwrites the color in which the tie values are displayed.

By default, valued networks are displayed dichotomized. All non-zero tie values are plotted as tie value 1. Option **nodichotomize** does not dichotomize tie values and plots different tie values in different colors. For the next example, I load the dataset "gang" from the internet (**webnwuse gang, nwcLEAR**). This data holds information about the co-offending network of a youth gang. There are two networks in this dataset: "gang" and "gang_valued". The network "gang_valued" has different tie values for the intensity of a relationship. The command **nwplotmatrix gang_valued, nodichotomize legend(on)** produces the plot displayed in Figure 16.14.

16.4 Animate network

Another feature of the **nwcommands** is that evolving networks can be animated extremely easily. The analysis and visualization of network dynamics take a prominent role in social network analysis. A large stream of research attempts to understand the micro-level processes leading to the formation and dissolution of network ties. In that context, scholars are also interested in the change of actor attributes and want to find out whether existing network relationships promote the spread of node-level behaviors (e.g. individuals start smoking because their friends smoke) or whether similarity in node-level attributes affects tie formation (e.g. individuals become friends after meeting other smokers). Or sometimes, the simple growth of networks is important for the understanding of markets, organizations or groups. The animation of network dynamics is a very powerful and useful tool to get a better understanding of the processes behind network change. Most often one starts with x observations of the network G at different points in time $G_{t1}, G_{t2}, \dots, G_{tx}$. Such a list of network panel waves can then be animated by producing network plots for each network G_{ti} and attaching these plots in temporal sequence. Furthermore, a smooth transition between observations can be achieved by producing f additional frames between observations. Notice that even continuous network data can be conceptualized as panel data and visualized in such a way. All plots for the x observations, as well as the $(x - 1) * f$ additional frames between observations, can then be transformed into a movie with third-party video-editing software, such as e.g. Camtasia or FFmpeg.

Previous software for the animation of networks (e.g. R-package **networkDynamic**) are often cumbersome and require advanced programming knowledge. The **nwcommands** come with the **nwmovie** command, which provide an extremely simple engine for making network movies. In contrast, to earlier attempts to bring animations into Stata, **nwmovie** does not use video-editing software (such as FFmpeg), but instead the image editing software ImageMagick. ImageMagick is a free and open-source command-line software suite for image editing and converting. Originally, developed for Unix-environments, it is now available for Windows and MacOSX. When the **nwmovie** command is called for the first time, the user is prompted with the appropriate download link and instructions

for the installation of ImageMagick. After successful installation of ImageMagick (and having some networks in Stata memory), making a network movie can be as easy as this:

The command `nwmovie` accepts a list of networks and ultimately produces an Animated GIF file for the sequence of networks that can be directly inserted into presentations (e.g. Powerpoint) or websites (e.g. <http://www.liu.se/ias>). In more detail, `nwmovie` draws on `nwplot` to make network plots for each network in a sequence. The order of the networks in the list is interpreted as the temporal ordering of network panel waves. Next, `nwmovie` calculates the position and size of the network nodes in between these network panel waves for the desired number of frames. Lastly, both the original plots for the networks and the plots for the frames are combined in one Animated GIF file.

Chapter 17

Advanced programming with networks

17.1 Getting the adjacency matrix

Writing own network programs is very easy. A lot of the *nwcommands* are implemented in Mata, Stata's internal matrix language. Especially when it comes to networks, it makes a lot of sense to use Mata because of its matrix calculation capabilities. Networks are ultimately saved as matrices in Mata as well. You do not need to know Mata to use any of the *nwcommands*, but sometimes you might want to obtain the adjacency matrix, for example, when programming your own network commands.

There are two ways to obtain the adjacency matrix of a network. The first one is with `nwtomata` and the second one is with `nwtomatafast`. I recommend using the former command, and in fact, almost all *nwcommands* make use of it as well. `nwtomata` returns a copy of the adjacency matrix of a network. The command is used together with a *netname* and with the option `mat(newmatamatrix)`. It returns the adjacency matrix of the network *netname* in the Mata matrix *newmatamatrix*. For example, let me generate a random network with 10 nodes and call this network "myrandom". After that, I use `nwtomata myrandom, mat(myrandommat)` to get a copy of the adjacency matrix in Mata. You can see that the adjacency matrix is now available in Mata. With `mata: myrandommat` I simply display this matrix. Now, you can do whatever you want with this adjacency matrix in Mata. But notice, that you are only interacting with a copy of the adjacency matrix. This has several advantages, .e.g. you will not change the underlying network in ways that are inconsistent with its meta-information. At the same time, this also requires a little more memory because Stata has to generate a copy of the adjacency matrix first. In practice, however, this is never really a problem (unless you work with very large networks).¹

¹In its current implementation, this strategy to copy the adjacency matrix and only work with such copies is a limiting factor for some commands. Some *nwcommands* call `nwtomata` internally several times.

```
. set seed 123
. nwrando 10, prob(.1) name(myrandom)
. nwtomata myrandom, mat(myrandommat)
. mata: myrandommat
```

	1	2	3	4	5	6	7	8	9	10
1	0	0	0	0	0	0	0	0	0	0
2	0	0	1	0	0	0	0	0	1	1
3	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	1	0	0	0
7	0	1	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	0
9	1	0	1	0	0	0	0	0	0	0
10	0	0	0	1	0	0	0	0	0	0

Alternatively, one can also interact with the adjacency matrix of a network directly with `nwtomatafast`. But I only recommend this for advanced users. The reason is that you can accidentally change the adjacency matrix of your network without updating the relevant meta-information. When you interact with the network through the other interfaces that the *nwcommands* provide this cannot happen. Furthermore, when using the interfaces you will not be affected by future updates of the internal structure of the *nwcommands*, e.g. how networks are saved. In the next example, you can see how `nwtomatafast` `myrandom` populates the return vector with the name of a Mata matrix. In this case, the network "myrandom" is internally saved in the Mata matrix "nw_mata1".

```
. nwtomatafast myrandom
. return list
macros:
      r(mata) : "nw_mata1"
. mata: `r(mata)'
```

	1	2	3	4	5	6	7	8	9	10
1	0	0	0	0	0	0	0	0	0	0
2	0	0	1	0	0	0	0	0	1	1
3	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	1	0	0	0
7	0	1	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	0
9	1	0	1	0	0	0	0	0	0	0
10	0	0	0	1	0	0	0	0	0	0

17.1.1 Syntax for obtaining adjacency matrix

```
nwtomata [netname], mat(newmatamatrix)
```

17.1.2 Options for obtaining adjacency matrix

`mata(newmatamatrix)` specifies the name of a new Mata matrix where a copy of the adjacency matrix of a network is stored.

17.2 Unabbreviate network

One key feature of the *nwcommands* is that networks can be treated almost as if they were normal Stata variables. That also means they can be abbreviated just like normal Stata variables (see Table `tab:netlist`). For example, `net*` refers to all networks beginning with "net". In addition, there is the reserved word `_all`, which refers to all networks in memory. In Stata one can explicitly unabbreviate variable lists with `unab`. This returns the long-form of a variable list. Networks can be explicitly unabbreviated with `nwunab`. This command works almost exactly like its native Stata counterpart and transforms a short-form of a list of networks to the long-form explicitly, where one word corresponds to one network.

17.2.1 Example: Unabbreviate networks

The command is called followed by a local macro *lmacname*. This is where the unabbreviated list of networks will be stored. After that comes a colon and the network list that should be unabbreviated. For example, the command `nwunab mynets: _all` unabbreviates the network list `_all` and saves the result in the local macro "mynets".

```
. webnwuse florentine, nwclear
Loading successful

(2 networks)
-----
      flobusiness
      flomarriage
. nwunab mynets : _all
. di "`mynets'"
flobusiness flomarriage
```

17.2.2 Syntax for unabbreviating networks

`nwunab lmacname: netlist [ , min(integer) max(integer) ]`

17.2.3 Options for unabbreviating networks

`min(integer)` minimum number of networks.

`max(integer)` maximum number of networks.

17.3 Parsing networks

Another command that is extremely useful when programming own network commands is `_nwsyntax`. It works almost like the normal `syntax` command, but parses networks. Such parsing does three things, 1) it unabbreviates network lists, 2) it checks if networks actually exist, 3) it checks if a network list has a certain number of networks. Almost all *nwcommands* parse networks in this way.

17.3.1 Syntax for parsing networks

```
_nwsyntax anything [ , min(integer) max(integer) networks(local_macro1)
  name(local_macro2) id(local_macro3) directed(local_macro4)
  nodes(local_macro5) ]
```

17.3.2 Options for parsing networks

`min(integer)` minimum number of networks in unabbreviated *anything*.

`max(integer)` maximum number of networks in unabbreviated *anything*.

`networks(local_macro1)` saves the number of networks in *local_macro1*; default = networks.

`name(local_macro2)` saves the unabbreviated names of all networks *anything* in the macro *local_macro2*; default = netname.

`id(local_macro3)` saves the *networkid* of the last network in *anything* in the macro *local_macro3*.

`directed(local_macro4)` saves the directionality of the last network in *anything* in the macro *local_macro4*.

`nodes(local_macro5)` saves the number nodes of the last network in *anything* in the macro *local_macro5*.

17.4 Example: Own network program

Let us write our own program `myindegree` to calculate indegree centrality (see Section 14.1) of a network *netname*. Let us program it in such a way that it can be called with a *netname* as an argument and an option `generate(newvarname)` to save the result.

The logic of the little program outlined in Figure ?? is that after normal use of `syntax`, `_nwsyntax` is called to check if *anything* is a valid network and (if necessary) unabbreviates it (this is very similar to `syntax`). By default, `_nwsyntax` only allows one network in *anything*, which is just what we want. When used with the defaults, `_nwsyntax` also leaves a local macro *netname* behind which holds the unabbreviated network name. Next, in `nwtomata` we refer to this *netname* and obtain the adjacency matrix of this network and save it in the Mata matrix *net*. The next three lines assign a default variable name in case option `generate(newvarname)` is unspecified. The actual indegree of the network is calculated straightforwardly with `mata: indegree =`


```

capture program drop myindegree
program myindegree
    syntax [anything(name=netname)] , [ generate(string)]
    _nwsyntax `netname'
    nwtomata `netname', mat(net)

    if ("`generate'" == "") {
        local generate = "_myindegree"
    }

    mata: indegree = colsum(net!=0)'
    getmata `generate' = indegree, force
    mata: mata drop indegree net
end

```

Figure 17.1: Own network program

`colsum(net!=0)'`.² Lastly, the result, which is stored in the Mata matrix `indegree`, is brought back to Stata with `getmata`. One would call this program using the Florentine data as follows. It generates the Stata variable `floindeg`.

```

. webnwuse florentine, nwcLEAR
. myindegree flomarriage, generate(floindeg)

```

17.5 Example: Another network program

Let us write another simple program which takes an existing network as an argument, inverts the adjacency matrix and saves the result as a new network. This is exactly what `nwtranspose` does. It basically transforms incoming ties into outgoing ties and vice-versa.

As you can see, the code of the program (see Figure 17.2 is very similar to the one above. The first few lines are almost identical. Again, we perform some checks and obtain the adjacency matrix of the network as Mata matrix `net`. The line `mata: inverse = net'` simply inverts the adjacency matrix of the network and saves it as `inverse`. Afterwards, all we have to do is `nwset` a new network using the option `mat(inverse)`.³ Now we can generate the network "floinv" as the inverse of "flomarriage" like this:

```

. webnwuse florentine, nwcLEAR
. myinverse flomarriage, name(floinv)

```

²Notice that this also takes care of valued networks.

³Notice that I could have also inverted the network on-the-fly with `nwset, mat(net')` `name("`generate'")`.

```
capture program drop myinverse
program myinverse
    syntax [anything(name=netname)] , [ generate(string)]
    _nwsyntax `netname'
    nwtomata `netname', mat(net)

    if ("`generate'" == "") {
        local generate = "_myinverse"
    }

    mata: inverse = net'
    nwset, mat(inverse) name("`generate'")
    mata: mata drop inverse net
end
```

Figure 17.2: Another network program

References

- Barabasi, A.-L., and R. Albert. 1999. Emergence of Scaling in Random Networks. *Science* 286(5439): 509–512. URL <http://www.sciencemag.org/cgi/content/abstract/286/5439/509>.
- Barrat, A., and M. Weigt. 2000. On the properties of small-world network models. *The European Physical Journal B - Condensed Matter and Complex Systems* 13(3): 547–560. URL <http://dx.doi.org/10.1007/s100510050067>.
- Bastian, M., S. Heymann, and M. Jacomy. 2009. Gephi: An Open Source Software for Exploring and Manipulating Networks. URL <http://www.aaai.org/ocs/index.php/ICWSM/09/paper/view/154>.
- Batagelj, V., and A. Mrvar. 1998. Pajek - Program for Large Network Analysis. *Connections* 2: 47–57.
- Bavelas, A. 1948. A Mathematical Model of Group Structure. *Human Organizations* 7: 16–30.
- Borgatti, S. P., M. G. Everett, and L. C. Freeman. 2002. UCINET 6 For Windows: Software for Social Network Analysis. URL <http://www.analytictech.com/>.
- Bossart, G., and N. Meidert. 2013. We are only as strong as we are united, as weak as we are divided'. A dynamic analysis of the peer support networks in the Harry Potter books. *Open Journal of Applied Sciences* 3(2): 174–185.
- Brandes, U. 2001. A Faster Algorithm for Betweenness Centrality. *Journal of Mathematical Sociology* 25(2): 163–177.
- . 2008. On variants of shortest-path betweenness centrality and their generic computation. *Social Networks* 30: 136–145.
- Butts, C. T. 2008a. network: A Package for Managing Relational Data in R. *Journal of Statistical Software* 24(2): 1–36. URL <http://www.jstatsoft.org/v24/i02>.
- . 2008b. Social Network Analysis with sna. *Journal of Statistical Software* 24(6): 1–51. URL <http://www.jstatsoft.org/v24/i06>.
- . 2014. *sna: Tools for Social Network Analysis*. R package version 2.3-2. URL <http://CRAN.R-project.org/package=sna>.

- Butts, C. T., M. S. Handcock, and D. R. Hunter. 2014. *network: Classes for Relational Data*. Irvine, CA. R package version 1.10.2. URL <http://statnet.org/>.
- Corten, R. 2010. PAJEK2STATA: Stata module to import network data in Pajek's .net format. Statistical Software Components, Department of Economics, Bost College. URL <http://econpapers.repec.org/software/bocbocode/s457149.htm>.
- . 2011. Visualization of social networks in Stata using multidimensional scaling. *Stata Journal* 11(1): 52–63.
- Csardi, G., and T. Nepusz. 2006. The igraph software package for complex network research. *InterJournal Complex Systems*: 1695. URL <http://igraph.sf.net>.
- Davis, J. A., and S. Leinhardt. 1972. The Structure of Positive Interpersonal Relations in Small Groups. In *Sociological Theories in Progress*, ed. J. Berger, vol. 2, 218–251. Houghton Mifflin.
- Erdős, P., and A. Rényi. 1959. On random graphs, I. *Publicationes Mathematicae (Debrecen)* 6: 290–297. URL http://www.renyi.hu/~p_erdos/Erdos.html#1959-11.
- Freeman, L. C. 1977. A set of measures of centrality based on betweenness. *Sociometry* 40: 35–41.
- . 1978. Centrality in social networks conceptual clarification. *Social Networks* 215.
- . 1979. Centrality in Social Networks: Conceptual Clarification. *Social Networks* 1(215–239).
- Gilbert, E. N. 1959. Random Graphs. *Ann. Math. Statist.* 30(4): 1141–1144. URL <http://dx.doi.org/10.1214/aoms/1177706098>.
- Grund, T. U., and J. A. Densley. 2012. Ethnic heterogeneity in the activity and structure of a Black street gang. *European Journal of Criminology* 9(4): 388–406. URL <http://euc.sagepub.com/content/9/4/388.abstract>.
- . 2014. Ethnic Homophily and Triad Closure: Mapping Internal Gang Structure Using Exponential Random Graph Models. *Journal of Contemporary Criminal Justice*. URL <http://ccj.sagepub.com/content/early/2014/10/08/1043986214553377.abstract>.
- Hagberg, A. A., D. A. Schult, and P. J. Swart. 2008. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy2008)*, 11–15. Pasadena, CA USA.
- Handcock, M. S., D. R. Hunter, C. T. Butts, S. M. Goodreau, and M. Morris. 2003. *statnet: Software tools for the Statistical Modeling of Network Data*. Seattle, WA. URL <http://statnetproject.org>.

- Hansen, D., B. Shneiderman, and M. A. Smith. 2010. *Analyzing Social Media Networks with NodeXL: Insights from a Connected World*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc.
- Holland, P. W., and S. Leinhardt. 1970. A Method for Detecting Structure in Sociometric Data. *American Journal of Sociology* 76(3): pp. 492–513. URL <http://www.jstor.org/stable/2775735>.
- Krackhardt, D., and R. N. Stern. 1988. Informal Networks and Organizational Crises: An Experimental Simulation. *Social Psychology Quarterly* 51(2): 123–140.
- Miura, H. 2012. Stata graph library for network analysis. *Stata Journal* 12(1): 94–129.
- Neal, Z. P. 2012. CENTPOW: Stata module to calculate centrality and power indices for each node in a network. URL <http://EconPapers.repec.org/RePEc:boc:bocode:s457186>.
- de Nooy, W., A. Mrvar, and V. Batagelji. 2005. *Exploratory Social Network Analysis with Pajek*. Cambridge University Press.
- Padgett, J. F., and C. K. Ansell. 1993. Robust Action and the Rise of the Medici, 1400-1434. *American Journal of Sociology* 98(6): pp. 1259–1319. URL <http://www.jstor.org/stable/2781822>.
- Pearson, M., and P. West. 2003. Drifting Smoke Rings: Social Network Analysis and Markov Processes in a Longitudinal Study of Friendship Groups and Risk-Taking. *Connections* 25(2): 59–76.
- Rossman, G. 2009. STATA2PAJEK: Stata module to export data to Pajek .net format. Statistical Software Components, Boston College Department of Economics. URL <http://ideas.repec.org/c/boc/bocode/s457075.html>.
- Sabidussi, G. 1966. The Centrality Index of a Graph. *Psychometrika* 31(4): 581–603.
- Snijders, T. A., G. G. van de Bunt, and C. E. Steglich. 2010. Introduction to stochastic actor-based models for network dynamics. *Social Networks* 32(1): 44 – 60. Dynamics of Social Networks. URL <http://www.sciencedirect.com/science/article/pii/S0378873309000069>.
- Staudt, C. L., A. Sazonovs, and H. Meyerhenke. 2014. NetworKit: An Interactive Tool Suite for High-Performance Network Analysis. *arXiv preprint arXiv:1403.3005*.
- Travers, J., and S. Milgram. 1969. An Experimental Study of the Small World Problem. *Sociometry* 32(4): pp. 425–443. URL <http://www.jstor.org/stable/2786545>.
- Watts, D. J. 1999. Networks, Dynamics, and the SmallWorld Phenomenon. *American Journal of Sociology* 105(2): pp. 493–527. URL <http://www.jstor.org/stable/10.1086/210318>.
- Watts, D. J., and S. H. Strogatz. 1998. Collective dynamics of ‘small-world’ networks. *Nature* 393: 440–442.

Index

`_nwsyntax`, 14

adjacency list, 11

adjacency matrix, 11

`centpow`, 13

`collapse`, 46

edge list, 11

`mdsmat`, 14

`netlist`, 18

`netname`, 18

`netplot` , 13

`netsis`, 13

network expression, 20, 62

network functions, 64

network generator, 137

network programming, 133

network subsetting, 47

`nodeid`, 19, 44

`odelab`, 19

`nw`, 14

`nwaddnodes`, 46

`nwclear`, 14, 45

`nwcloseness`, 109

`nwclustering`, 111, 113

`nwcollapse`, 14, 46

`nwcontext`, 93

`nwcorrelate`, 14

`nwdegree`, 103, 136

`nwdrop`, 14, 43

`nwdropnodes`, 44

`nwdyadprob`, 60

`nwdyads`, 40, 53

`nwevcent`, 107

`nwexpand`, 14, 67

`nwexport`, 30

`nwgen`, 14, 20

`nwgenerate`, 14

`nwgeodesic`, 101

`nwimport`, 26

`nwininstall`, 15

`nwkeep`, 14, 43

`nwkeepnodes`, 44

`nwlattice`, 55

`nwload`, 32

`nwmovie`, 131

`nwname`, 36

`nwnneighbor`, 93

`nwpath`, 99

`nwpermute`, 69

`nwplot`, 14, 117

`nwrandom`, 52, 74

`nwrecode`, 14, 50

`nwreplace`, 14, 20, 47

`nwreplacemat`, 47

`nwring`, 54

`nwsave`, 14, 31

`nwset`, 14, 23

`nwsmall`, 56

`nwsubset`, 44

`nwsummarize`, 14, 39, 74

`nwsym`, 36

`nwtabulate`, 14, 77

`nwtoedge`, 83

`nwtomata`, 18, 133

`nwtomatafast`, 18, 134

`nwtranspose`, 137

`nwtriads`, 41

`nwunab`, 14, 135

`nwuse`, 14, 31

INDEX

143

nwvalue, [34](#)

pajek2stata, [13](#)

stata2pajek, [13](#)

webnwuse, [14](#), [31](#)